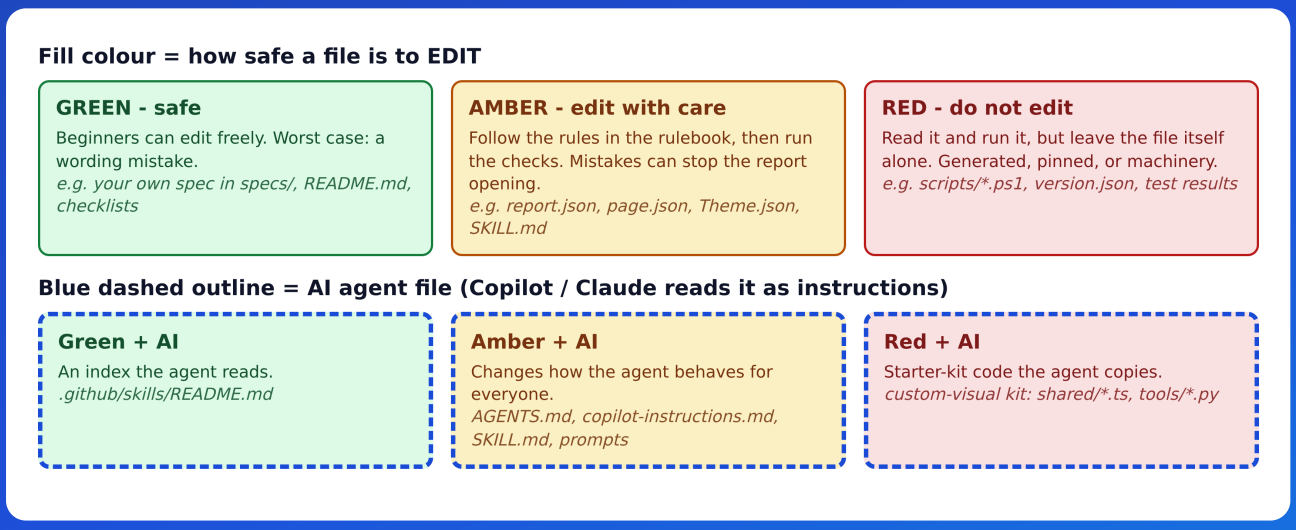
A VISUAL LEARNING GUIDE

PGE-PBIP Masterclass

Power BI projects, Git, VS Code and AI agents, explained one picture at a time, using the GibbonGambits/PGE-PBIP-Template repository.

20 sections | 176 files and folders covered | for absolute beginners



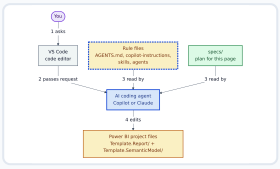
Colour legend shown above is used in every diagram.
Facts checked against official documentation on 10 November 2026.

Contents

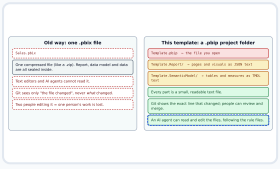
Part 1 - Foundations



00
Welcome, how to use this guide, and the colour legend
page 4



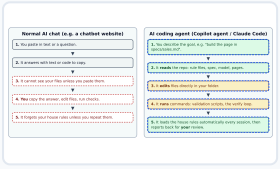
01
The big picture: what this repository is, and who it is for
page 9



02
Power BI and the Power BI Project format (.pbip)
page 15



03
Git, GitHub, and why we fork
page 24



04
VS Code, and what an AI coding agent does compared with a normal chat
page 31

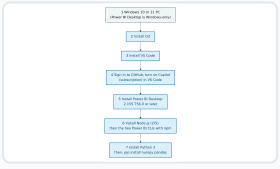
Part 2 - How it fits



05
The master map: how every part connects, and in what order it runs
page 37



06
Rule layers: steering, specs, hooks, skills, and "who wins"
page 48



07
Local setup checklist
page 56

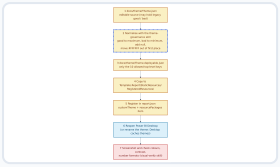
Part 3 - Doing the work



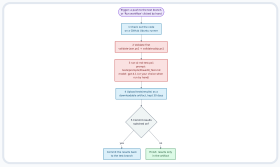
08
Building a dashboard page from start to finish
page 65



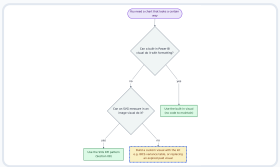
09
Checks and the screenshot (verify) loop
page 75



10
The theme pipeline: from Theme.json to the live report
page 85



11
Testing and automation: AI tests, GitHub Actions and branches
page 89



12
The custom-visual (.pbiviz) starter kit
page 96

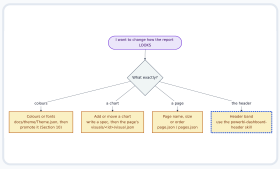


13
The Python email-report and alert guides
page 106

Part 4 - Look it up



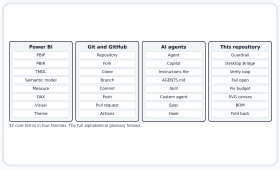
14
File-by-file directory
page 111



15
"Which file do I touch?" decision trees
page 151



16
Gaps found in the repository, and how to fix them
page 155



17
Beginner glossary
page 162



18
Verified official references
page 167

19
Appendix: the Stage 1 inventory table
page 170

00. Welcome, how to use this guide, and the colour legend

Fill colour = how safe a file is to EDIT

GREEN - safe

Beginners can edit freely. Worst case: a wording mistake.
e.g. your own spec in specs/, README.md, checklists

AMBER - edit with care

Follow the rules in the rulebook, then run the checks. Mistakes can stop the report opening.
e.g. report.json, page.json, Theme.json, SKILL.md

RED - do not edit

Read it and run it, but leave the file itself alone. Generated, pinned, or machinery.
e.g. scripts/.ps1, version.json, test results*

Blue dashed outline = AI agent file (Copilot / Claude reads it as instructions)

Green + AI

An index the agent reads.
.github/skills/README.md

Amber + AI

Changes how the agent behaves for everyone.
AGENTS.md, copilot-instructions.md, SKILL.md, prompts

Red + AI

Starter-kit code the agent copies.
custom-visual kit: shared/.ts, tools/*.py*

Figure 0.1 — The colour legend. Every diagram in this guide uses these colours. Learn them once and every picture will tell you, at a glance, what you may safely touch.

Walkthrough of Figure 0.1:

1. **Green boxes** are safe. You can edit these files while you are still learning. The worst you can do is make a sentence unclear.
2. **Amber boxes** need care. They control how the report looks or how the AI behaves. Edit them by following the repository's rules, then run the checks described in Section 09.
3. **Red boxes** are "look, don't touch". They are scripts, generated results, or files with exact required contents. You will often *run* or *read* them, but you should not change them.
4. **A blue dashed outline** means "AI agent file". An AI coding assistant (GitHub Copilot or Claude) reads it as instructions. A blue outline is always combined with a fill colour, so you also know whether it is safe to edit.

5. In diagrams that show actions rather than files, you will also see purple boxes for **you** (the human), light-blue boxes for **outside services** such as GitHub, and grey boxes for neutral steps.

Who this guide is for

You are learning four things at once: **Power BI** (Microsoft's dashboard tool), **Git** (a tool that saves the history of your files), **VS Code** (a free code editor), and **AI coding agents** (AI assistants that can read and change files for you). That is a lot. This guide takes it one picture at a time.

You do not need to know how to program. You need to be willing to follow numbered steps and to look closely at diagrams.

What this guide covers

This guide explains one specific repository: **GibbonGambits/PGE-PBIP-Template**. A *repository* (or "repo") is a project folder whose full history is tracked by Git. Think of it as a shared binder that remembers every page ever added, changed or removed.

Every one of its 176 files and folders is covered. Nothing was guessed: each file was opened and read.

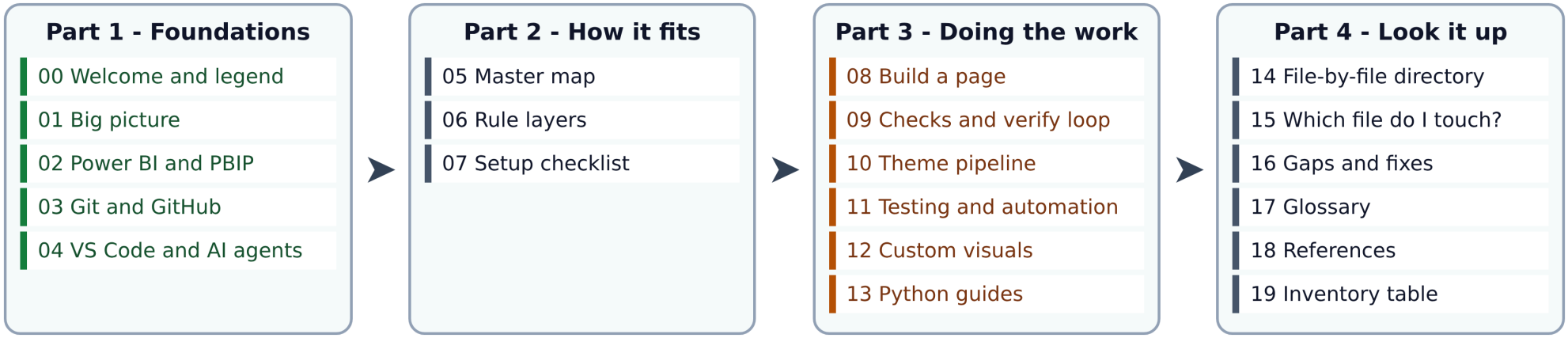


Figure 0.2 — Road map of this guide. The guide has four parts. Read Parts 1 and 2 in order. Dip into Parts 3 and 4 when you need them.

Walkthrough of Figure 0.2:

1. **Part 1 — Foundations (Sections 00–04).** Plain-English introductions to the repository, Power BI's project format, Git and GitHub, VS Code, and AI agents. Start here if any of those words are new.
2. **Part 2 — How it fits (Sections 05–07).** The master map of how every part connects, the "who wins" rules for AI instructions, and a setup checklist for your own computer.
3. **Part 3 — Doing the work (Sections 08–13).** Step-by-step walkthroughs: building a dashboard page, running the checks, changing the theme, testing, custom visuals, and the Python email guides.
4. **Part 4 — Look it up (Sections 14–19).** A file-by-file directory, "which file do I touch?" decision trees, known gaps in the repository and how to fix them, a glossary, verified official links, and the complete inventory table.

How each section is laid out

- **Picture first.** Each section opens with a diagram. Read the picture, then the words.
- **Numbered captions.** “Figure 8.2” means the second diagram in Section 08.
- **Numbered walkthroughs.** The numbers under a diagram match the numbered steps or boxes in it.
- **Terms in bold** are defined the first time they appear and again in the Glossary (Section 17).
- **Commands** appear in grey code boxes. You type them into a terminal (a text window for typing commands). Most commands in this repository are for **Windows PowerShell**, because Power BI Desktop only runs on Windows.

Two safety promises

1. **Secrets stay secret.** This repository contains a file called `.env.example` that lists *names* of passwords (for example `DB_PASSWORD`) but no real values. This guide never prints real values, and Section 07 shows you how to create your own private `.env` file without exposing it.
2. **Read before you change.** If a file is Amber or Red, read its entry in Section 14 before touching it.

A note on facts and dates

Facts about Power BI, Git, GitHub, VS Code and Claude were checked against the official documentation pages listed in Section 18, opened on 10 November 2026. Software changes quickly. If a menu or button has moved, the official page is the place to check.

01. The big picture: what this repository is, and who it is for

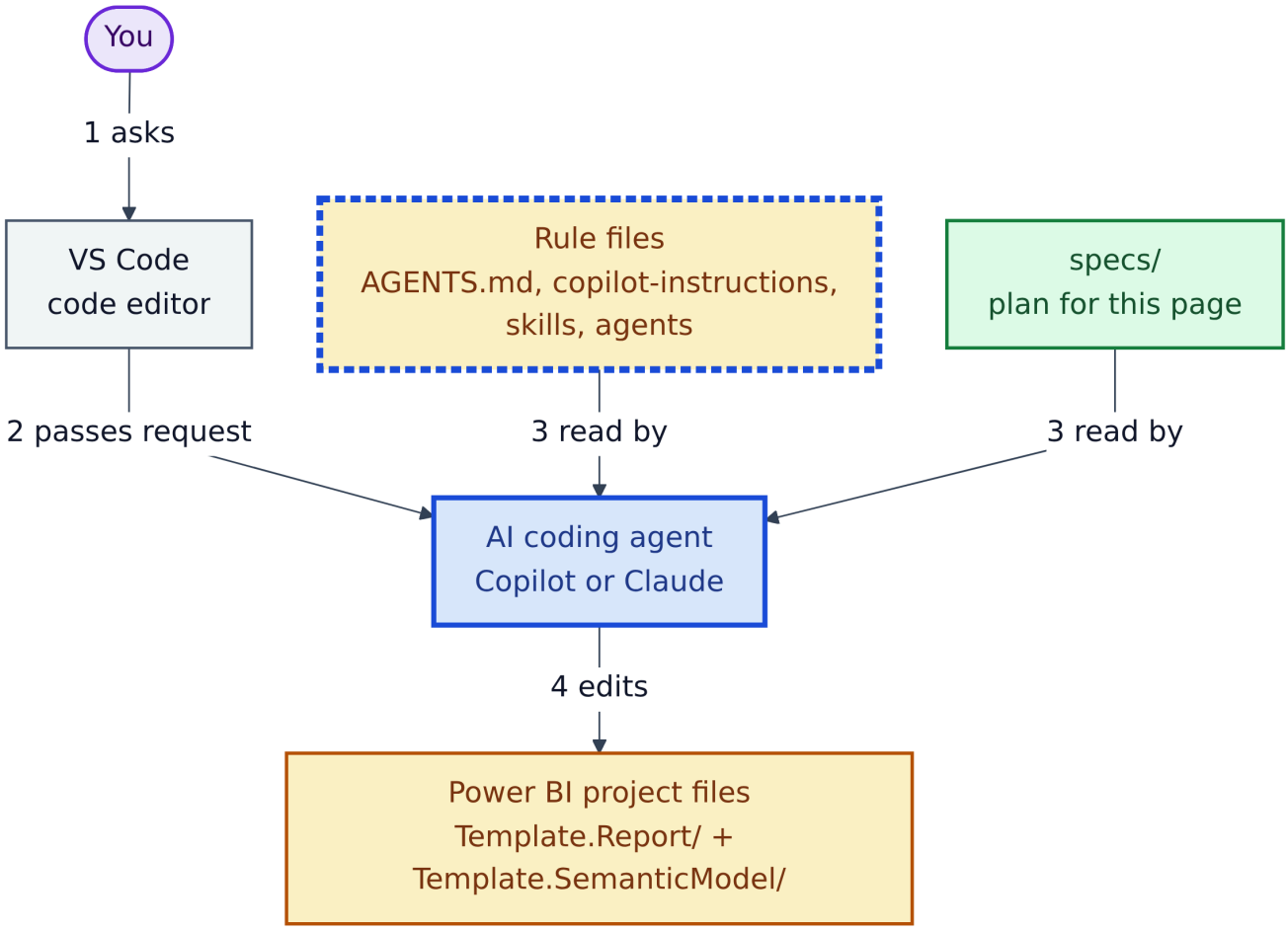


Figure 1.1 — The system, part 1: making a change. Arrows 1–4 show how your idea becomes edited project files.

Walkthrough of Figure 1.1:

1. **You ask for a change** in VS Code, for example “add a sales page”. VS Code is a free code editor: a word processor built for code and configuration files.
2. **VS Code passes the request** to an **AI coding agent**: GitHub Copilot or Claude. An agent is an AI that can read files, write files and run commands, not just chat.
3. **The agent reads the rule files and the spec.** Rule files (Amber, blue outline) tell it *how* this team always builds. The spec (Green) says *what* to build this time.
4. **The agent edits the Power BI project files.** These are plain text files that describe the report and its data model.

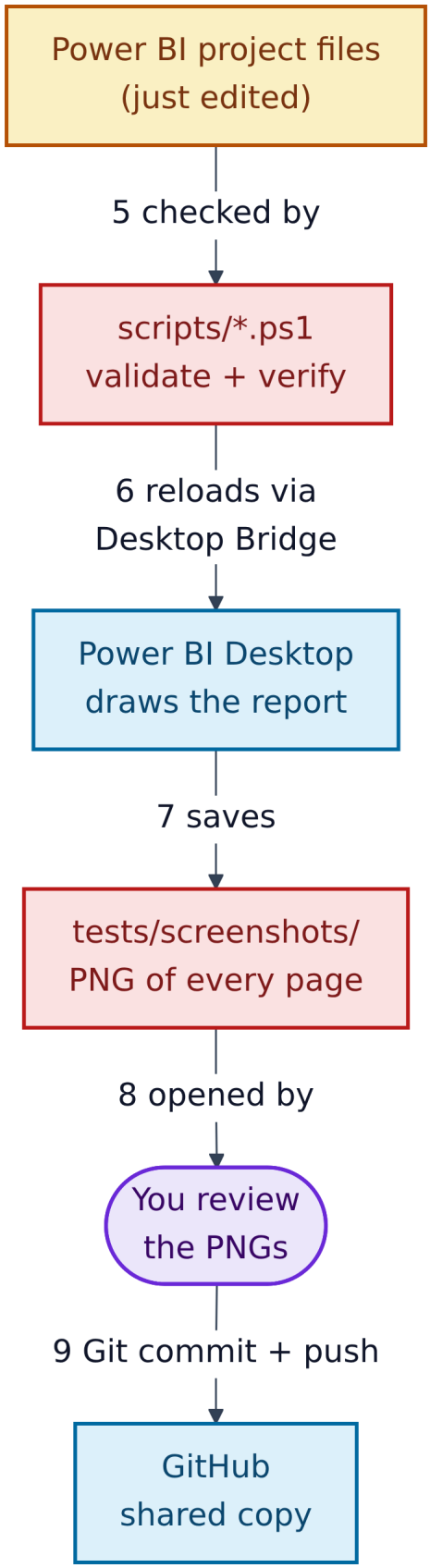


Figure 1.2 — The system, part 2: checking and sharing. Arrows 5–9 continue from Figure 1.1, top to bottom.

Walkthrough of Figure 1.2:

5. **Scripts check the files.** PowerShell scripts in `scripts/` confirm the files are well formed before Power BI ever sees them.
6. **The verify loop reloads Power BI Desktop** through the **Desktop Bridge**, a local connection that lets programs control a running copy of Power BI Desktop.
7. **Power BI Desktop saves a screenshot** of every page into `tests/screenshots/`.
8. **You (or the agent) open and review the screenshots.** The repository's core rule: a page is not done because its files are valid; it is done when what Power BI *drew* looks right. If something is wrong, go back to Figure 1.1, step 1.
9. **You save the work with Git and push it to GitHub**, the online home of the shared copy. GitHub can also run automated tests ("Actions").

What the repository is, in one paragraph

PGE-PBIP-Template is a *starter kit* for building Power BI dashboards with an AI assistant. A dashboard is a page of charts and numbers that helps people make decisions. Normally a Power BI dashboard is stored as one sealed `.pbix` file. This template uses Power BI's newer **project format (PBIP)**, which saves the dashboard as many small, readable text files. Readable files mean Git can track every change and an AI agent can edit them. Around those files the template adds written rules, step-by-step "skills" for the AI, quality checklists, and scripts that check the result.

Real-world comparison: think of a restaurant franchise kit. It contains an empty kitchen (the blank report and model), the house recipe book (the rule files), recipe cards for specific dishes (skills), food-safety checklists (docs/checklists), and a quality inspector who tastes every plate before it leaves the kitchen (the screenshot loop).

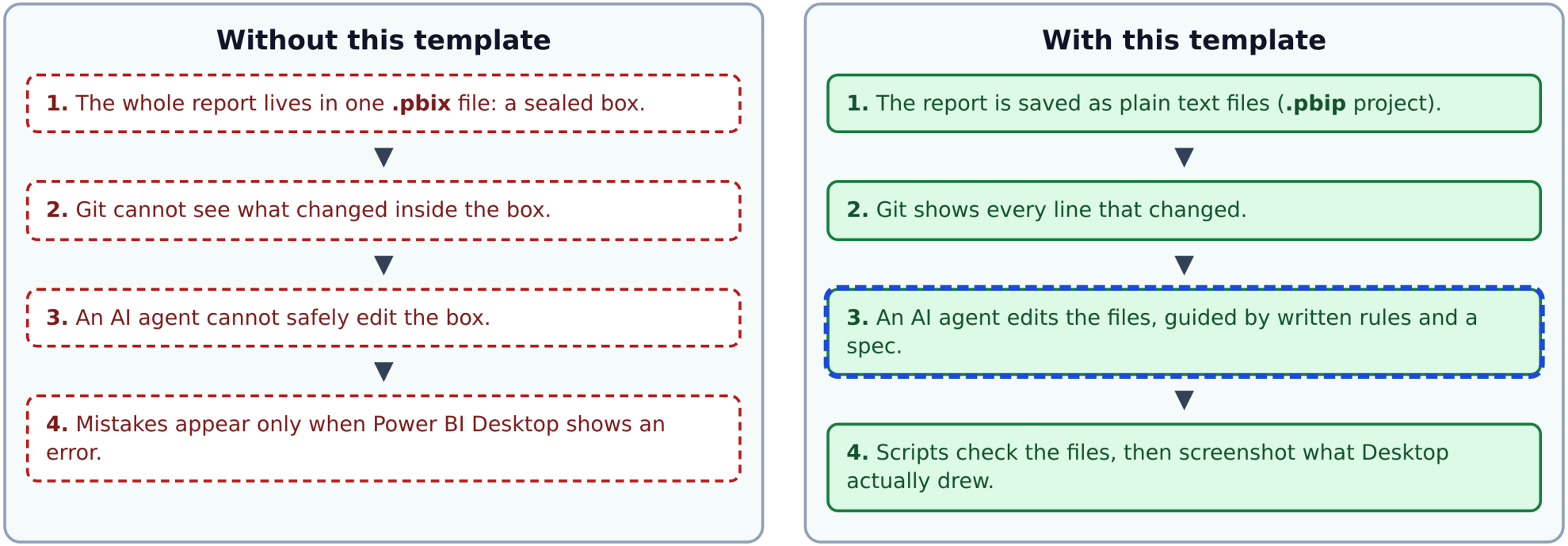


Figure 1.3 — The problem this template solves. Left: the traditional single-file approach. Right: the text-file approach this template uses.

Walkthrough of Figure 1.3:

1. **Sealed box vs. plain text.** A `.pbix` file is a compressed bundle you cannot read in a text editor. A `.pbip` project is a folder of readable files.
2. **Invisible vs. visible changes.** Git can only tell you "the box changed". With text files, Git shows the exact line that changed, for example a colour going from blue to green.
3. **Unsafe vs. guided AI edits.** An AI cannot open a `.pbix`. It *can* edit text files, and the repository's rule files keep those edits within the team's standards. The blue dashed outline marks this as the AI step.
4. **Late vs. early mistakes.** Instead of discovering errors when Power BI refuses to open the file, scripts catch most errors in the text first, then screenshots catch anything that still *looks* wrong.

Who this repository is for

Person	What they get from it
Report builders learning Power BI	A ready-made, correctly structured project to start from
Business intelligence (BI) teams	One shared standard for how every dashboard is built, reviewed and tested
People using GitHub Copilot (or Claude)	Rule files and skills that keep AI-written changes consistent
Reviewers and managers	Every change visible in Git, plus screenshot evidence
Platform engineers	Validation scripts and a GitHub Actions workflow to extend

The `README.md` states four goals: standardise how dashboards are built, reviewed and tested; keep report and model files source-controlled and auditable; support AI-assisted development with clear guardrails; and provide repeatable test evidence.

What is in the box when you start

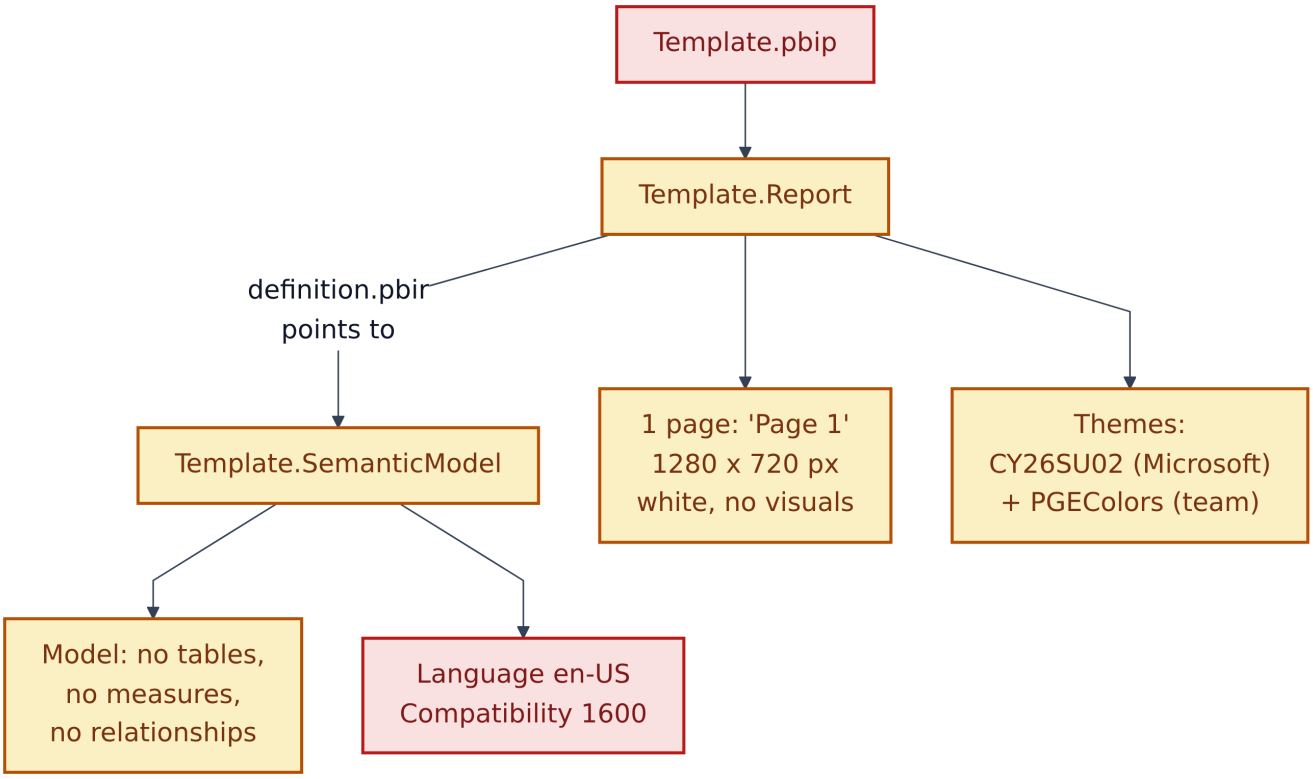


Figure 1.4 — The starting state. The template is deliberately almost empty, like a new notebook with the margins already ruled.

Walkthrough of Figure 1.4:

1. **`Template.pbip`** is the file you open in Power BI Desktop. It points to the report folder.
2. **`Template.Report`** holds one page called “Page 1”, 1280 by 720 pixels, white, with no charts yet.

3. **Two themes** are registered: Microsoft's base theme `CY26SU02` and the team's custom colour theme `PGEColors`.
4. **`Template.SemanticModel`** is the data model. It has no tables, measures or relationships yet. You or the AI add them.
5. **Language and compatibility settings** (English US, compatibility level 1600) are fixed technical settings. Leave them alone.
6. The labelled arrow shows that the report finds its model through a pointer file, `definition.pbir` (explained in Section 02).

The ten guardrails (the house rules in one list)

The README, `AGENTS.md` and `.github/copilot-instructions.md` all repeat these "non-negotiable" rules. You will meet each one again later.

1. New business measures (calculations such as "Total Sales") go in a table called `_Measures`.
2. Measures that draw SVG picture cards go in `_SVG_Measures`, named by page: `P1_...`, `P2_...`.
3. Background images and custom themes live in `Template.Report/StaticResources/RegisteredResources/` and must be listed in `report.json`.
4. Follow the exact file layout rules in `copilot-instructions.md`, including properties that are forbidden.
5. Save `.tmdl` model files as UTF-8 *without* a byte-order mark, and never use `//` comments in them.
6. Format numbers sensibly and consistently, with thousand separators.
7. SVG picture cards use the "image visual" pattern.
8. Make fake test data with Python (numpy and pandas).
9. Never call a page done because its files are valid; check the screenshot.
10. Give every measure a description.

`.github/copilot-instructions.md` adds two more: keep "SVG header" mode as the default page style, and treat `docs/theme/Theme.json` as the editable source, never registering it directly.

02. Power BI and the Power BI Project format (.pbip)

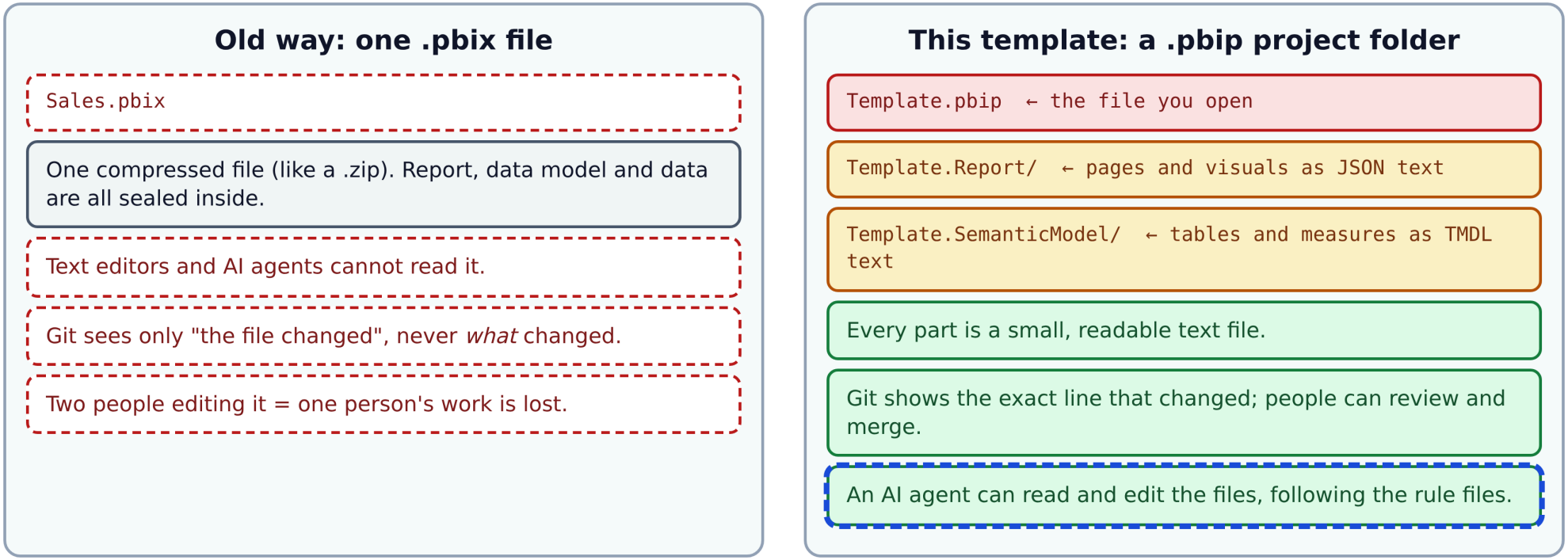


Figure 2.1 — A sealed box vs. a labelled filing cabinet. On the left, the traditional single `.pbix` file. On the right, the project folder this repository uses.

Walkthrough of Figure 2.1:

1. **`.pbix`** is Power BI's classic file type: one compressed file that contains the report, the data model and a copy of the data. It is like a sealed box: you can only open it with Power BI Desktop.
2. Because it is sealed, **text editors and AI agents cannot read it**, and Git (Section 03) can only say "the box changed".
3. **`.pbip`** stands for **Power BI Project**. When Power BI Desktop saves as a project, Microsoft's documentation says it saves "report and semantic model item definitions … as individual plain text files" in a folder. It is like a filing cabinet with a labelled drawer for every part.
4. **`Template.pbip`** is a small text file that acts as the cabinet's front door: it is the file you double-click.
5. **`Template.Report/`** holds the report pages and visuals as **JSON** files. JSON (JavaScript Object Notation) is a common text format made of named values inside curly braces, like `"displayName": "Page 1"`.
6. **`Template.SemanticModel/`** holds the data model as **TMDL** files. TMDL (Tabular Model Definition Language) is a text format for data models that uses indentation (tabs) instead of braces, so it reads more like an outline.

7. The result: Git shows exactly what changed, people can review changes, and an AI agent can safely edit the files (blue outline).

Key ideas in plain English

Term	Plain meaning	Real-world comparison
Power BI Desktop	Microsoft’s free Windows app for building reports	The workshop where the furniture is built
Report	The pages of charts people look at	The plated meal
Page	One screen of the report	One page of a magazine
Visual	One chart, table, card or image on a page	One photo on that page
Semantic model	The tables, relationships and calculations behind the report	The kitchen and pantry
Table	Rows and columns of data, like a spreadsheet tab	One shelf in the pantry
Relationship	A link between two tables, e.g. each sale belongs to one date	The label that says which shelf goes with which recipe
Measure	A named calculation, e.g. “Total Sales”, written in DAX	A recipe
DAX	Data Analysis Expressions: Power BI’s formula language	The cooking language the recipes are written in
Theme	A JSON file of colours and fonts	The restaurant’s tableware and colour scheme
PBIR	Power BI enhanced report format: the report saved as many JSON files in a `definition/` folder. Microsoft says it is now the default report format	The labelled drawers of the report cabinet
TMDL	The model saved as indented text files	A neatly indented recipe book

The report folder

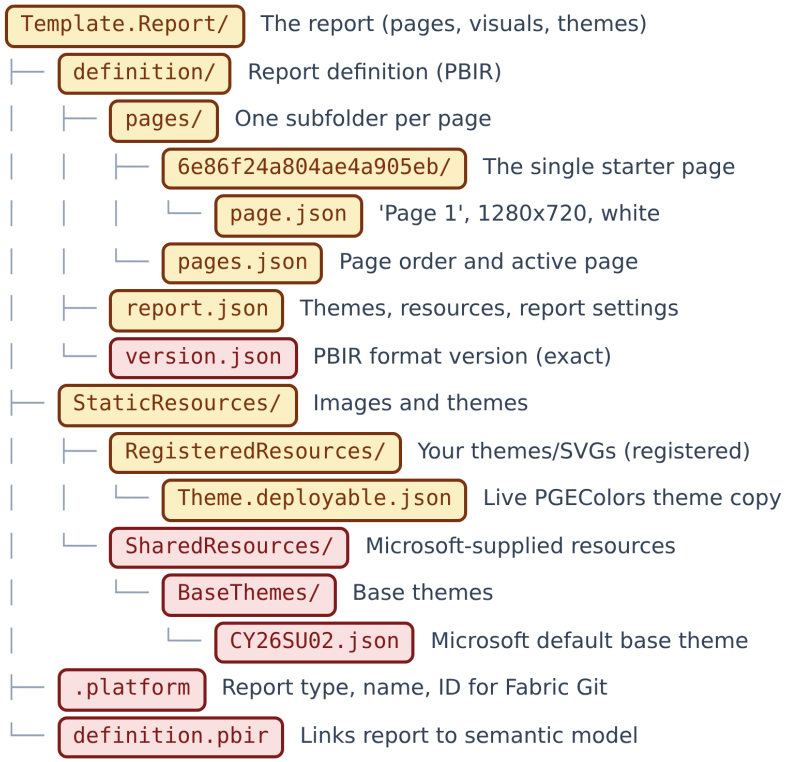


Figure 2.2 — Inside `Template.Report/`. Each label is coloured with its safety flag from the legend in Section 00.

Walkthrough of Figure 2.2:

1. **`definition/`** is the PBIR report definition.
2. **`pages/`** has one subfolder per page. The only page today has the machine-made name `6e86f24a804ae4a905eb`; its `page.json` gives it the human name “Page 1”, a size of 1280 × 720 pixels and a white surround.
3. **`pages.json`** lists the page order and which page opens first.
4. **`report.json`** registers the two themes and sets report-wide options such as “allow export of summarised data”. It is Amber: you will edit it when you add a theme or background image.
5. **`version.json`** must contain exactly the content shown in `copilot-instructions.md`. A wrong value stops the report opening, so it is Red.
6. **`StaticResources/RegisteredResources/`** is where *your* themes and SVG background images go. Each one must also be listed in `report.json`.

7. **`StaticResources/SharedResources/BaseThemes/CY26SU02.json`** is Microsoft's base theme. Leave it alone.
8. **`.platform`** is an ID card used by **Microsoft Fabric** (Microsoft's online analytics platform) when the project is synced with Git. Microsoft calls it a "Fabric platform file". Do not change it.
9. **`definition.pbir`** points the report at its semantic model (see Figure 2.4).

As you build, more files appear: `pages/<page-id>/visuals/<visual-id>/visual.json` for each visual, and `bookmarks/` if you use bookmarks.

The semantic model folder

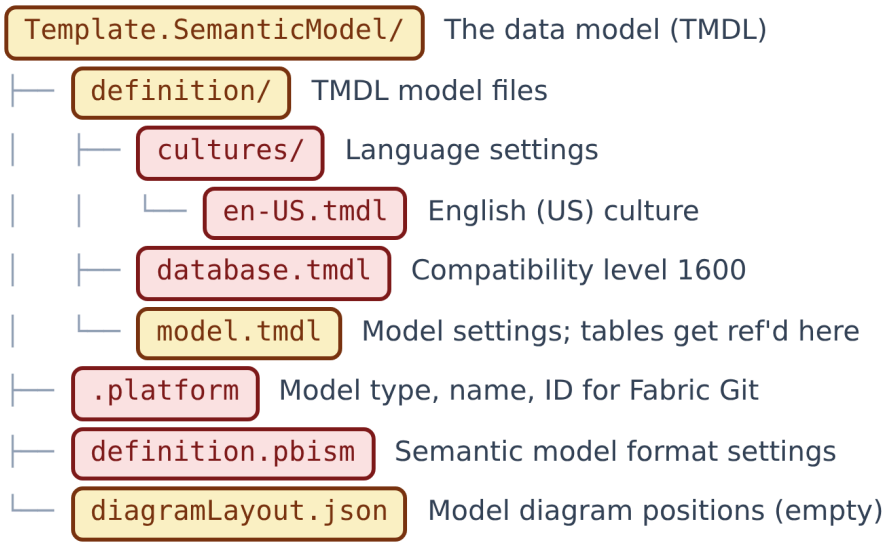


Figure 2.3 — Inside `Template.SemanticModel/`.

Walkthrough of Figure 2.3:

1. **`definition/database.tmdl`** sets the compatibility level (1600), a technical version number for the model engine. Red.
2. **`definition/model.tmdl`** holds model-wide settings (culture `en-US`, the "DevMode" tooling annotation, time-intelligence setting). When tables are added, each one gets a `ref table` line here. Amber.
3. **`definition/cultures/en-US.tmdl`** declares the English (US) language. Red.
4. **`diagramLayout.json`** remembers where tables sit in Power BI's model diagram. Empty today. Amber.
5. **`definition.pbism`** declares the semantic model format version (4.2). Red.
6. **`.platform`** is the model's Fabric ID card. Red.

Later, each table becomes its own file: `definition/tables/<TableName>.tmdl`, including the special tables `_Measures` and `_SVG_Measures` required by the guardrails.

How the files find each other

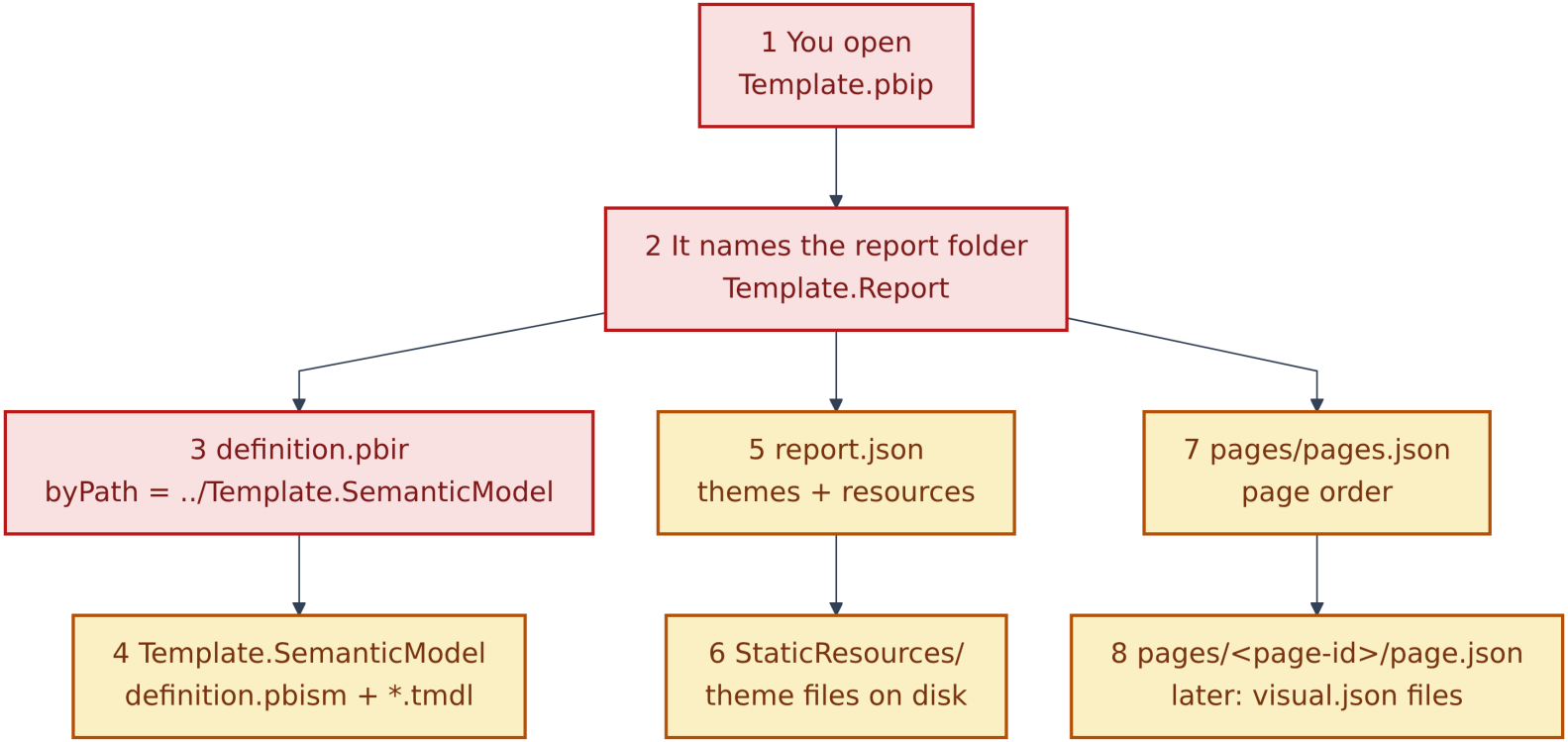


Figure 2.4 — The pointer chain. Like a treasure hunt: each file holds the clue to the next.

Walkthrough of Figure 2.4:

1. You open **`Template.pbip`** in Power BI Desktop.
2. Inside it, `artifacts > report > path` says the report lives in **`Template.Report`**.
3. The report's **`definition.pbir`** says `datasetReference > byPath > path = ../Template.SemanticModel`. The `../` means "go up one folder".
4. Power BI loads the **semantic model**: `definition.pbism` plus the `.tmdl` files.
5. **`report.json`** lists the themes and other resource packages.
6. Those resources must exist on disk under **`StaticResources/`**. The script `validate-pbip.ps1` checks this.
7. **`pages.json`** gives the page order.
8. Each **`page.json`** (and later each `visual.json`) describes one page or visual. Visuals ask the model for data by table name and column or measure name. If a name is misspelled, the visual breaks. `Test-PbipSemantics.ps1` checks many of these links (Section 09).

Model vs. report: the kitchen and the meal

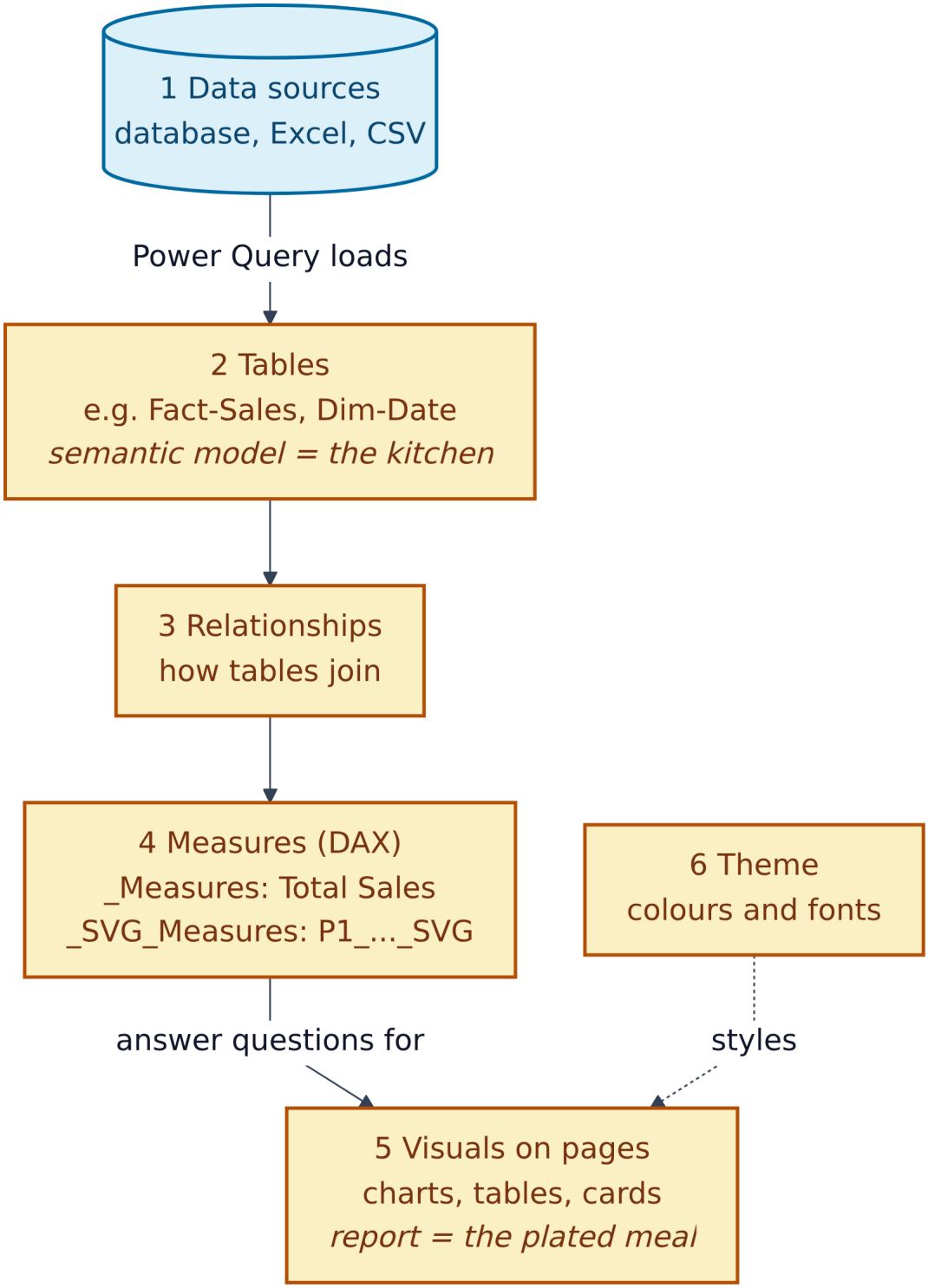


Figure 2.5 — From data to picture.

Walkthrough of Figure 2.5:

1. **Data sources** (a database, Excel, CSV files) hold the raw data.
2. **Power Query** (Power BI's data-loading tool) loads them into **tables** in the semantic model.

3. **Relationships** join the tables. This template's checklists prefer a **star schema**: Microsoft describes *fact* tables as storing "observations or events" (sales, orders) and *dimension* tables as describing "business entities" (products, dates, customers).
4. **Measures** written in DAX turn rows into answers, like "Total Sales" or "Sales YoY %". Business measures live in `_Measures`; measures that draw picture cards live in `_SVG_Measures`.
5. **Visuals** on pages show those answers.
6. The **theme** styles every visual with the team's colours and fonts.

What this template expects you to know about the format

- **Never edit while Desktop has the file open** unless a script tells you to reload. Desktop can save over your edits.
- **Save `.tmdl` files as UTF-8 without a BOM.** A BOM (byte-order mark) is an invisible marker some programs add at the start of a file. Power BI's TMDL reader rejects it. Windows PowerShell's `Set-Content -Encoding UTF8` adds one, so the rulebook tells you to write files with Python instead.
- **No `//` comments in TMDL.** Use `///` description lines above a measure instead; those become the measure's description.
- **Desktop keeps two local files out of Git:** `.pbi/localSettings.json` and `.pbi/cache.abf` (the data cache). This repository's `.gitignore` excludes them, plus `.pbi/editorSettings.json`.

03. Git, GitHub, and why we fork

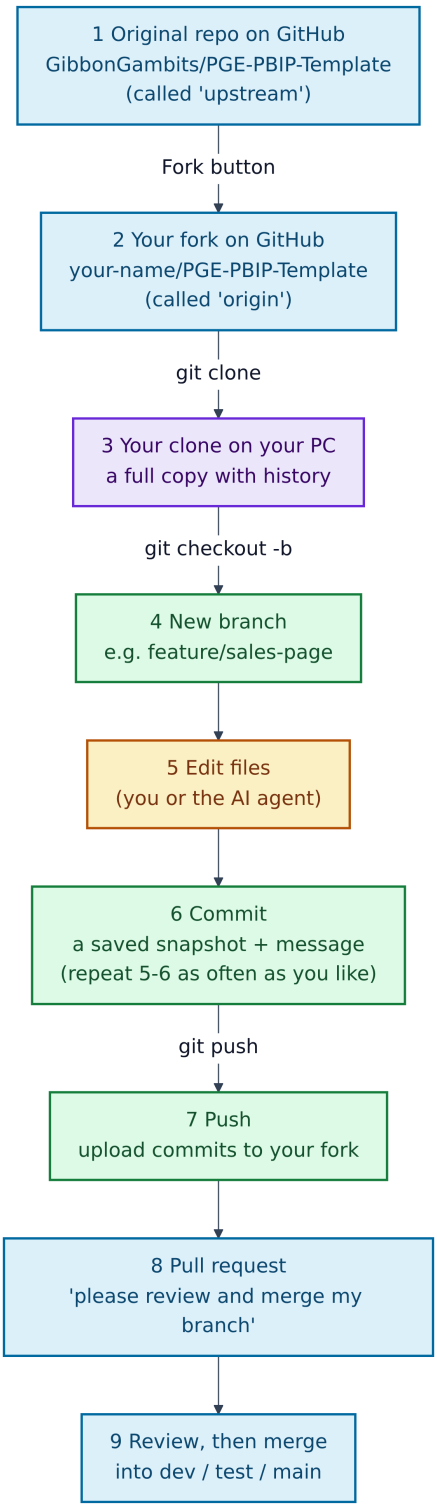


Figure 3.1 — The life of a change. From the original template on GitHub, through your own copy, to a reviewed and merged change.

Walkthrough of Figure 3.1:

1. **The original repository** (`GibbonGambits/PGE-PBIP-Template`) lives on GitHub. In Git language this is the **upstream**: the source you copied from.

2. **Your fork** is your own copy of that repository on GitHub, made with GitHub's *Fork* button. GitHub's documentation says forking "lets you propose changes to a project without affecting the upstream repository". Your fork is usually called **origin**.
3. **Your clone** is a full copy of your fork, downloaded to your computer with `git clone`. It includes the entire history.
4. **A branch** is a separate line of work, e.g. `feature/sales-page`. It lets you experiment without disturbing the main line.
5. **You or the AI agent edit files.**
6. **A commit** saves a snapshot of your changes with a short message. You can repeat steps 5 and 6 as often as you like.
7. **A push** uploads your commits to your fork on GitHub.
8. **A pull request** (PR) asks the team to review your branch and merge it. GitHub defines pull requests as "proposals to merge code changes into a project".
9. **Review and merge.** Reviewers comment, you fix, and the branch is merged into `dev`, then `test`, then `main` (Figure 3.3).

Later improvements to the template reach you the other way: you fetch them from **upstream** and merge them into your copy (commands below).

Git and GitHub in plain words

- **Git** is a free program on your computer that records changes to files over time. The official Git book describes version control as a system that "records changes to a file or set of files over time so that you can recall specific versions later". Comparison: an unlimited "undo" history with a label on every save.
- **GitHub** is a website that hosts Git repositories online and adds collaboration tools: pull requests, reviews, issues and automation (GitHub Actions). Comparison: Git is the camera that takes the snapshots; GitHub is the shared photo album where the team looks at them together.

Why we fork (and not just clone)

Reason	What it means for you
It is a template	Each team or dashboard gets its own copy to customise: rename `Template.pbip`, add tables, change the theme.
No risk to the original	You cannot break the upstream template by experimenting in your fork.
You still get updates	`docs/REPOSITORY-GUIDE.md` shows how to pull in template improvements from upstream.
Pull requests work both ways	You can propose an improvement back to the template from your fork.

The repository's own update recipe (`docs/REPOSITORY-GUIDE.md`, "How to update from upstream"):

```
git remote add upstream https://github.com/<upstream-owner>/<upstream-repo>.git   # once
git fetch upstream
git checkout main
git merge upstream/main    # or: git rebase upstream/main
```

After merging, re-run the validation commands (Section 09) and resolve any conflicts in `.github/copilot-instructions.md`, `.github/skills/`, `scripts/` and the PBIP project files.

The three places your changes live

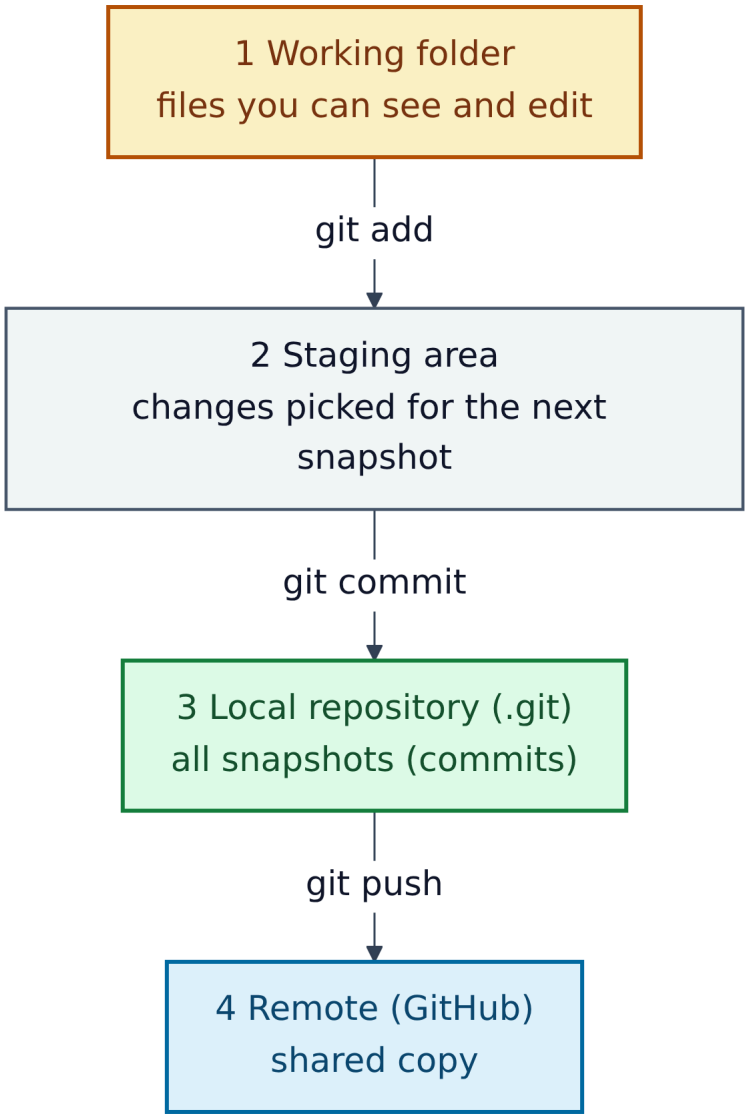


Figure 3.2 — Where a change goes on its way to GitHub.

Walkthrough of Figure 3.2:

1. **Working folder:** the files you see in File Explorer and VS Code. Edits land here first.
2. **Staging area:** `git add` picks which changes go into the next snapshot. Comparison: putting photos in an envelope before posting it.
3. **Local repository:** `git commit` seals the envelope and files it in the hidden `.git` folder, with a message.
4. **Remote:** `git push` sends your commits to GitHub. The reverse, `git pull`, brings other people's commits down into your working folder.

VS Code does all four with buttons in its **Source Control** view (`Ctrl+Shift+G`), so you rarely need to type these commands.

The branch path this repository recommends

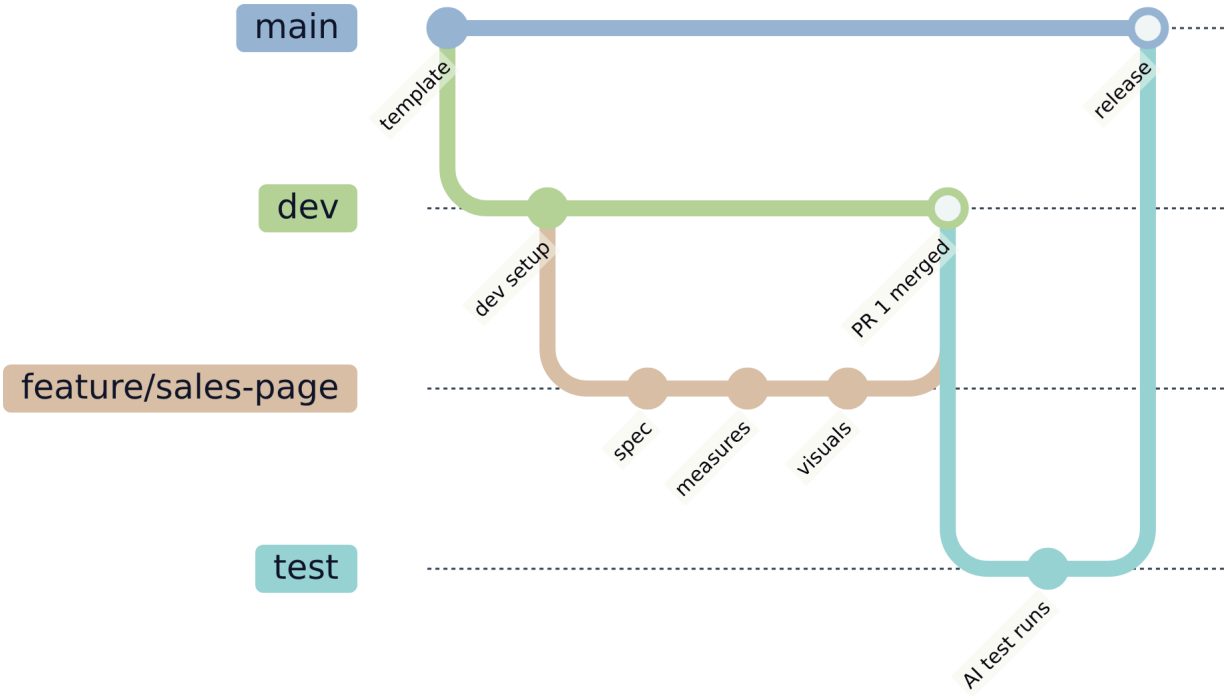


Figure 3.3 — Branches over time. Each coloured line is a branch; each dot is a commit.

Walkthrough of Figure 3.3:

1. `main` holds what is released (production).
2. `dev` is where finished features are combined first.
3. `feature/sales-page` is your short-lived branch: write the spec, add measures, add visuals.
4. **"PR 1 merged":** your pull request into `dev` is approved and merged.
5. `test` is the release-candidate branch. Pushing to `test` runs the AI test workflow in `.github/workflows/ai-testing.yml` (Section 11).
6. **"release":** `test` is merged into `main`.

`docs/CICD-SETUP.md` and `docs/STARTER-PACK-CHECKLIST.md` describe this path as feature → `dev` → `test` → `main`, with branch protection rules (required reviews and status checks) on the three long-lived branches.

Who does what when you save work

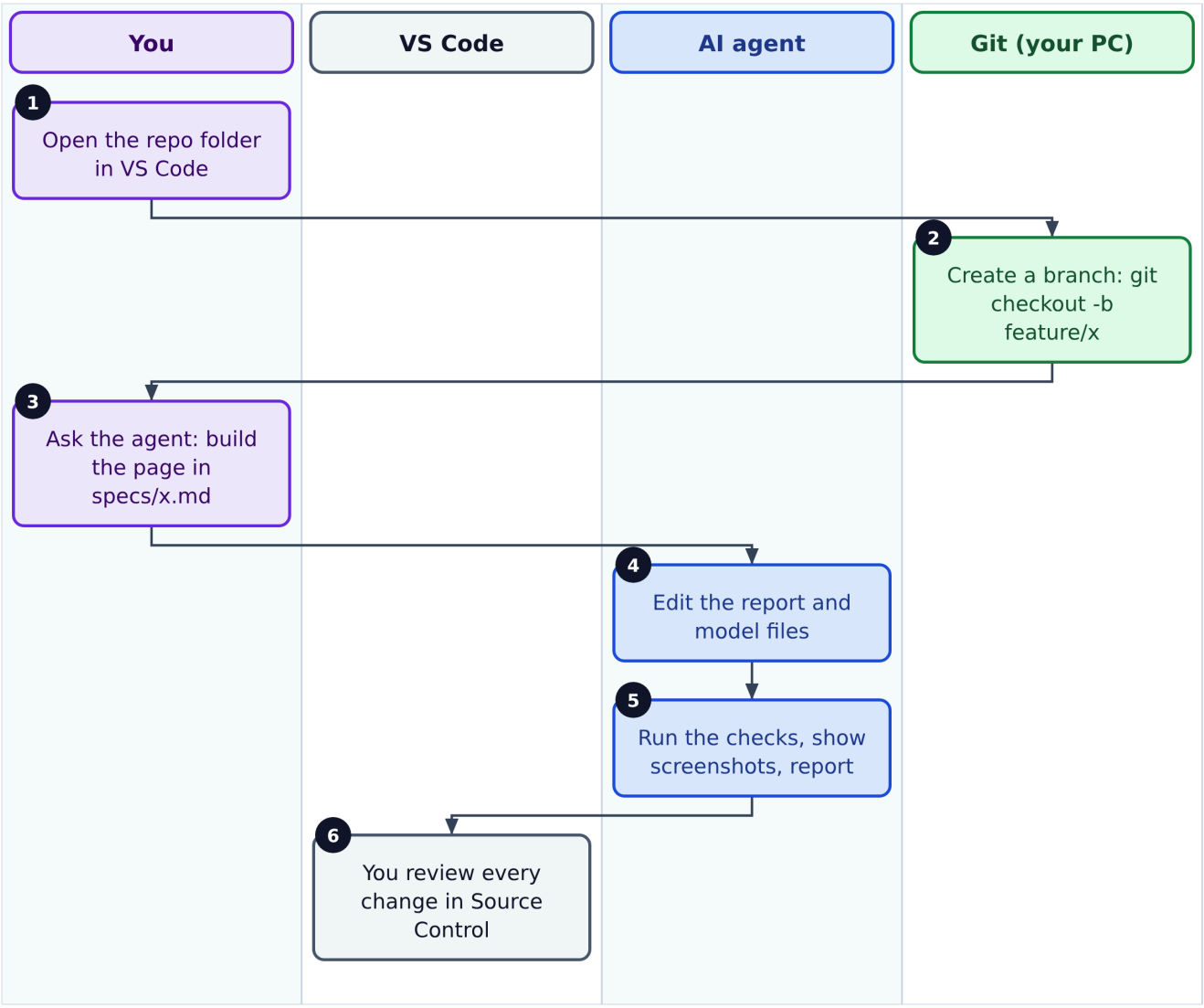


Figure 3.4 — Swimlane, part 1: building the change. Each column is a “lane” for one person or tool. Follow the numbered boxes from top to bottom; each arrow hands the work to the next box.

Walkthrough of Figure 3.4:

1. You open the repository folder in VS Code.
2. Git creates a branch for your work (`git checkout -b feature/x`, or the branch button in VS Code).
3. You ask the AI agent to build what the spec describes.
4. The agent edits the report and model files.
5. The agent runs the checks and shows you the screenshots and a summary.
6. You review every changed line in VS Code’s Source Control view.

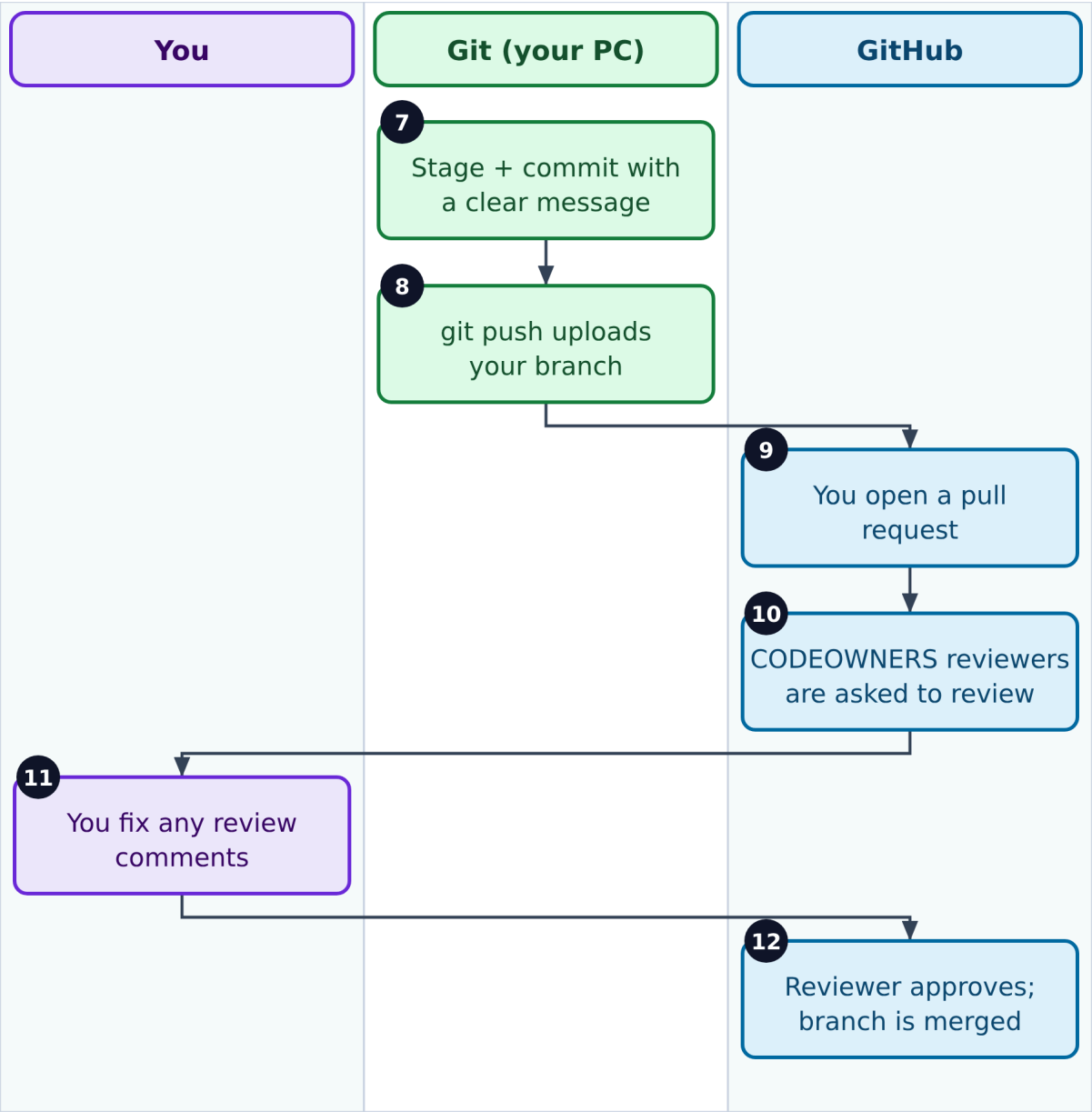


Figure 3.5 — Swimlane, part 2: saving and sharing. Continues from step 6.

Walkthrough of Figure 3.5:

7. Git stages and commits your changes with a clear message, e.g. “Add Sales Overview page”.
8. `git push` uploads your branch to GitHub.
9. You open a pull request on GitHub, filling in `.github/PULL_REQUEST_TEMPLATE.md` (a summary plus a three-item checklist).
10. GitHub automatically asks the owners named in `.github/CODEOWNERS` to review. GitHub’s docs: owners “are automatically requested for review” when a pull request changes their files.
11. You fix any review comments (new commits on the same branch update the pull request).
12. A reviewer approves and the branch is merged.

Files in this repository that are about Git and GitHub

File	What it does
`.gitignore`	Lists files Git must never save: local Power BI caches (`**/.pbi/...`), your private `.env`, Python caches, `tests/screenshots/`, `.github/.pbip-state/`, and `Proposals/`.
`.github/CODEOWNERS`	Names which team reviews which folder. The handles (`@your-org/...`) are placeholders to replace.
`.github/PULL_REQUEST_TEMPLATE.md`	The form every pull request starts with.
`.github/ISSUE_TEMPLATE/*.md`	Forms for reporting a bug or requesting a feature.
`.github/workflows/ai-testing.yml`	Automation that runs on GitHub (Section 11).
`CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`	How to contribute, and how to behave.

04. VS Code, and what an AI coding agent does compared with a normal chat

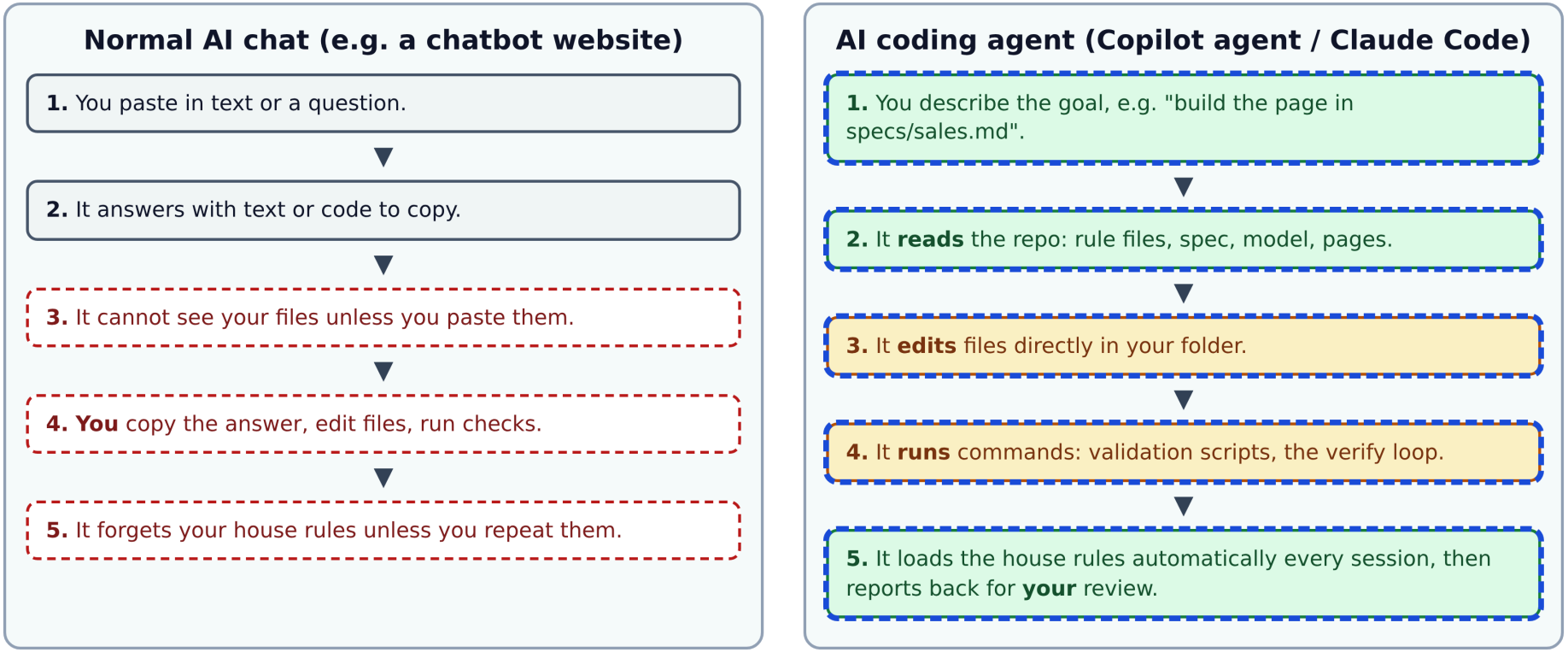


Figure 4.1 — A chatbot gives advice; an agent does the work. Left: a normal AI chat. Right: an AI coding agent working inside your repository (blue dashed outlines mark AI actions).

Walkthrough of Figure 4.1:

1. **Chat:** you paste a question and get text back. **Agent:** you describe a goal, such as “build the page in `specs/sales.md`”.
2. **Chat:** it cannot see your files. **Agent:** it *reads* the repository: rule files, the spec, the model and the pages.
3. **Chat:** you must copy its answer into files yourself. **Agent:** it *edits* the files directly.
4. **Chat:** you run any checks. **Agent:** it *runs* commands, such as `scripts\validate-pbip.ps1`, and reads the output.

5. **Chat:** it forgets your house rules unless you repeat them. **Agent:** it loads the rule files automatically each session, then reports back so *you* can review.

VS Code's documentation puts it simply: "An agent uses a model and tools to find relevant code, make changes, and run checks."

Real-world comparison: a normal chat is a consultant on the phone. An agent is a contractor in your house, with the building code (rule files) pinned to the wall and a checklist to sign off. You still inspect the work before you accept it.

VS Code in one picture

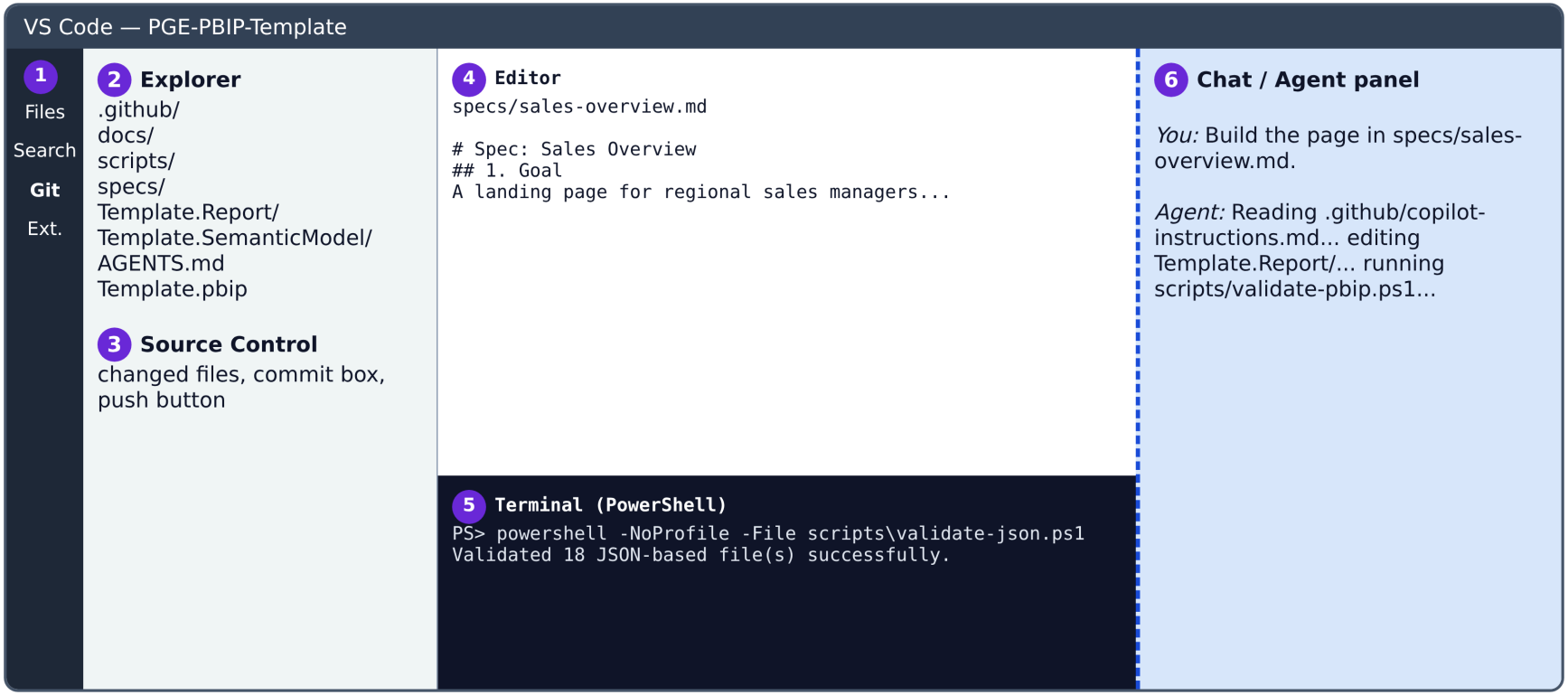


Figure 4.2 — The VS Code window. VS Code (Visual Studio Code) is Microsoft's free code editor for Windows, macOS and Linux.

Walkthrough of Figure 4.2:

1. **Activity bar:** switch between Files, Search, Git (Source Control) and Extensions.
2. **Explorer:** the repository's folders and files. Open the whole `PGE-PBIP-Template` folder, not single files.
3. **Source Control:** shows every changed file, a box for the commit message, and push/pull buttons. VS Code "has built-in Git support", but Git itself must be installed separately.
4. **Editor:** where you read and edit a file, e.g. your spec.
5. **Terminal:** a PowerShell window inside VS Code for running the repository's scripts.
6. **Chat / Agent panel:** where you talk to GitHub Copilot (or another agent such as Claude) and watch it read, edit and run commands.

How the agent learns this repository’s rules

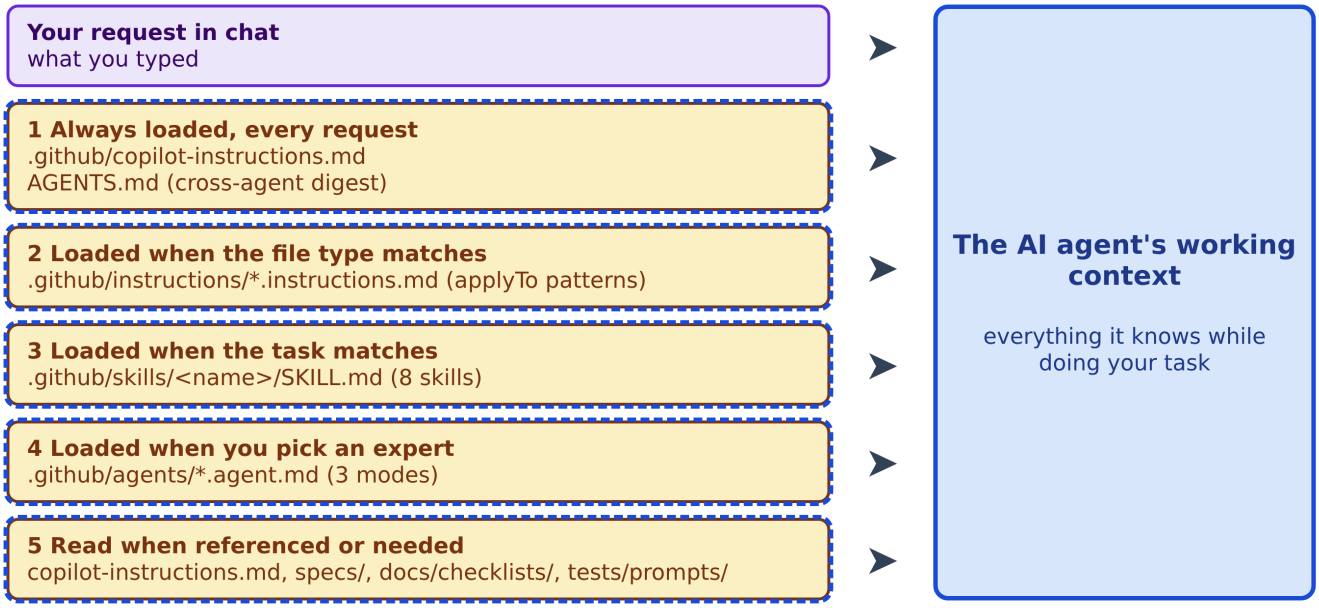


Figure 4.3 — Five ways rule files reach the agent. All are Amber with a blue outline: AI files that change the agent’s behaviour for everyone, so edit them with care.

Walkthrough of Figure 4.3:

1. **Always loaded.** GitHub’s docs say repository instructions in `.github/copilot-instructions.md` “are automatically added to requests that you submit to Copilot”. Copilot also supports `AGENTS.md` files, which VS Code calls “a cross-agent format”. Claude Code reads `AGENTS.md` when there is no `CLAUDE.md`. This repository has both files and keeps them in step.
2. **Loaded by file type.** Files named `*.instructions.md` in `.github/instructions/` have an `applyTo` pattern. For example, the custom-visuals rules apply to `**/*.{ts,tsx,js,jsx,less,css}`. They load only when the agent works on matching files.
3. **Loaded when the task matches. Skills** are folders with a `SKILL.md` file: “folders of instructions, scripts, and resources” that the agent loads when relevant. The `description` line is how the agent decides. This repository has 8 skills under `.github/skills/`.
4. **Loaded when you pick an expert. Custom agents** (`.github/agents/*.agent.md`) bundle instructions and tools for one role. Here: a data-modelling expert, a DAX expert and a visualisation expert.
5. **Read when needed.** The deep rulebook `copilot-instructions.md` (727 lines), your spec in `specs/`, the checklists in `docs/checklists/`, and the test prompts in `tests/prompts/` are read when the agent or you point to them.

The agent’s working loop

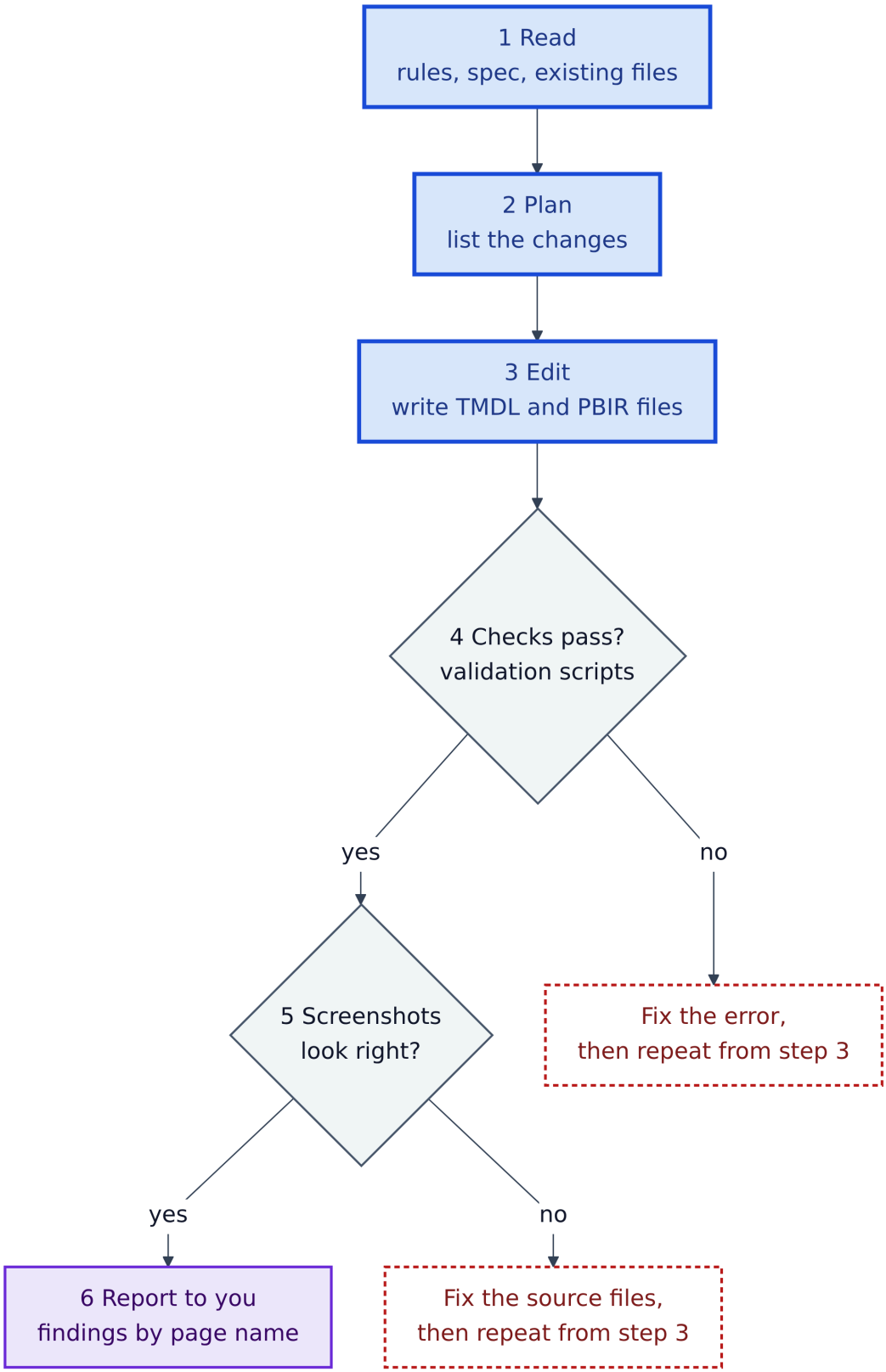


Figure 4.4 — The agent’s working cycle. Read top to bottom. The two “no” branches end in red dashed boxes that send the agent back to step 3.

Walkthrough of Figure 4.4:

1. **Read** the rules, the spec and the existing files.
2. **Plan** the list of changes.
3. **Edit** the report (PBIR JSON) and model (TMDL) files.
4. **Checks pass?** The validation scripts run. If not, the agent fixes the error and repeats from step 3.
5. **Screenshots look right?** Power BI Desktop draws every page. If something looks wrong, the agent fixes the source files (never the image) and repeats from step 3.
6. **Report** findings to you by page *display name* (e.g. “Sales Overview”, not `6e86f24a804ae4a905eb`). If you want changes, the cycle starts again at step 1.

GitHub Copilot and Claude in this repository

	GitHub Copilot	Claude (Claude Code)
Main rule file	`.github/copilot-instructions.md` (primary authority, per README)	`AGENTS.md` (read when no `CLAUDE.md` exists)
Skills	`.github/skills/*/SKILL.md`	Claude Code looks in `.claude/skills/`; this repo’s skills are written for Copilot
Expert modes	`.github/agents/*.agent.md`	Not used by Claude
Hooks	Run manually (`scripts/hooks/*.ps1`)	The hook scripts read the same JSON fields that Claude Code hooks send (`tool_input`, `session_id`, `stop_hook_active`), and use exit code 2 to block, which matches Claude Code’s documented hook behaviour

The README names one extra, one-time setup for **GitHub Copilot CLI** (the command-line version of Copilot): adding Microsoft’s `skills-for-fabric` plugin collection. Section 07 covers it.

Golden rules for working with an agent

1. **Write the spec first** (Section 08). An agent with a clear plan makes fewer guesses.
2. **Work on a branch**, never directly on `main`.
3. **Read every changed line** in Source Control before committing. You are responsible for what you commit.
4. **Never paste secrets** (passwords, API keys) into a chat.
5. **Do not accept “the JSON is valid” as done.** Ask for the screenshots (guardrail 9).

05. The master map: how every part connects, and in what order it runs

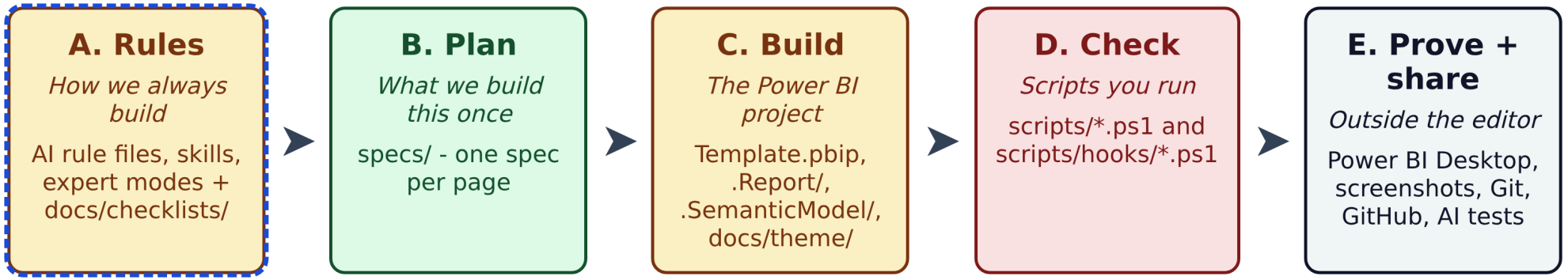


Figure 5.1 — The master map. Five stages, read left to right. Every top-level part of the repository belongs to one of them.

Walkthrough of Figure 5.1:

1. **A. Rules — how we always build.** The AI rule files (blue outline) plus the human quality checklists. They are permanent and apply to every dashboard.
2. **B. Plan — what we build this once.** A spec in `specs/`, copied from `TEMPLATE.md`. One per dashboard or page.
3. **C. Build — the Power BI project.** `Template.pbip`, the model, the report, and the theme source in `docs/theme/`.
4. **D. Check — scripts you run.** PowerShell scripts, from "does every JSON file parse?" up to "reload Power BI and screenshot every page".
5. **E. Prove and share.** Power BI Desktop draws the report, screenshots are saved locally, Git and GitHub share the work, and a GitHub workflow runs AI tests.

What sits in each stage:

Stage	Files and folders	Safety
A. Rules	`.github/copilot-instructions.md`, `AGENTS.md`, `copilot-instructions.md`, `.github/instructions/` (3), `.github/agents/` (3), `.github/skills/` (8), `docs/checklists/` (6)	Amber + AI; checklists Green

Stage	Files and folders	Safety
B. Plan	`specs/TEMPLATE.md`, `specs/EXAMPLE-sales-overview.md`, your own `specs/<page>.md`	Green
C. Build	`Template.pbip`, `Template.SemanticModel/`, `Template.Report/`, `docs/theme/`	Amber (entry and fixed files Red)
D. Check	`scripts/validate-json.ps1`, `validate-pbip.ps1`, `Test-PbipSemantics.ps1`, `Test-PbipBridge.ps1`, `Invoke-PbipVerify.ps1`, `Get-PbipLoadError.ps1`, `Invoke-PbipRefresh.ps1`, `PbipBridge.Common.ps1`, `run-ai-md-test.ps1`, `scripts/hooks/` (4)	Red
E. Prove and share	Power BI Desktop, `tests/screenshots/` (generated, not in Git), Git, GitHub, `.github/workflows/ai-testing.yml`, `tests/results/`	Red (tools are not files)

The order things run in

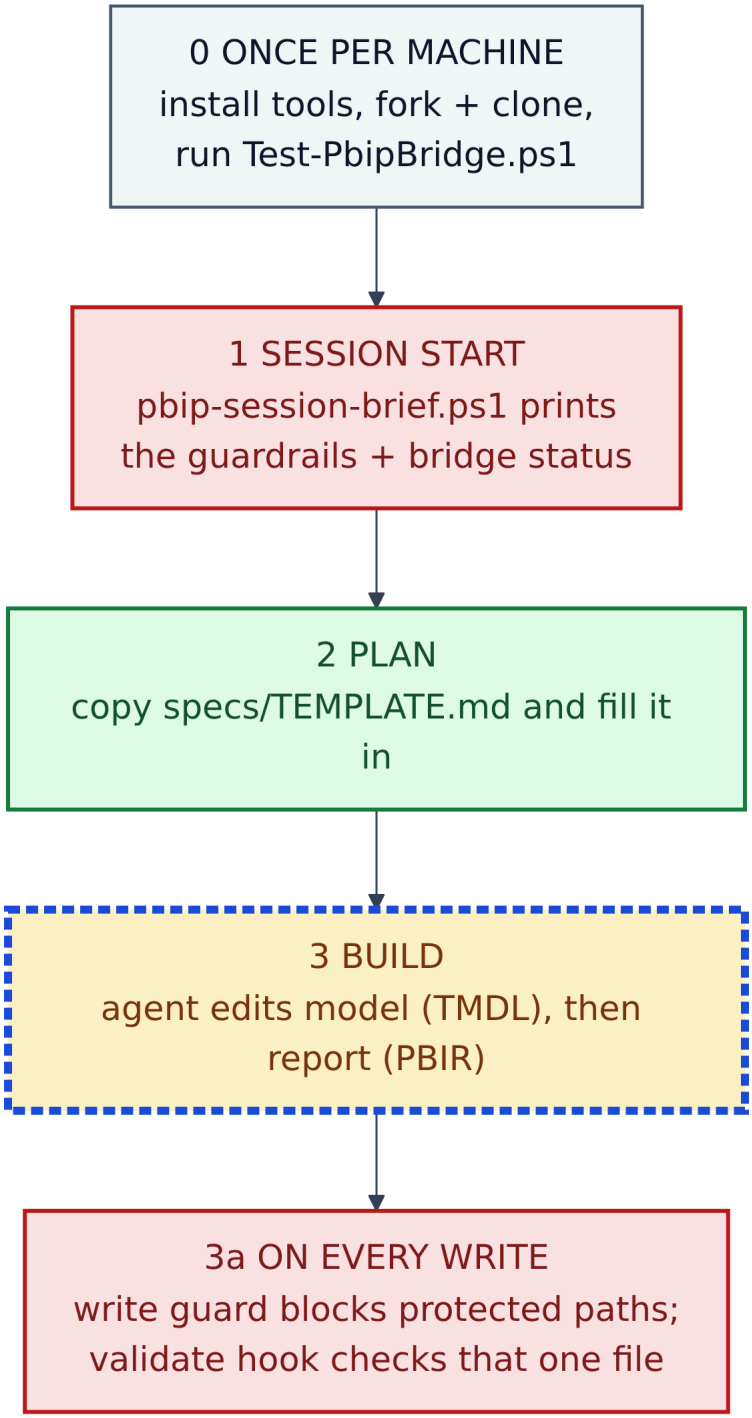


Figure 5.2 — Run order, part 1: before and during the build. Read top to bottom.

Walkthrough of Figure 5.2:

0. **Once per machine:** install the tools, fork and clone, and run the preflight `scripts\Test-PbipBridge.ps1` (Section 07).
1. **Session start:** `scripts\hooks\pbip-session-brief.ps1` prints a short brief of the guardrails and whether Power BI Desktop's bridge is up.

2. **Plan:** copy `specs/TEMPLATE.md` to `specs/<name>.md` and fill it in. Answer its “Open questions” before building.
3. **Build:** the agent (or you) edits the model first, then the report, following the build order in `copilot-instructions.md`.
    - **3a, on every write:** `pbip-write-guard.ps1` refuses writes to protected folders, and `pbip-validate-hook.ps1` checks the single file just written.

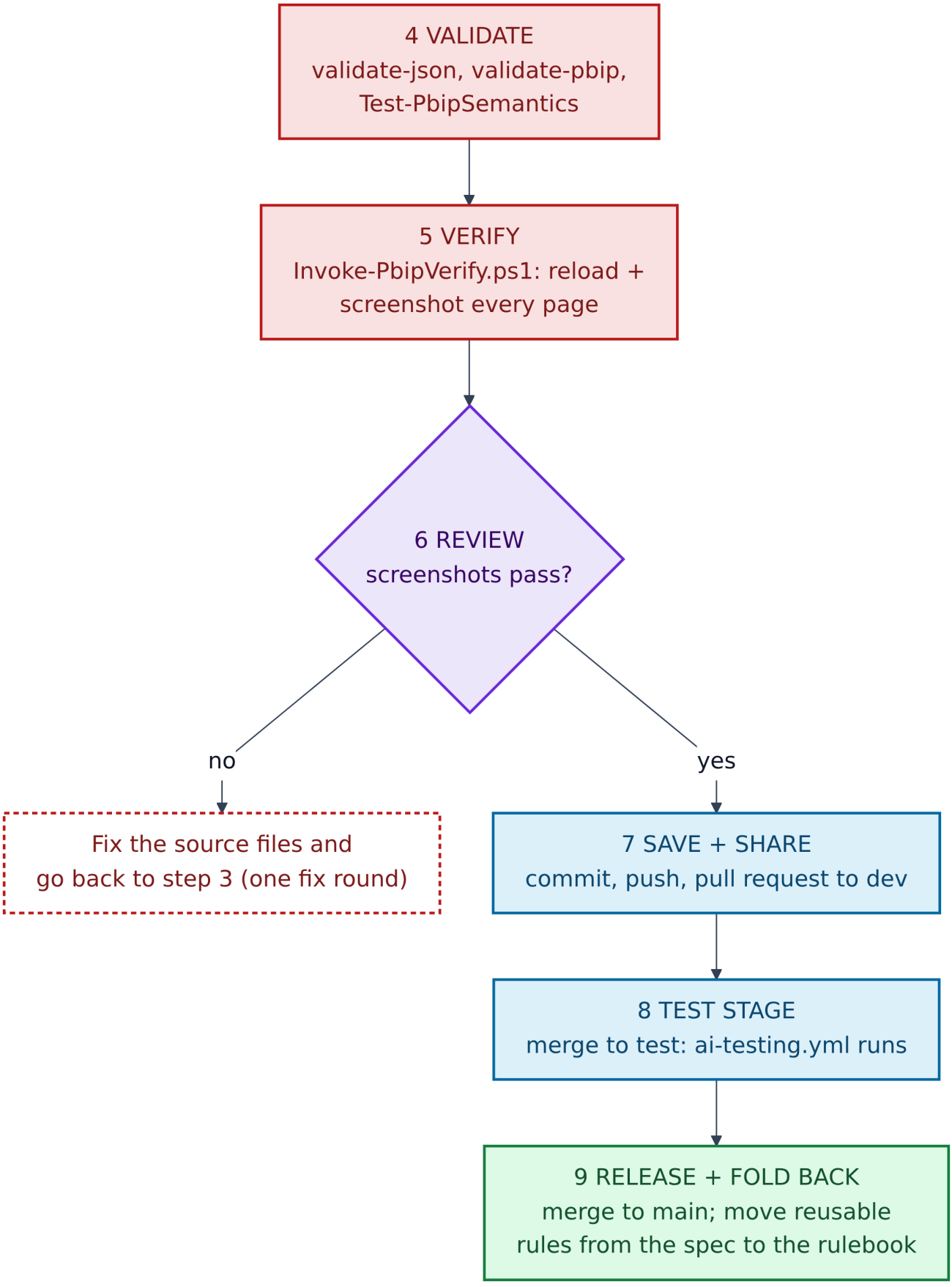


Figure 5.3 — Run order, part 2: checking, sharing and releasing. Continues from step 3.

Walkthrough of Figure 5.3:

4. **Validate:** `validate-json.ps1`, `validate-pbip.ps1` and `Test-PbipSemantics.ps1`.
5. **Verify:** `Invoke-PbipVerify.ps1` (or its wrapper `pbip-stop-hook.ps1`) reloads Power BI Desktop and screenshots every page.
6. **Review:** compare the screenshots with the `powerbi-visual-verify` skill's checklist. If they fail, fix the source files and go back to step 3. The repository allows **one fix round**: review and fix, then confirm and report.
7. **Save and share:** commit, push the branch, and open a pull request into `dev`.
8. **Test stage:** merging to `test` starts `.github/workflows/ai-testing.yml`.
9. **Release and fold back:** merge to `main`. Then move any rule from the spec that will apply to the *next* dashboard into `.github/copilot-instructions.md`, and keep `copilot-instructions.md` aligned.

Who does what, end to end

The full process has 19 hand-offs, so it is split into three swimlanes. In each one, columns are "lanes" (a person or tool) and the numbered boxes run top to bottom.

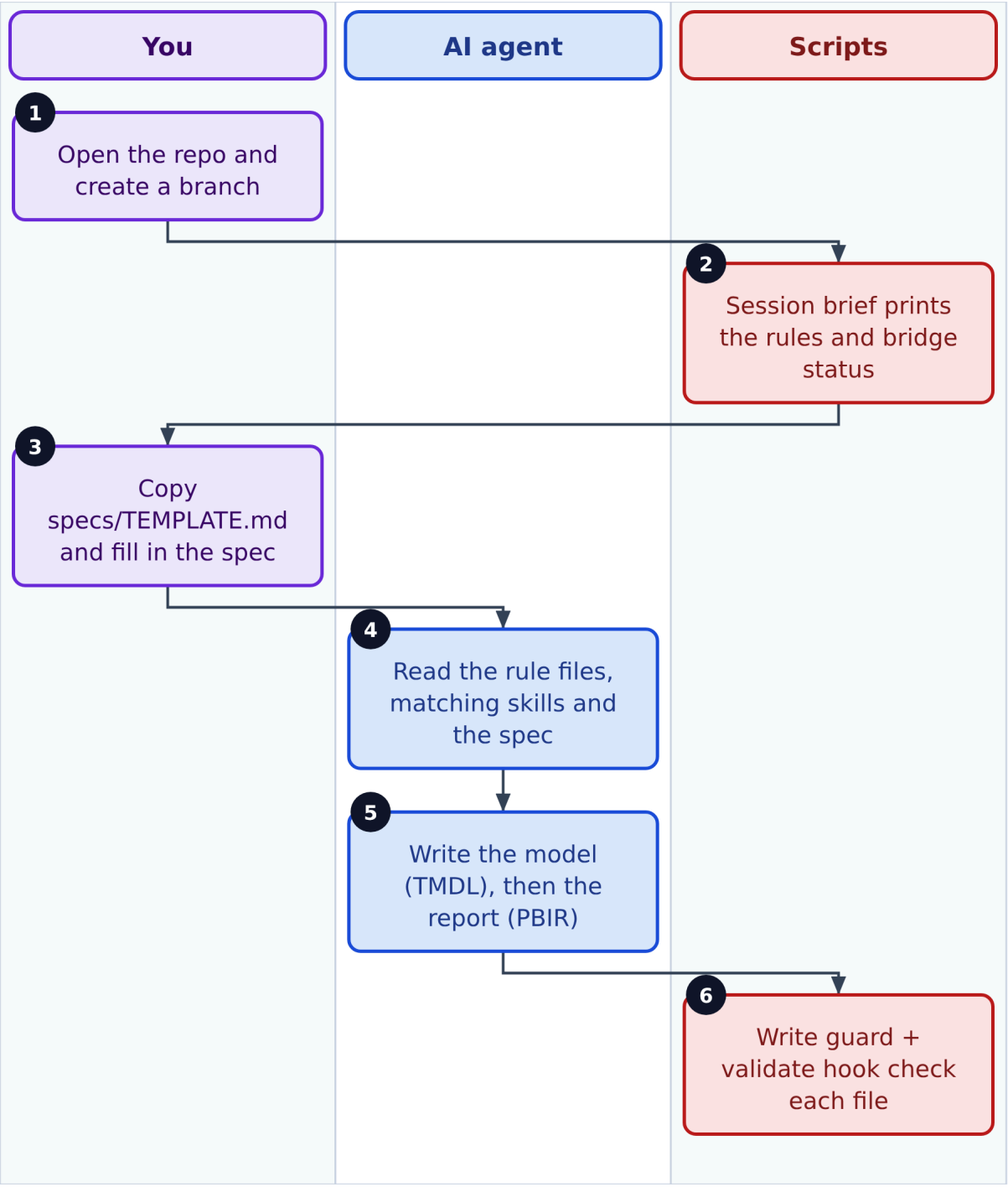


Figure 5.4 — Swimlane 1: plan and build (steps 1–6).

Walkthrough of Figure 5.4:

1. You open the repository and create a branch.
2. The session-brief script prints the rules and the bridge status.
3. You copy `specs/TEMPLATE.md` and fill in the spec.
4. The agent reads the rule files, any matching skills, and your spec.
5. The agent writes the model (TMDL) first, then the report (PBIR).
6. The write guard and the post-write validator check each file as it is written.

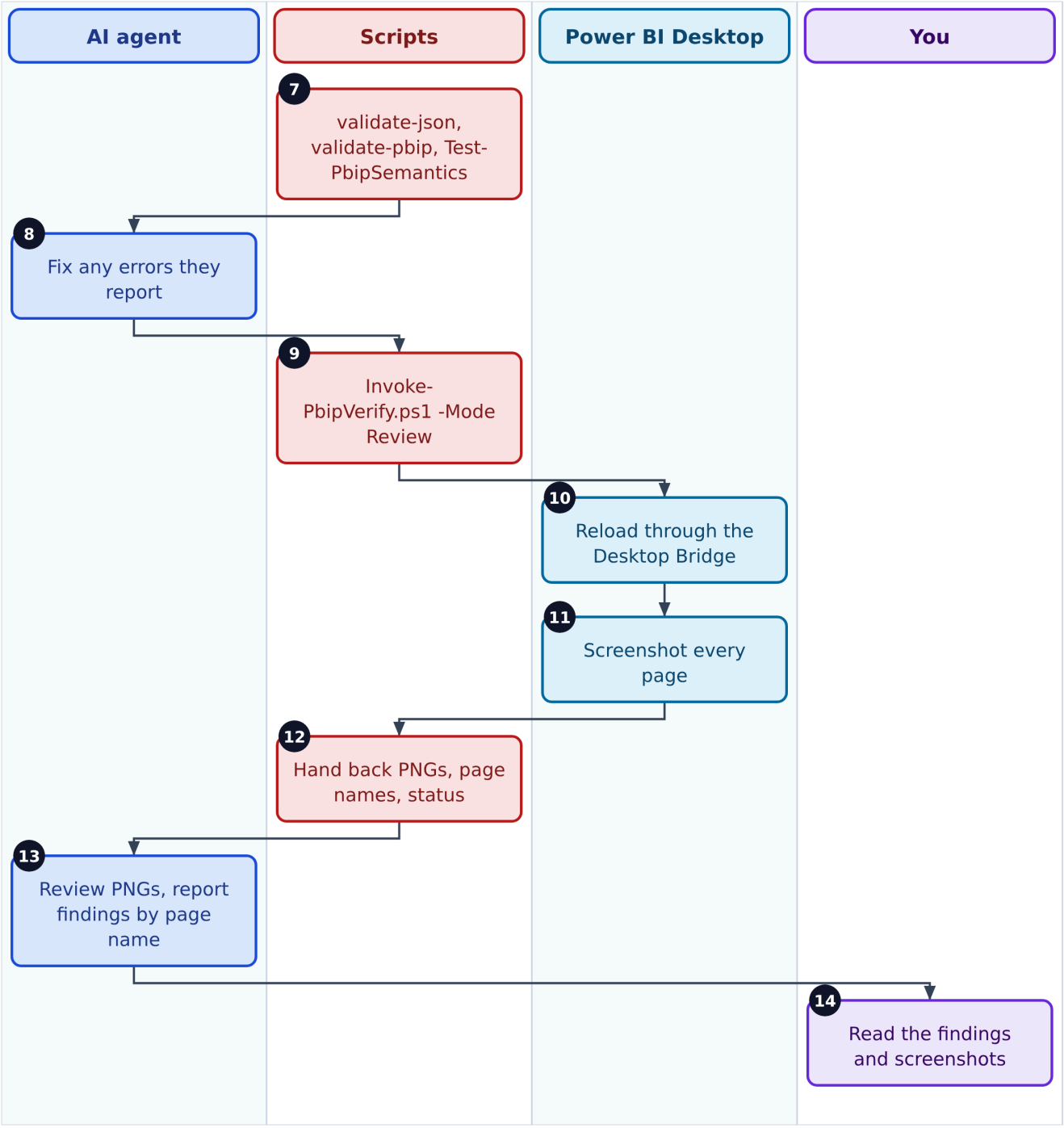


Figure 5.5 — Swimlane 2: check and verify (steps 7–14).

Walkthrough of Figure 5.5:

7. The scripts run the three validation checks.
8. The agent fixes any errors they report.
9. The agent runs `Invoke-PbipVerify.ps1 -Mode Review`.
10. Power BI Desktop reloads the project through the Desktop Bridge.
11. Power BI Desktop screenshots every page.
12. The scripts hand back the PNG files, page names and a status.
13. The agent reviews the PNGs and reports findings by page display name.
14. You read the findings and look at the screenshots yourself.

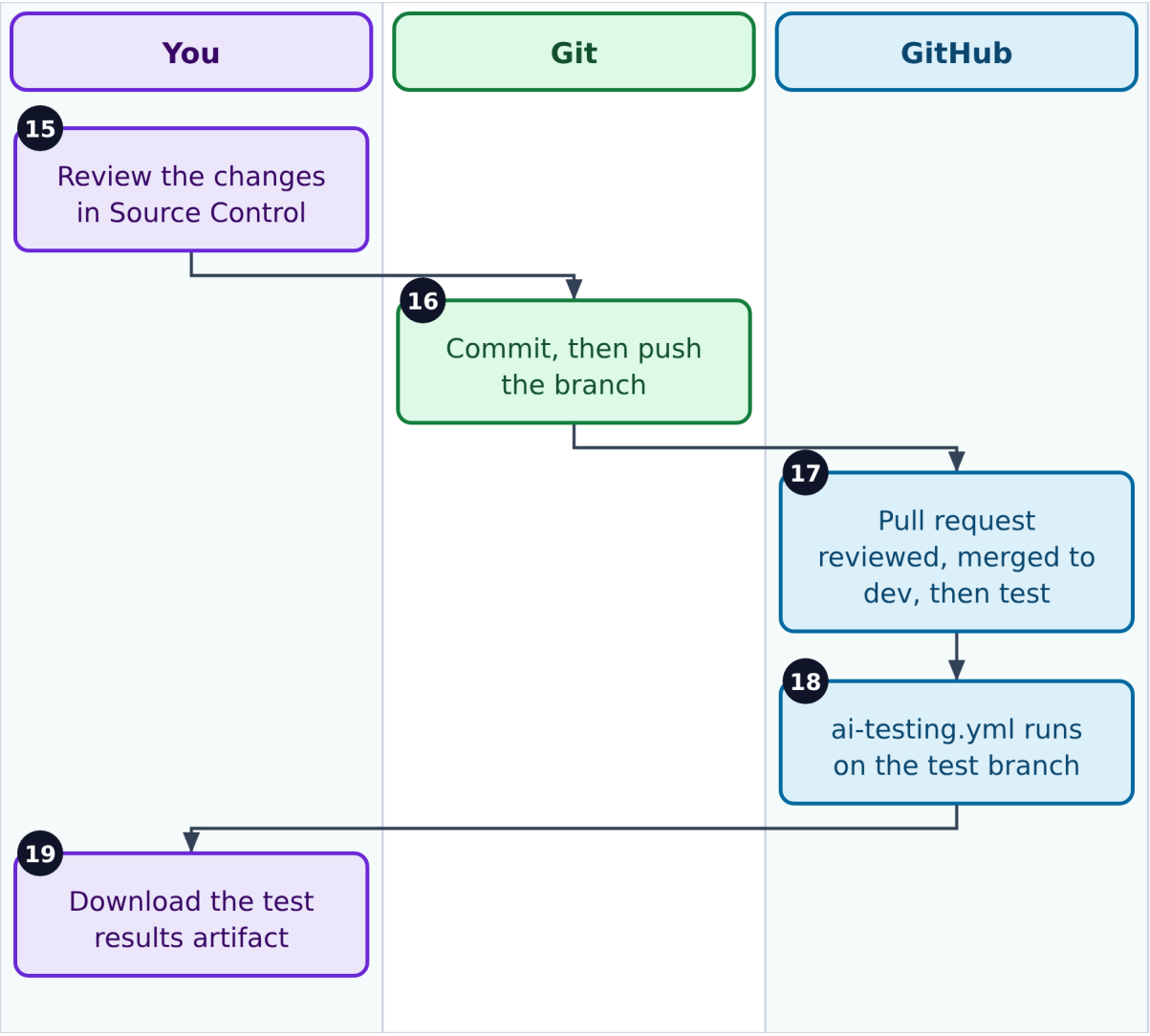


Figure 5.6 — Swimlane 3: save and share (steps 15–19).

Walkthrough of Figure 5.6:

15. You review the changes in VS Code’s Source Control view.
16. Git commits the changes and pushes the branch.
17. On GitHub, the pull request is reviewed and merged to `dev`, and later to `test`.
18. On `test`, the AI testing workflow runs.
19. You download the test results from the workflow run.

Which script uses which

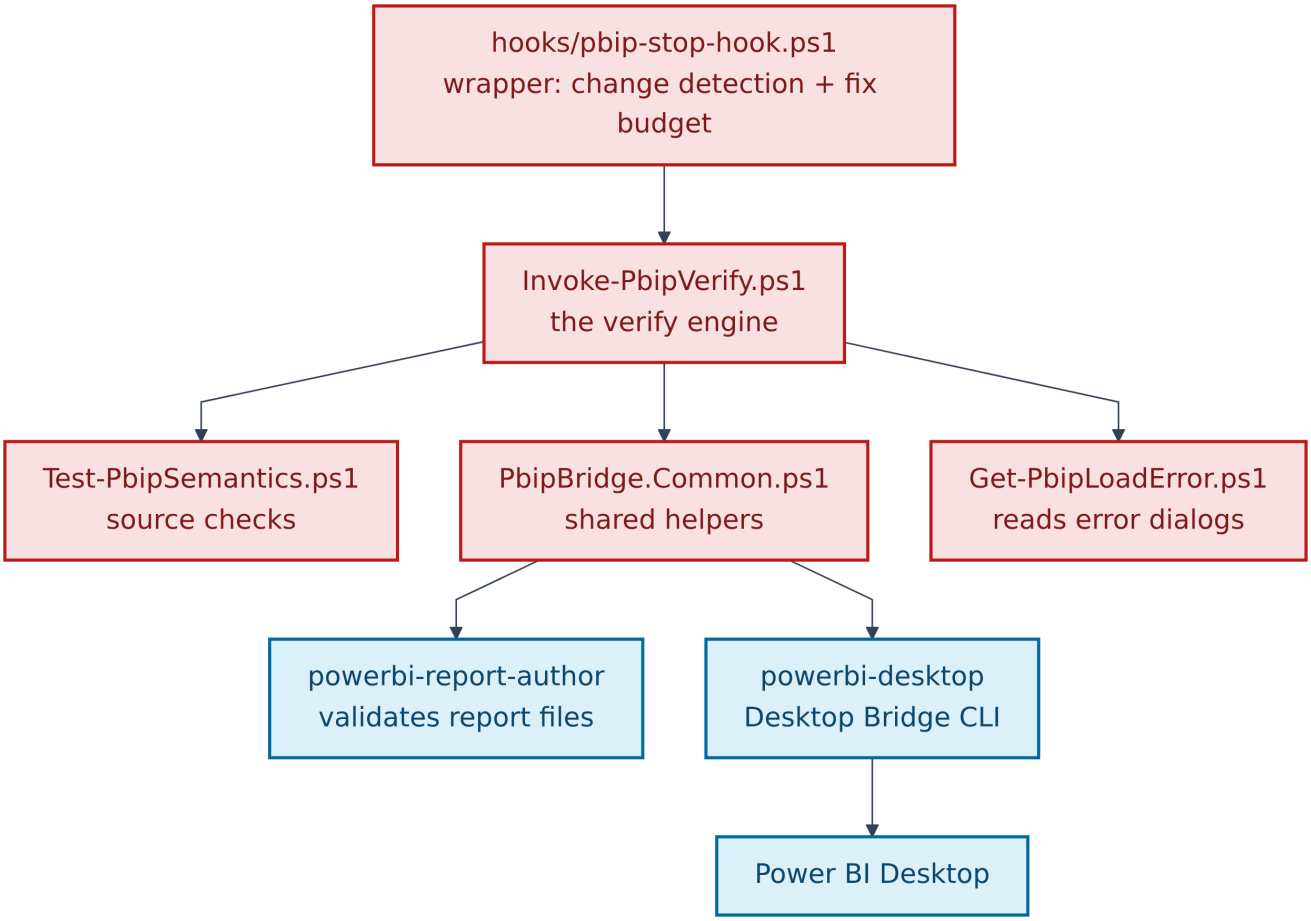


Figure 5.7 — The verify engine. An arrow means “loads or calls”. All scripts are Red: run them, do not edit them.

Walkthrough of Figure 5.7:

1. `hooks/pbip-stop-hook.ps1` is a thin wrapper that adds change detection and the one-round fix budget.
2. `Invoke-PbipVerify.ps1` is the engine of the screenshot loop.
3. It loads `Test-PbipSemantics.ps1` (checks the source for load errors), `PbipBridge.Common.ps1` (the shared toolbox), and starts `Get-PbipLoadError.ps1` in a separate process when Power BI shows an error dialog.
4. The toolbox calls Microsoft’s two command-line tools: `powerbi-report-author` (used here to validate report files) and `powerbi-desktop` (the Desktop Bridge CLI).
5. `powerbi-desktop` talks to the running **Power BI Desktop**.

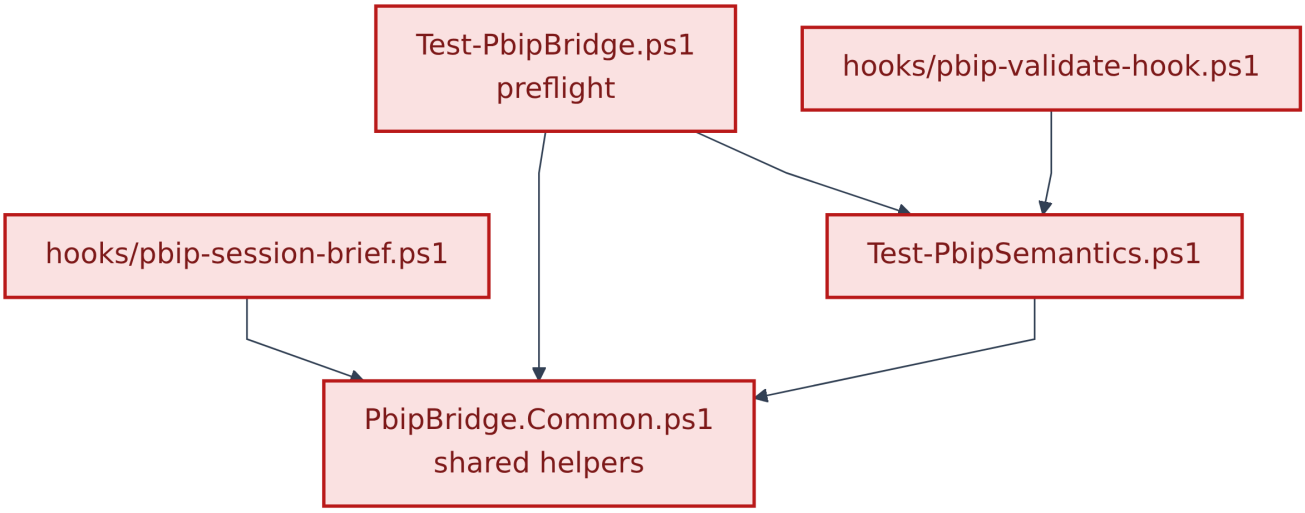


Figure 5.8 — The other scripts that share the toolbox.

Walkthrough of Figure 5.8:

1. **`Test-PbipBridge.ps1`** (preflight) loads the toolbox and the semantic checks.
2. **`hooks/pbip-validate-hook.ps1`** uses `Test-PbipSemantics.ps1` to check one file at a time.
3. **`hooks/pbip-session-brief.ps1`** loads the toolbox only, to read the bridge status quickly.
4. **`Test-PbipSemantics.ps1`** itself loads the toolbox.

Stand-alone scripts need nothing else from the repository, so they have no arrows: `validate-json.ps1`, `validate-pbip.ps1`, `run-ai-md-test.ps1`, `hooks/pbip-write-guard.ps1`, and `Invoke-PbipRefresh.ps1` (which clicks Power BI's Refresh button through Windows UI Automation).

06. Rule layers: steering, specs, hooks, skills, and “who wins”

	1. STEERING the rulebook	2. SPECS the plan for one job	3. HOOKS the automatic checks
Answers	**How** do we always build?	**What** are we building, this once?	**When** does a check run?
Lives in	.github/copilot-instructions.md, copilot-instructions.md, AGENTS.md, .github/instructions/, .github/skills/	specs/ (one file per dashboard or page)	scripts/hooks/ + scripts/Invoke-PbipVerify.ps1
Lifetime	Permanent, versioned, maintained	Disposable: written before the build, then left alone	Permanent; code, not prose
Loads	Every session, automatically	On demand, for one task	Run manually in this template
Authority	Highest	Local to its task only	Deterministic: no judgement
How it fails	Grows too long and stops being read	Gets maintained and becomes a second, conflicting rulebook	Encodes judgement and blocks work it should not
Real-world comparison	The building code	The architect's drawing for one house	The smoke alarm

Source: docs/AI-CONTEXT.md (vocabulary borrowed from Kiro). Skills are procedures inside the steering layer.

Figure 6.1 — The three layers. `docs/AI-CONTEXT.md` explains that this repository deliberately keeps three kinds of guidance apart, and why.

Walkthrough of Figure 6.1:

1. **Steering — the rulebook.** It answers *how* we always build. It lives in `.github/copilot-instructions.md`, `copilot-instructions.md`, `AGENTS.md`, `.github/instructions/` and `.github/skills/`. It is permanent, loaded every session, and has the highest authority. Real-world comparison: the building code.
2. **Specs — the plan for one job.** They answer *what* we are building, this once. One file per dashboard or page in `specs/`. They are written before the build, then left alone. Comparison: the architect’s drawing for one house.

3. **Hooks — the automatic checks.** They answer *when* a check runs. They live in `scripts/hooks/` plus `scripts/Invoke-PbipVerify.ps1`. They are code, not prose, and make no judgement calls. Comparison: a smoke alarm.
4. **How each one fails** is the reason they are kept apart: a rulebook that keeps growing stops being read; a spec that keeps being updated turns into a second rulebook that contradicts the first; a hook that tries to make judgement calls blocks work it should not.

The README also lists the same ideas as five “context and execution layers”: repository policy, scoped instructions, skills, specs, and validation wrappers. That is the same picture with steering split into three parts.

Who wins when rules disagree

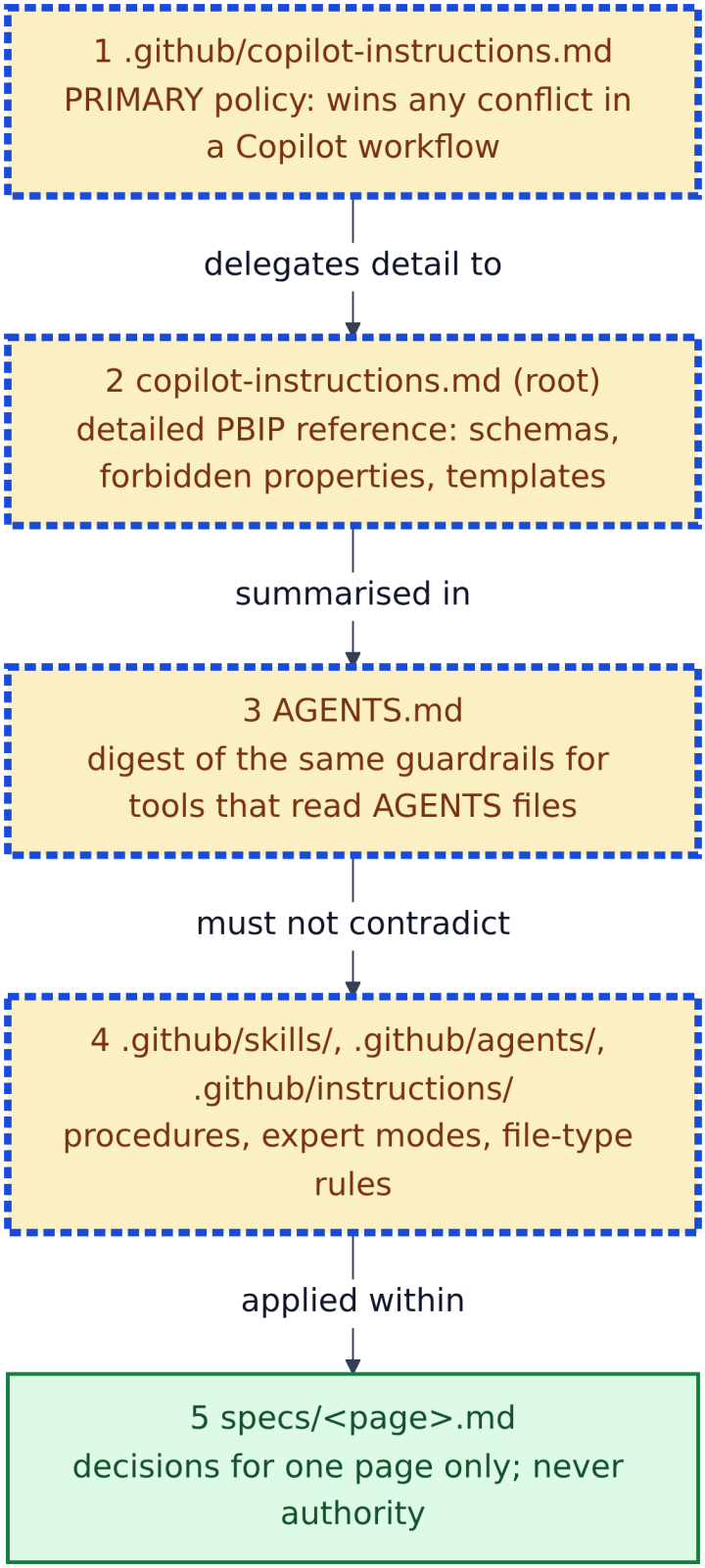


Figure 6.2 — The precedence ladder. The higher box wins any conflict. Read top to bottom.

Walkthrough of Figure 6.2:

1. **`.github/copilot-instructions.md`** is the primary policy for GitHub Copilot work. The README calls it the “Final Boss”: “If there is any conflict … follow `.github/copilot-instructions.md`.”
2. **`copilot-instructions.md` (root)** holds the detailed technical reference: exact file schemas, forbidden properties, JSON templates, the SVG measure pattern and the Python build-script pattern. The primary file delegates this detail to it.
3. **`AGENTS.md`** repeats the same guardrails for tools that read `AGENTS.md` files.
4. **Skills, expert modes and scoped instruction files** apply procedures and topic rules. They must not contradict the files above.
5. **Specs** record decisions for one page only. `specs/README.md` is clear that a spec is “never” authority.

One wrinkle: the three expert-mode files in `.github/agents/` and the `powerbi-dashboard-header` skill say the *root* `copilot-instructions.md` is “the final authority”. Section 16 explains this gap and how to fix it.

Which layer does a new rule belong to?

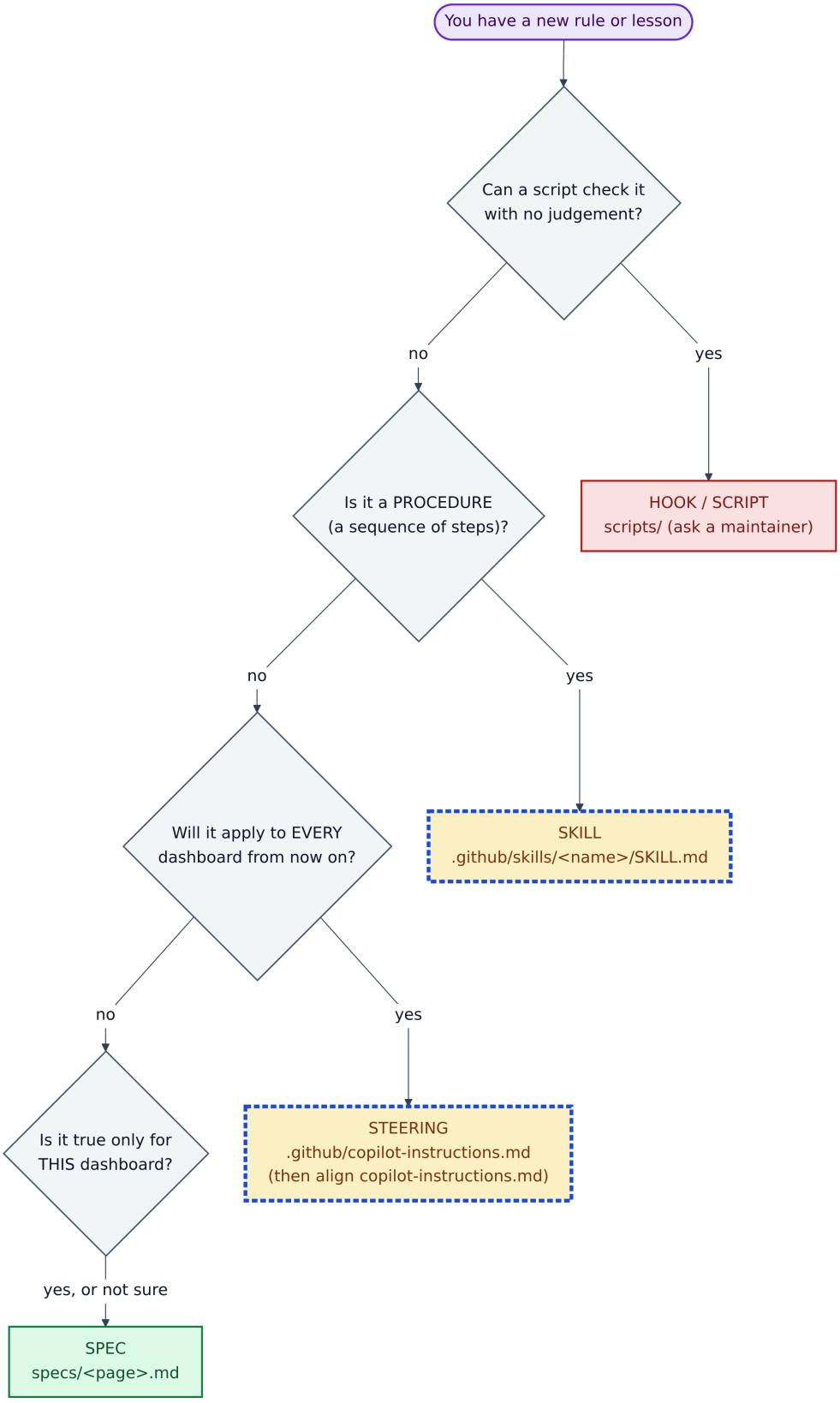


Figure 6.3 — Where to put a new rule. Start at the top and answer each question. This tree follows the four questions at the end of `docs/AI-CONTEXT.md`.

Walkthrough of Figure 6.3:

1. **Can a script check it with no judgement?** Then it should be a hook or script: “If a machine can check it, do not spend context asking an agent to remember it.” Ask a maintainer, because scripts are Red.
2. **Is it a procedure**, a sequence of steps someone follows? Then it is a skill: `.github/skills/<name>/SKILL.md`. The skill’s `description` line is what makes the agent pick it up.
3. **Will it apply to every dashboard from now on?** Then it is steering: add it to `.github/copilot-instructions.md`, then keep `copilot-instructions.md` in step.
4. **Is it true only for this dashboard**, or are you unsure? Put it in the spec.

The life of a spec

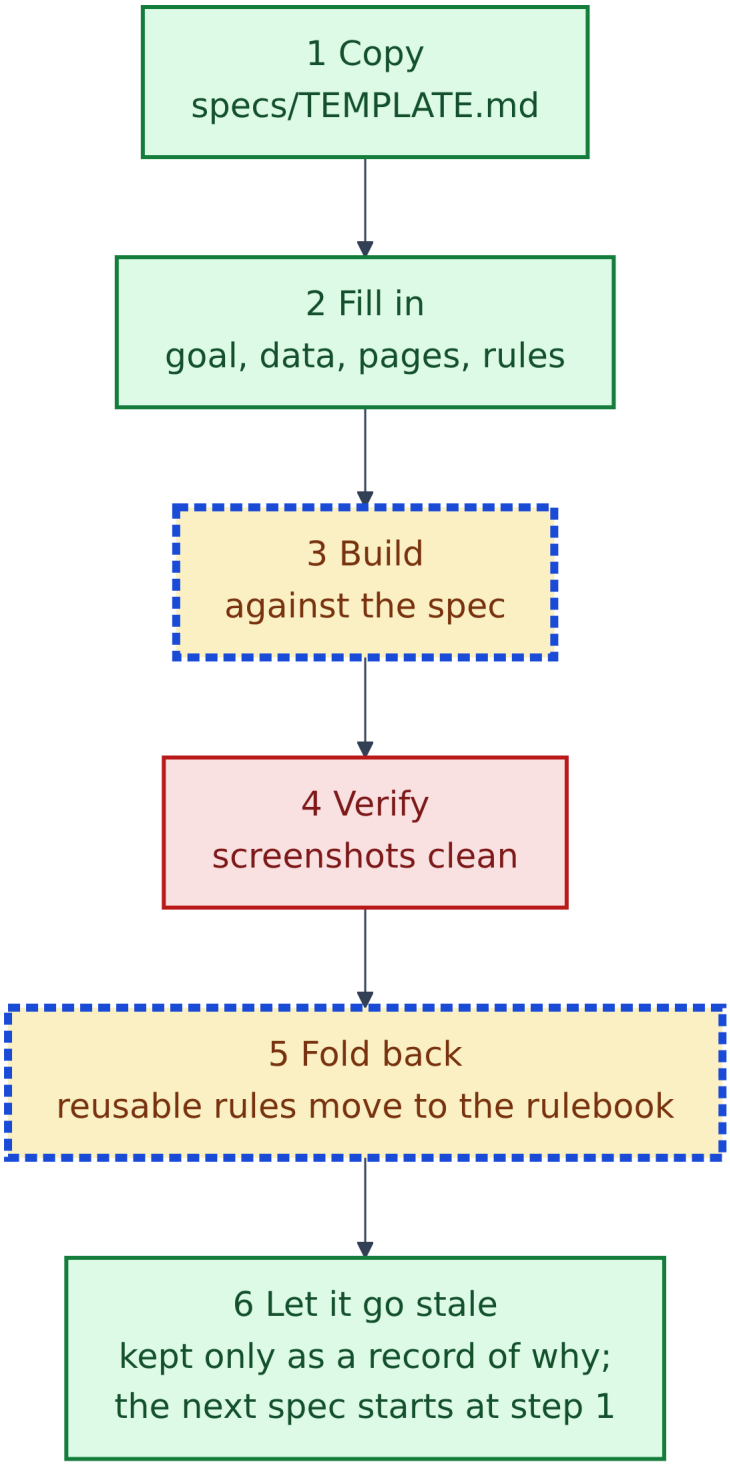


Figure 6.4 — The spec lifecycle, from `specs/README.md`.

Walkthrough of Figure 6.4:

1. **Copy** `specs/TEMPLATE.md` to `specs/<dashboard-or-page-name>.md` (lower-case words joined by hyphens).
2. **Fill in** the eight sections: goal, data required, pages, rules that apply, build tasks, definition of done, open questions, fold-back.

3. **Build** against the spec.
4. **Verify** that every page renders cleanly.
5. **Fold back:** move any rule that will matter for the next dashboard into the rulebook. The worked example `specs/EXAMPLE-sales-overview.md` folded back two rules: visuals must sit at least 30 pixels below their card title (`card_y + 30`), and a visual labelled “Top 10” must really be filtered to 10.
6. **Let it go stale.** Keep the spec as a record of *why*, but do not keep updating it. The next dashboard starts with a fresh spec at step 1.

When a spec is required (`specs/README.md`): a new dashboard or page; a new fact table or shared set of measures; any rebuild that changes a page’s data. **Not required** for a colour tweak, a label fix or a measure rename.

How specs differ from `tests/prompts/`: test prompts are standing instructions reused unchanged (“execute the instructions in `PowerBI_Test.md` exactly”). A spec is written once for one deliverable.

Hooks in plain words

A **hook** is a small script that runs at a fixed moment, like a smoke alarm that checks the air every time a door opens. This repository has four, in `scripts/hooks/`. The README says they are run manually in this template, and all of them **fail open**: if anything unexpected happens, they let you carry on rather than lock you out.

Hook script	Moment	What it does
`pbip-session-brief.ps1`	Start of a session	Prints the guardrails and whether the Desktop Bridge is up
`pbip-write-guard.ps1`	Before a file is written	Blocks writes to `Proposals/`, `**/.pbi/`, `tests/screenshots/`, `.github/.pbip-state/`
`pbip-validate-hook.ps1`	After a file is written	Checks that one file: JSON parses, filter expressions are well formed, TMDL has no BOM, no `//` comments, no measure/column name clash
`pbip-stop-hook.ps1`	Before a turn finishes	Runs the verify loop if anything changed, and makes the agent review the screenshots

`docs/AI-CONTEXT.md` adds an honest limit: the write guard only sees the agent’s file-writing tools, so commands typed in a shell can bypass it. “It is a guardrail, not a sandbox.”

07. Local setup checklist

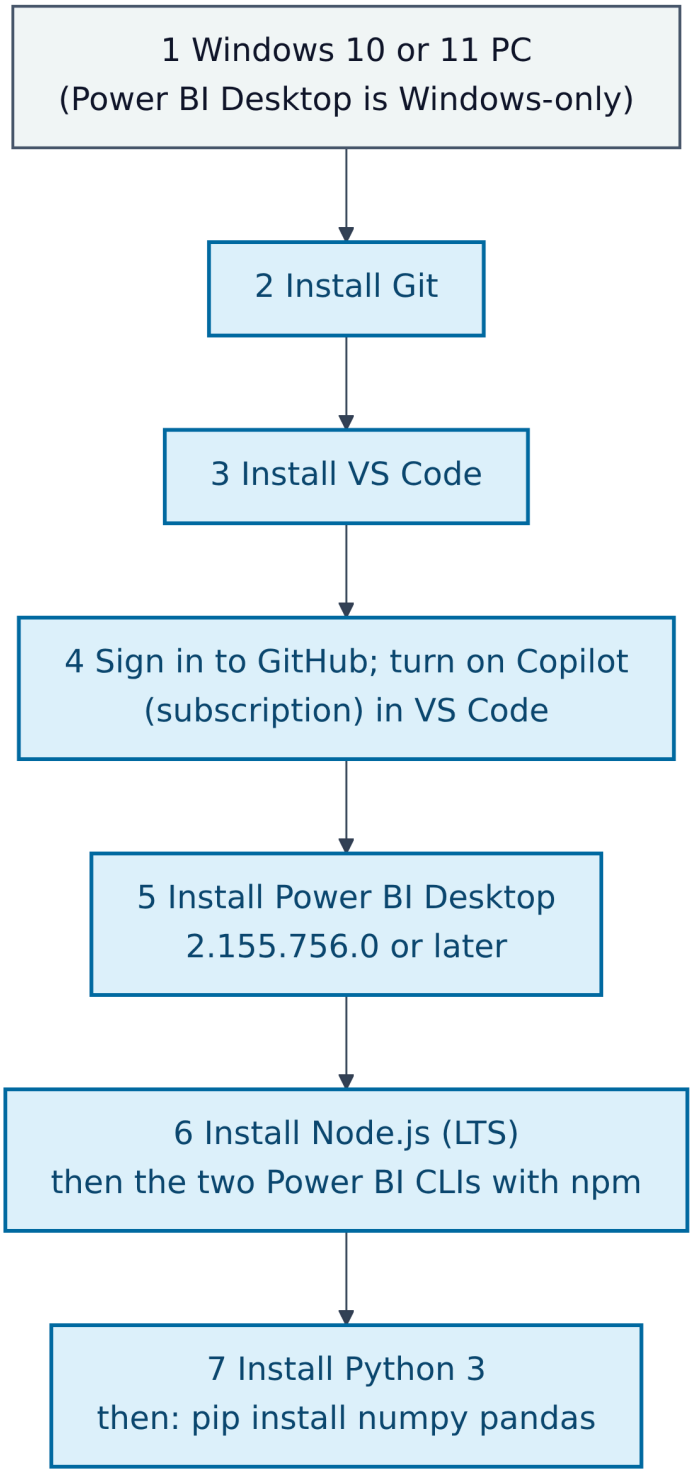


Figure 7.1 — Setup, part 1: install the tools. Do these once per computer, top to bottom.

Walkthrough of Figure 7.1:

1. **A Windows 10 or 11 computer.** Microsoft lists only Windows (“Windows 10 or Windows Server 2016 or later”) for Power BI Desktop, and every PowerShell script here targets Windows PowerShell 5.1, which is built into Windows.

2. **Install Git**, the version-history tool.
3. **Install VS Code**, the editor. It has Git built in, but Git itself must be installed separately (step 2).
4. **Sign in to GitHub and turn on GitHub Copilot** in VS Code. Copilot needs a GitHub Copilot subscription.
5. **Install Power BI Desktop 2.155.756.0 or later.** The README says older builds have no Desktop Bridge. Use the Microsoft Store or Microsoft's Download Center; Microsoft recommends the 64-bit version.
6. **Install Node.js** (the current LTS, "long-term support", version), then the two Power BI command-line tools with npm (Node's package installer).
7. **Install Python 3**, then the two libraries guardrail 8 asks for: numpy and pandas. No virtual environment is needed, according to the guardrail.

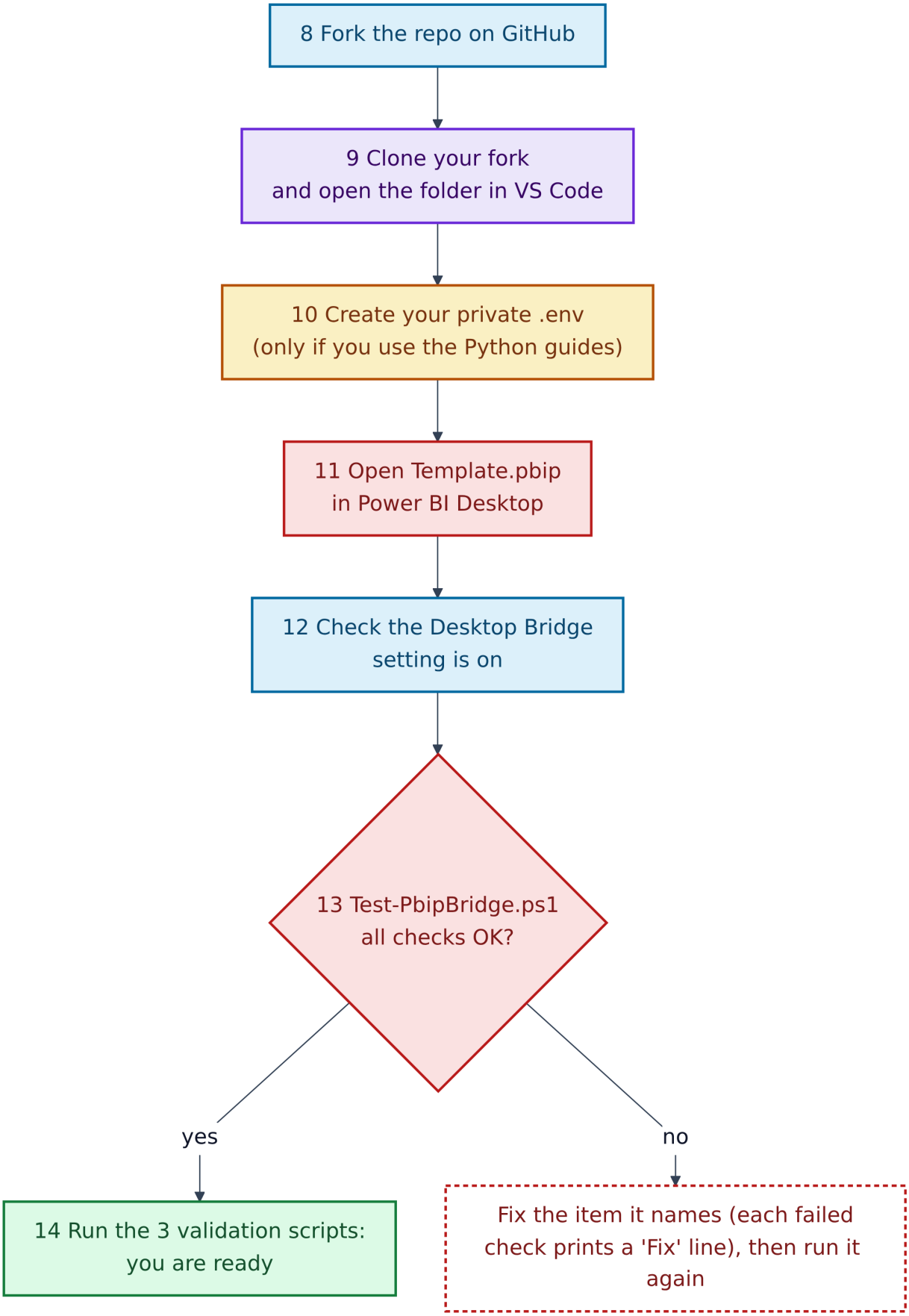


Figure 7.2 — Setup, part 2: get the repository and prove your setup works. Continues from step 7.

Walkthrough of Figure 7.2:

8. **Fork** `GibbonGambits/PGE-PBIP-Template` on GitHub (the *Fork* button, top right of the repository page).
9. **Clone your fork** and open the whole folder in VS Code.
10. **Create your private `.env` file**, but only if you will use the Python email and alert guides. Figure 7.4 shows the safe way.
11. **Open `Template.pbip`** in Power BI Desktop.
12. **Check that the Desktop Bridge setting is on** (see the box below).
13. **Run the preflight**, `scripts\Test-PbipBridge.ps1`. If any check fails, it prints a `Fix` line; do that and run it again.
14. **Run the three validation scripts.** When they pass, you are ready.

> **Where is the Desktop Bridge setting?** Microsoft's current page, "What is the Power BI Desktop Bridge?", says: *File > Options and settings > Options > **Security** > Desktop Bridge*, then "Enable external tool access to Power BI Desktop through secure local APIs", and that "this setting is on by default". This repository's README and `copilot-instructions.md` place the same checkbox under **Preview features**. Look in Security first; if your build shows it under Preview features, use that. Section 16 lists this as a documentation gap.

Which tool is needed for what

Task	Git	VS Code	Copilot	Power BI Desktop	Node.js + 2 CLIs	Windows PowerShell	Python 3	pbiviz tools
Save and share work (Git, GitHub)	✓	✓						
Ask the AI agent to build a page		✓	✓					
Open and look at the report				✓				
Run validate-json / validate-pbip						✓		
Run Test-PbipSemantics						✓		
Run the verify loop (screenshots)				✓	✓	✓		
Generate fake CSV data (guardrail 8)							✓	
Python email / alert guides							✓	
Build a custom visual (Section 12)				✓	✓	✓	✓	✓
AI test on GitHub (Section 11)								

A tick means the task needs that tool on your PC. The AI test on GitHub runs on GitHub's own computers, so it needs nothing locally, only a repository secret (Section 11). The custom-visual kit needs Node.js 18 or later.

Figure 7.3 — Tool and task matrix. A tick means the task needs that tool on your computer.

Walkthrough of Figure 7.3:

1. **Git + VS Code** are enough to save and share work.
2. **VS Code + Copilot** are what you need to ask an agent to build a page.
3. **Power BI Desktop** is needed to look at the report.
4. **Windows PowerShell** alone runs the three validation scripts. They need no other tools.
5. **The verify loop** needs Power BI Desktop, Node.js with the two CLIs, and PowerShell together.
6. **Python 3** is needed for fake data (guardrail 8) and for the Python email and alert guides.
7. **A custom visual** needs everything plus the `pbiviz` tools (Section 12). The custom-visual skill asks for Node.js 18 or later.
8. **The AI test on GitHub** runs on GitHub's own computers, so it needs nothing on yours, only a repository secret (Section 11).

The checklist, with commands

Tick each box as you go. Commands are for **Windows PowerShell**, run from the repository folder (VS Code: *Terminal > New Terminal*).

Tools

- ☐ **Git** installed. Check: `git --version`
- ☐ **VS Code** installed, with the GitHub Copilot extension signed in.
- ☐ **Power BI Desktop** 2.155.756.0 or later. Check in Desktop: *Help > About*.
- ☐ **Node.js** installed. Check: `node --version`
- ☐ **The two Power BI CLIs**, exactly as the README gives them:

```
npm install -g @microsoft/powerbi-desktop-bridge-cli@latest
```

```
npm install -g @microsoft/powerbi-report-authoring-cli@latest
```

- ☐ Check them: `powerbi-desktop --version` and `powerbi-report-author --version` (the same check `Test-PbipBridge.ps1` runs).
- ☐ **Python 3** with numpy and pandas:

```
pip install numpy pandas
```

- ☐ **Optional, for the Python guides:** `pip install python-dotenv pyodbc` (the libraries `python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md` lists, along with pandas).
- ☐ **Optional, for custom visuals:** `npm i -g powerbi-visuals-tools` (Section 12).

Optional: GitHub Copilot CLI and the Fabric skills. The README asks for a one-time step in **GitHub Copilot CLI** (Copilot in a terminal window). GitHub's install page offers, for example:

```
winget install GitHub.Copilot
```

GitHub notes Copilot CLI needs PowerShell 6 or later on Windows (the newer, separately installed "PowerShell 7"), or Node.js 22+ for the npm install route. Then, inside Copilot CLI, type the README's commands:

```
/plugin marketplace add microsoft/skills-for-fabric
/plugin install powerbi-authoring@fabric-collection
/skills list
```

The `microsoft/skills-for-fabric` repository confirms the first two commands. The README says `/skills list` should then show `powerbi-report-cli` and `semantic-model-authoring`.

The repository

- ☐ Forked on GitHub, cloned, and opened as a folder in VS Code.
- ☐ Preflight passes:

```
powershell -NoProfile -File scripts\Test-PbipBridge.ps1
```

It checks eight things: the PBIP project is found; Power BI Desktop is new enough; both CLIs are on the PATH (the list of folders Windows searches for commands); the bridge pipe exists; a running Desktop has *this* project open; the semantic checks are clean; the report files validate.

- ☐ The three validators pass:

```
powershell -NoProfile -File scripts\validate-json.ps1
```

```
powershell -NoProfile -File scripts\validate-pbip.ps1
```

```
powershell -NoProfile -File scripts\Test-PbipSemantics.ps1
```

Creating your private `.env` file safely

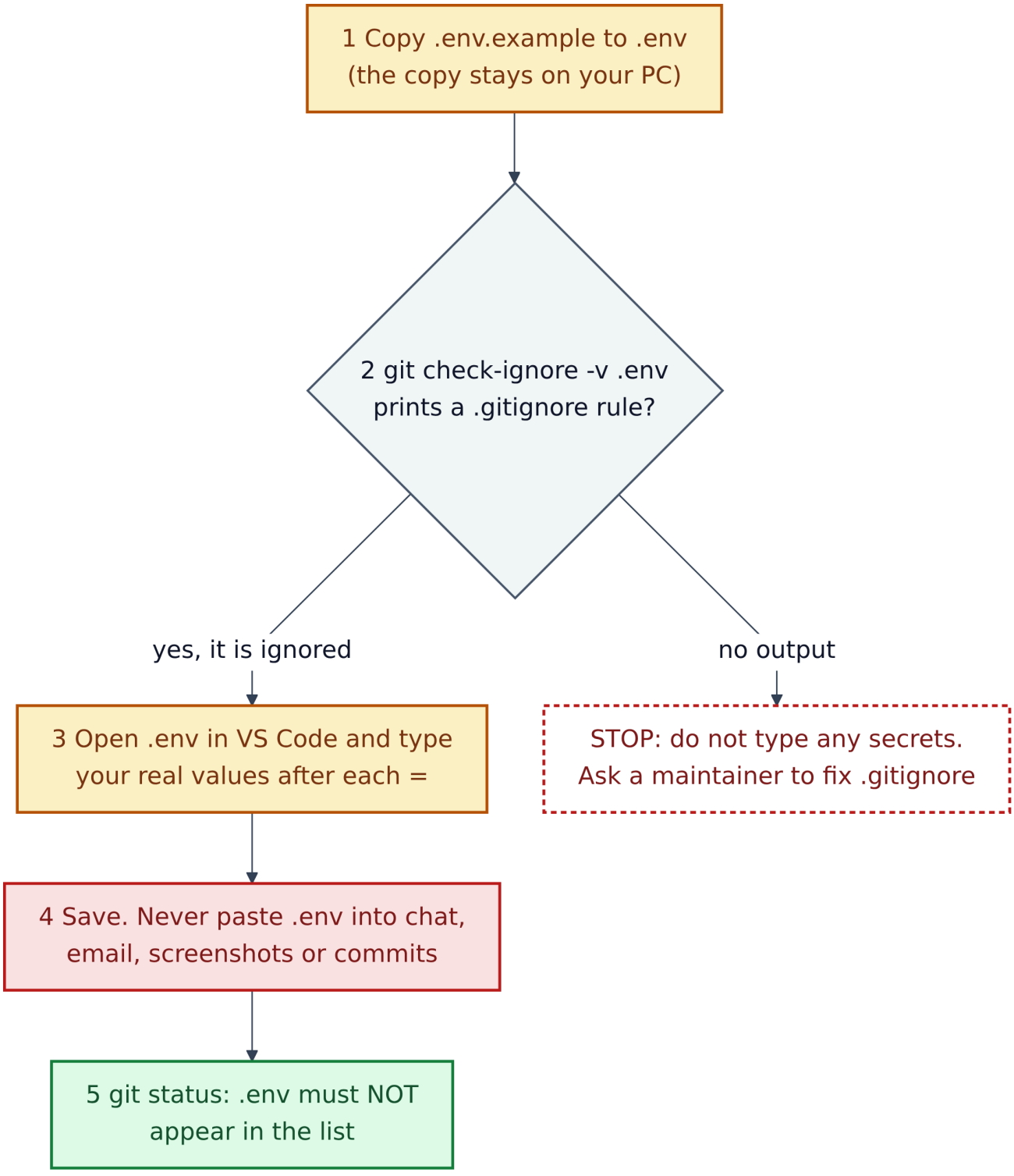


Figure 7.4 — Creating `.env` without exposing secrets. A **`.env` file** is a plain text file of `NAME=value` lines that programs read for settings such as passwords. It must never leave your computer.

Walkthrough of Figure 7.4:

1. **Copy the example.** In PowerShell, from the repository folder:

```
Copy-Item .env.example .env
```

`.env.example` contains only names: `DB_SERVER`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_DRIVER`, `SMTP_EMAIL`, `SMTP_PASSWORD`. Its first lines say: “Copy this file to .env and fill in real values. .env is gitignored - never commit real credentials.”

2. **Prove Git will ignore it** before you type any secret:

   ```
   git check-ignore -v .env
   ```

   It should print the `.gitignore` line that matches, such as `.gitignore:4:.env .env`. This repository’s `.gitignore` ignores `.env` and every `.env.*` file except `.env.example`. **If nothing is printed, stop** and ask a maintainer.

3. **Type your real values** after each `=` in VS Code. Never edit `.env.example` with real values: it *is* saved in Git.

4. **Save, and keep it private.** Never paste `.env` into a chat (including an AI chat), an email, a screenshot, an issue or a pull request.

5. **Double-check** with `git status`. `.env` must not appear in the list of changed files.

How the values are used: the Python guides load the file with the `python-dotenv` library (`load_dotenv()`), then read each value with `os.environ['DB_SERVER']` and so on. Nothing else in the repository reads `.env`.

Secrets for GitHub are different. Automated runs on GitHub never see your `.env`. They use **repository secrets**, set in the repository’s *Settings > Secrets and variables > Actions*, and read in workflows as `${{ secrets.NAME }}`. Section 11 lists the ones this repository uses.

08. Building a dashboard page from start to finish

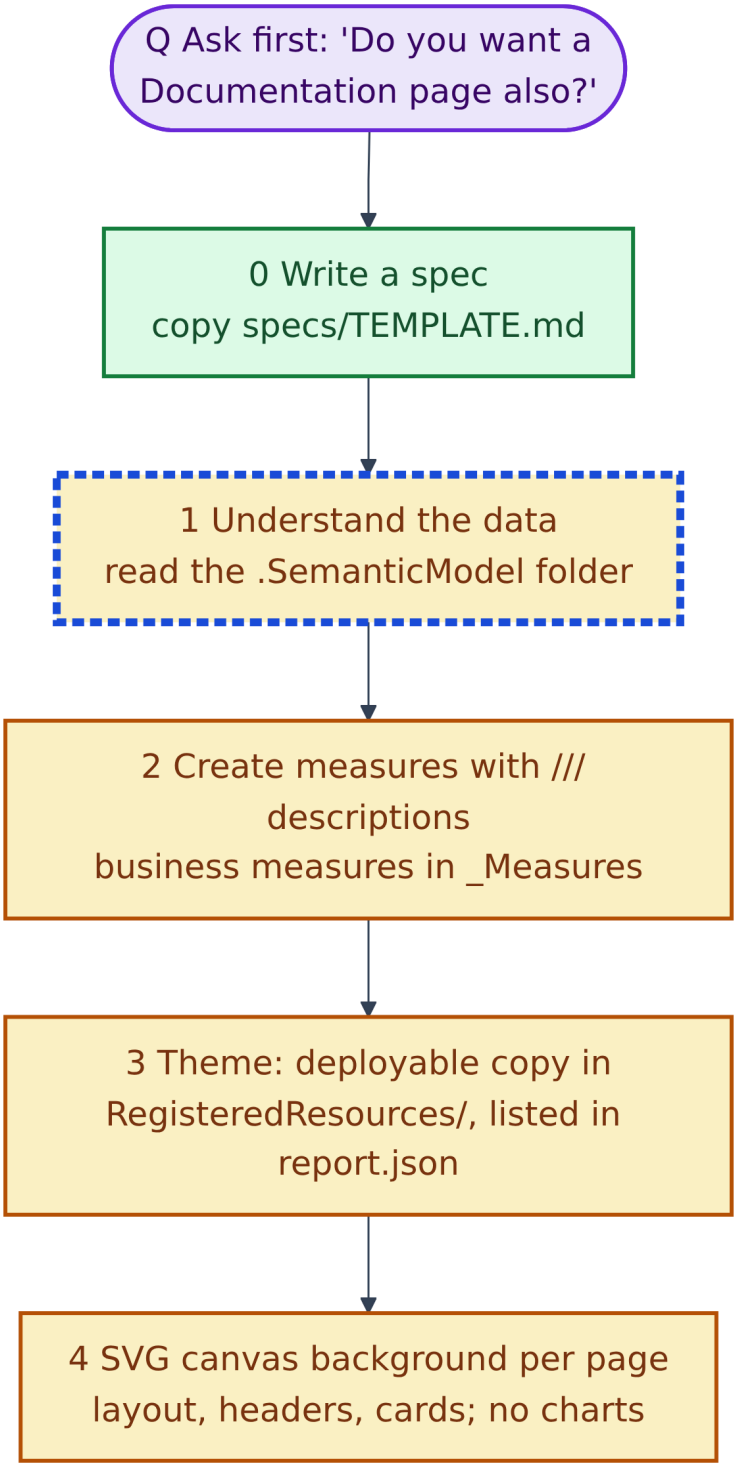


Figure 8.1 — Build steps, part 1: from question to background. These are the “Dashboard Creation Steps” in `copilot-instructions.md`, plus the question the rulebook says to ask first.

Walkthrough of Figure 8.1:

- **Q. Ask first.** Before generating pages on a first request, the agent must ask: “Do you want a Documentation page also?” If yes, it adds a page called `Documentation` whose background is an SVG summary of the project (see “The Documentation page” below). If no, it does not ask again in that conversation.

0. **Write a spec.** Copy `specs/TEMPLATE.md` to `specs/<name>.md` and fill it in (Section 06). “Decisions … are cheap to change in a spec and expensive to change in generated PBIR JSON.”
1. **Understand the data.** Read the `.SemanticModel` folder: tables, columns, relationships, measures. This step is Amber with a blue outline because the agent does it, using the rule files.
2. **Create measures.** Give every measure a `///` description line. Business measures go in the `_Measures` table, created if missing as a one-row placeholder table. (The rulebook writes this as `_Measures = BLANK{}`; the DAX table syntax it uses for `_SVG_Measures`, `{ BLANK() }`, is the correct form. Section 16 lists this typo.)
3. **Create or register the theme.** The deployable theme file goes in `StaticResources/RegisteredResources/` and is listed in `report.json` (Section 10).
4. **Create an SVG canvas background for each page. SVG** (Scalable Vector Graphics) is a text-based picture format. The background holds the layout, headers, card boxes and card titles, but no charts. Comparison: the printed paper form; the charts are the handwriting on it.

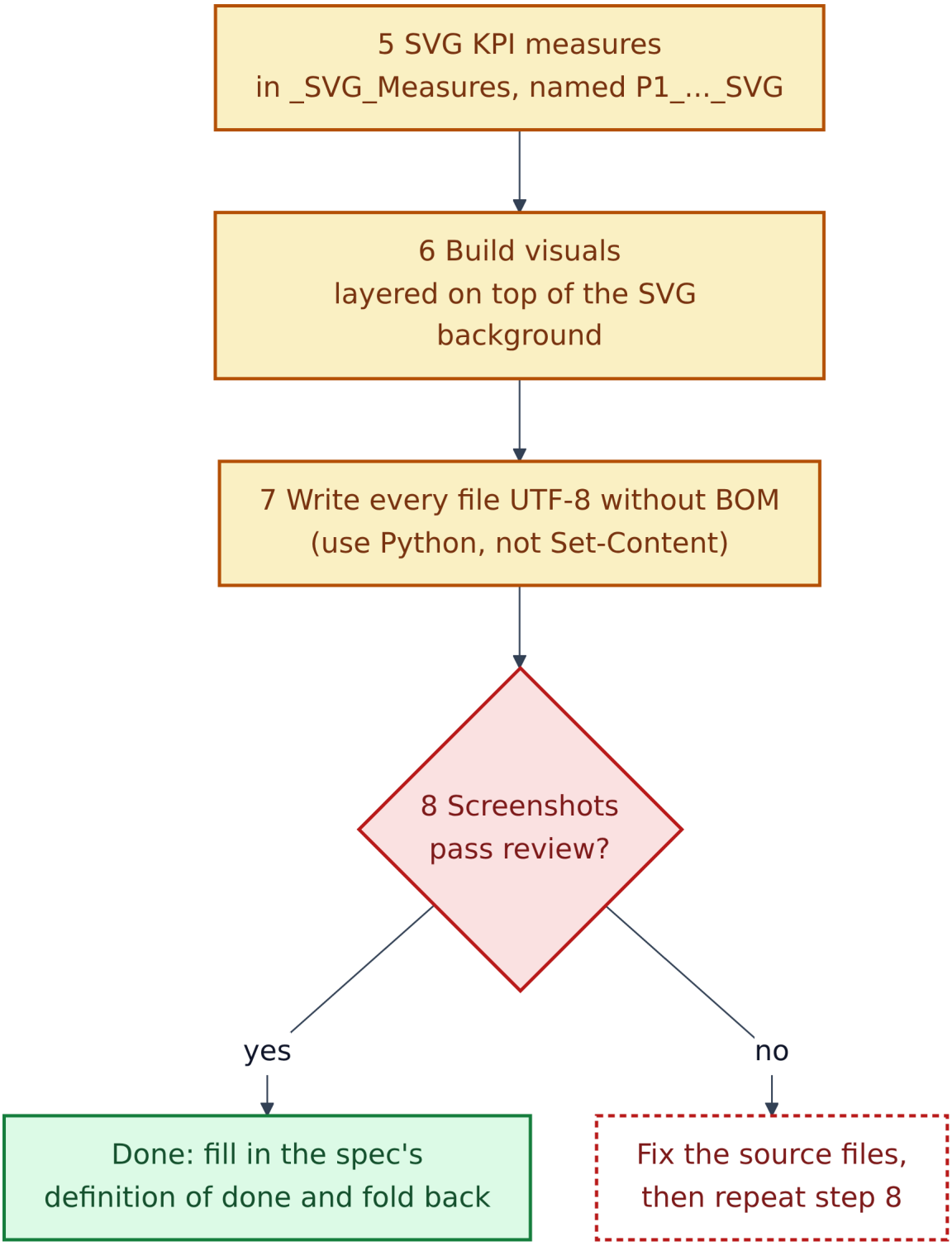


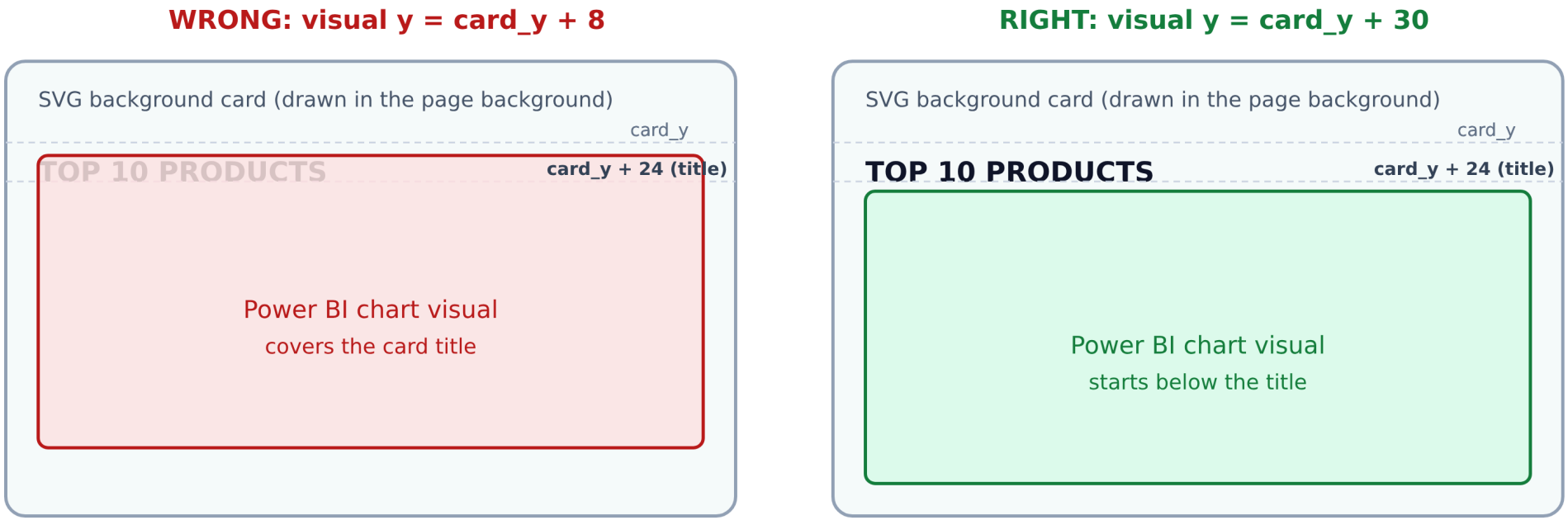
Figure 8.2 — Build steps, part 2: from measures to proof. Continues from step 4.

Walkthrough of Figure 8.2:

5. **Create SVG KPI measures** in the separate `_SVG_Measures` table, named by page: `P1_Sales_KPI_SVG`, `P2_Profit_KPI_SVG`. A **KPI** (key performance indicator) card shows one headline number.
6. **Build the visuals**, layered on top of the SVG background.

7. **Write every file as UTF-8 without a BOM.** The rulebook says to use Python, `open(f, 'w', encoding='utf-8')`, and never PowerShell's `Set-Content -Encoding UTF8`, which adds a BOM.
8. **Verify what actually rendered** (Section 09). If the screenshots pass, you are done: tick the spec's definition of done and fold back reusable rules. If not, fix the source files and repeat step 8.

How a page is layered



Rule 16 in copilot-instructions.md: the title is drawn at card_y + 24, so a visual must start at card_y + 30 or lower. Example: card at y=192 means visual at y=222 or later.

Figure 8.3 — The single most common layout defect. Left: wrong. Right: right.

Walkthrough of Figure 8.3:

1. The **SVG background** draws each card's box and title. The title is drawn at `card_y + 24` (24 pixels below the top of the card).
2. A **Power BI visual** sits on top of the background. If its top edge (`y`) is too high, it covers the title.
3. **Rule 16** in `copilot-instructions.md`: set the visual's `y` to at least `card_y + 30`. Example: card at `y=192` → visual at `y=222` or later.
4. `specs/TEMPLATE.md` calls this "the single most common rendering defect in this template", and JSON validation cannot see it. Only a screenshot can.

Because titles come from the background, the default style removes each visual's own title, background, border and shadow. `.github/copilot-instructions.md` calls this **SVG-header mode** (the default). **Visual-title mode** (visuals carry their own titles) is allowed only when the spec records it as an exception.

Where does a new measure go?

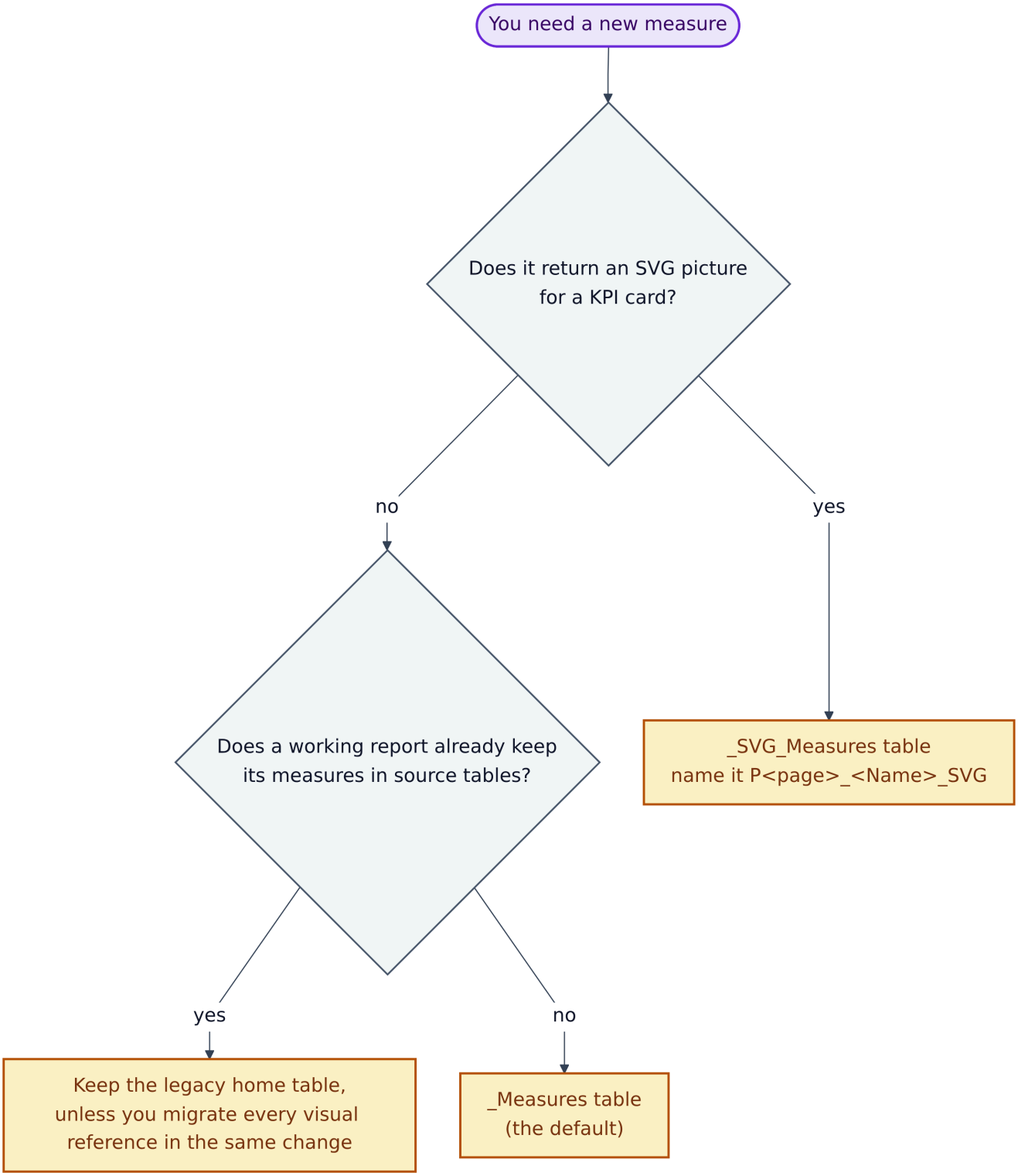


Figure 8.4 — Choosing a measure's home table (guardrails 1 and 2, and step 2 of the creation steps).

Walkthrough of Figure 8.4:

1. **Does the measure return an SVG picture for a KPI card?** Then it goes in `_SVG_Measures`, named `P<page number>_<Name>_SVG`.

2. **Otherwise, does an existing working report already keep its measures in source tables** (for example `Fact-MaterialMaster`)? Then keep them there ("legacy-safe mode"), unless you move every visual reference in the same change.
3. **Otherwise**, use `_Measures`, the default.

The Python build-script order

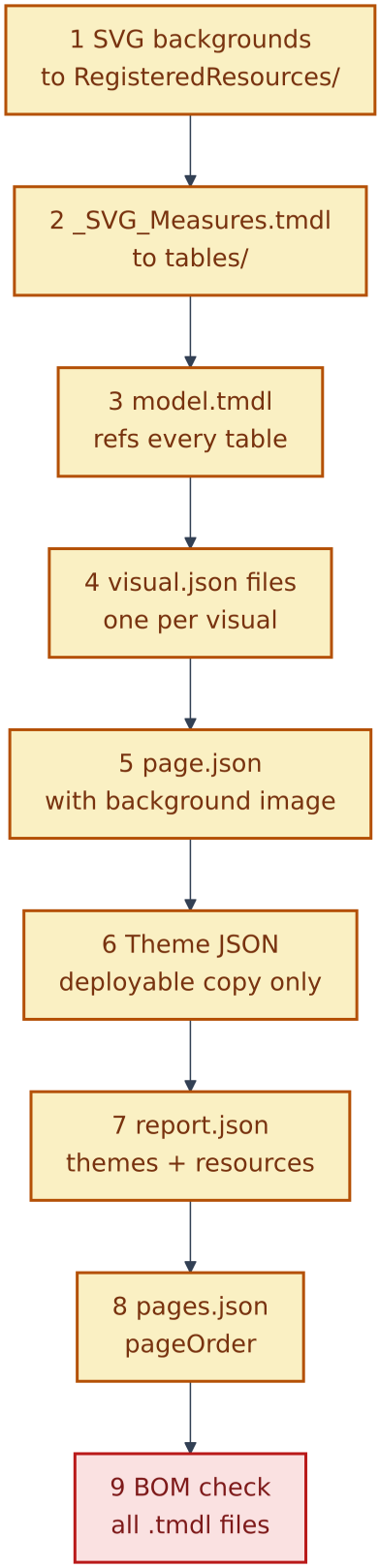


Figure 8.5 — The nine-step order for a Python build script. `copilot-instructions.md` recommends one Python script that writes everything in a single run.

Walkthrough of Figure 8.5:

1. SVG backgrounds → `RegisteredResources/`.
2. `_SVG_Measures.tmdl` → `tables/`.
3. `model.tmdl`, which must reference every table, including `_SVG_Measures`.
4. Each visual's `visual.json`.
5. Each `page.json`, with its background image.
6. The theme JSON (deployable copy only, valid top-level keys only).
7. `report.json`, with `themeCollection` and `resourcePackages`.
8. `pages.json`, with `pageOrder`.
9. A final check that every `.tmdl` file is UTF-8 without a BOM.

The script's helper functions (`write_utf8`, `write_json`, `col_field`, `meas_field`) and the visual schema address (`visualContainer/2.7.0`) are in the same section of `copilot-instructions.md`.

The formatting house style (summary)

`copilot-instructions.md` lists 16 "General Dashboard Formatting Rules". In plain words:

Rule	Meaning
Sensible number formats	Whole numbers by default; decimals, currency or percent when justified. Thousand separators everywhere.
Clean names	No "Sum of", no underscores in tooltips or column headers.
Data labels on	Every chart shows its values; full numbers, not K/M, unless the spec says so.
Sort	Descending by value, except months: January to December, shown as "Jan", sorted by a month-number column.
No chrome	No background, border, shadow or title on any visual (titles are in the SVG).
Axes	No axis titles; value axis hidden; category axis shown.
"Top N" means it	If an SVG label says "Top 10", the visual must be filtered to the top 10.
Position	Visual `y` at least `card_y + 30` (Figure 8.3).

The same file gives complete JSON templates for bar, line, donut and table visuals, and a section of **forbidden properties** that make Power BI reject a file. Examples: `"filters": []` at the top of a visual; `"selector": null`; a `sort` block inside `visual` (use `query.sortDefinition` instead); `wallpaper` in `page.json`. Never "tidy up" JSON by adding these.

SVG KPI cards in brief

- They are **image visuals** whose picture comes from a measure: `sourceType = 'imageData'`, with the measure named in `sourceField` (guardrail 7). They have no `query` section.
- The measure returns text that starts with `data:image/svg+xml;utf8,<svg...` (never base64).

- The rulebook suggests five sections inside the DAX: context and timeline; base metrics; derived metrics (maths only); layout and colours; assemble the SVG.
- Colours as `rgb(...)` or `#HEX`, font `Segoe UI`, transparent background, positions wrapped in `INT()`, and no headings inside the picture (the heading is on the background card).

The Documentation page

If you answer “yes” to the opening question, the page follows the pattern in `copilot-instructions.md`: display name `Documentation`, size 816 × 2740 at actual size, surround colour `#FBF7F2`, and an SVG background that summarises the project inventory, the model, the report, the resource packages and a timestamp. The ready-made prompt `tests/prompts/PBIP_Documentation_Page.md` builds it in seven phases.

The spec’s definition of done

From `specs/README.md` and `specs/TEMPLATE.md`, a page is done only when:

- [ ] `validate-json.ps1`, `validate-pbip.ps1` and `Test-PbipSemantics.ps1` pass.
- [ ] The PBIP opens in Power BI Desktop with no error dialog, and the window title shows the project, not *Untitled*.
- [ ] `powerbi-desktop status` reports the project path.
- [ ] The verify loop has run, and every page screenshot has been reviewed against `.github/skills/powerbi-visual-verify/SKILL.md`.
- [ ] No visual overlaps its card title; no truncated labels; no blank charts.
- [ ] Findings are reported by page display name.

09. Checks and the screenshot (verify) loop

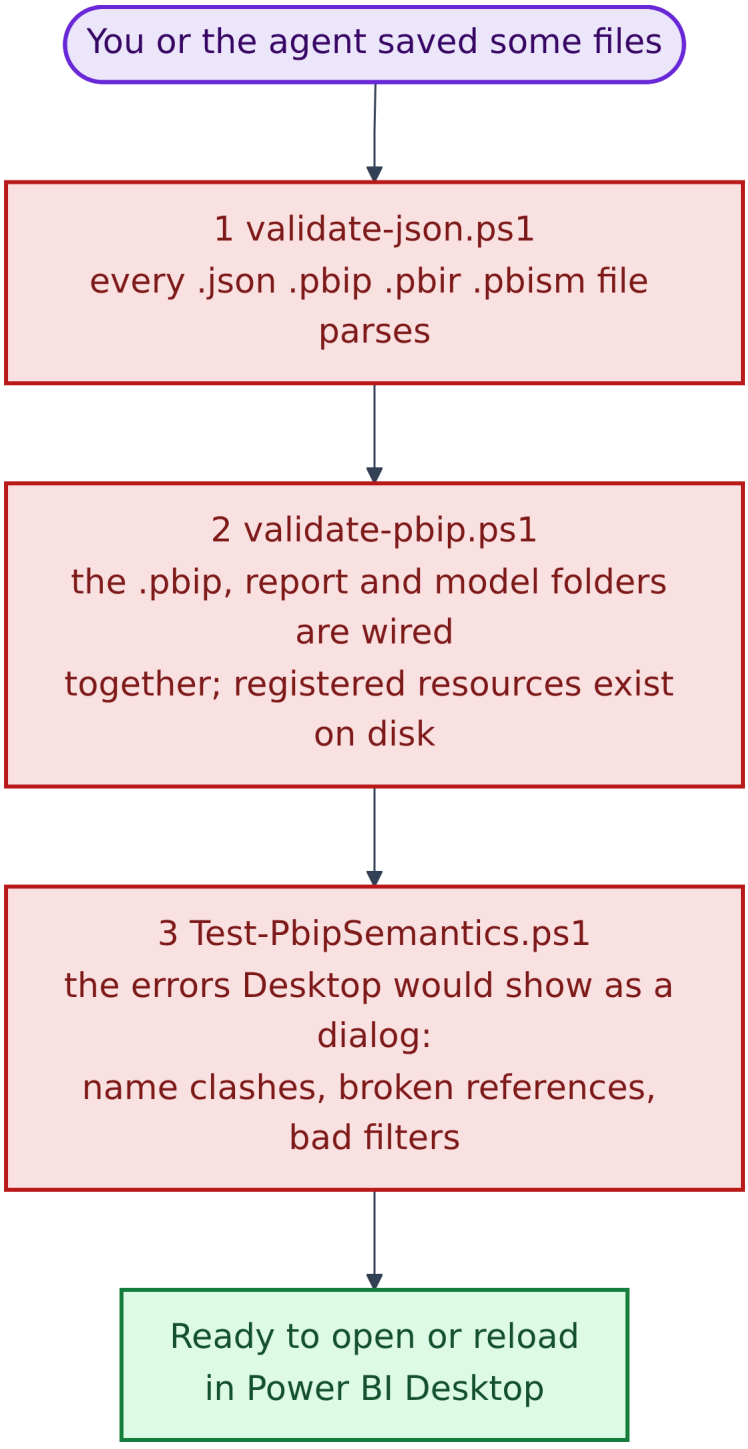


Figure 9.1 — The three validators, cheapest first. All three run in a few seconds, need only Windows PowerShell, and never open Power BI.

Walkthrough of Figure 9.1:

1. **`validate-json.ps1`** opens every `.json`, `.pbip`, `.pbir` and `.pbism` file and checks it parses. It skips other people's code (`node_modules`, `dist`, `.tmp`, `.vscode`, `.git`) and two files that are allowed to break plain JSON rules (`package-lock.json`, and `tsconfig.json`, which may contain comments).
2. **`validate-pbip.ps1`** follows the pointer chain from Section 02: each `.pbip` → its report folder → `definition.pbir` → the semantic model. It checks the required files exist (`report.json`, `pages.json`, `database.tmdl`, `model.tmdl`), that every resource listed in `report.json` exists on disk, and that every custom visual a page uses is registered. Use `-Project <name>` if the folder holds more than one project.
3. **`Test-PbipSemantics.ps1`** finds, in the text, the errors Power BI Desktop would otherwise show as a pop-up:
   - **checks on one file at a time** (safe to run on every save): a measure and a column with the same name in one table; duplicate column or measure names; a filter whose expression key is misspelled (it suggests the fix, e.g. `Inn` → `In`);
   - **checks across files** (run when the model is complete): measure names unique across the whole model; relationships pointing at tables or columns that exist; every `'Table'[Column]` reference resolves; every resource used is registered; no duplicate page ids.

   Add `-Path <file>` to check one file.
4. **Ready.** Only now is it worth opening or reloading Power BI Desktop.

```
powershell -NoProfile -File scripts\validate-json.ps1
```

```
powershell -NoProfile -File scripts\validate-pbip.ps1
```

```
powershell -NoProfile -File scripts\Test-PbipSemantics.ps1
```

Why `-NoProfile`? It starts PowerShell without your personal start-up settings, so the script behaves the same on every machine.

The verify loop, step by step

The README's one-line summary: "edit PBIR/TMDL → validate → reload → screenshot every page → review → fix → repeat". The engine is `scripts\Invoke-PbipVerify.ps1`. Its own code is divided into numbered steps, shown here in two parts.

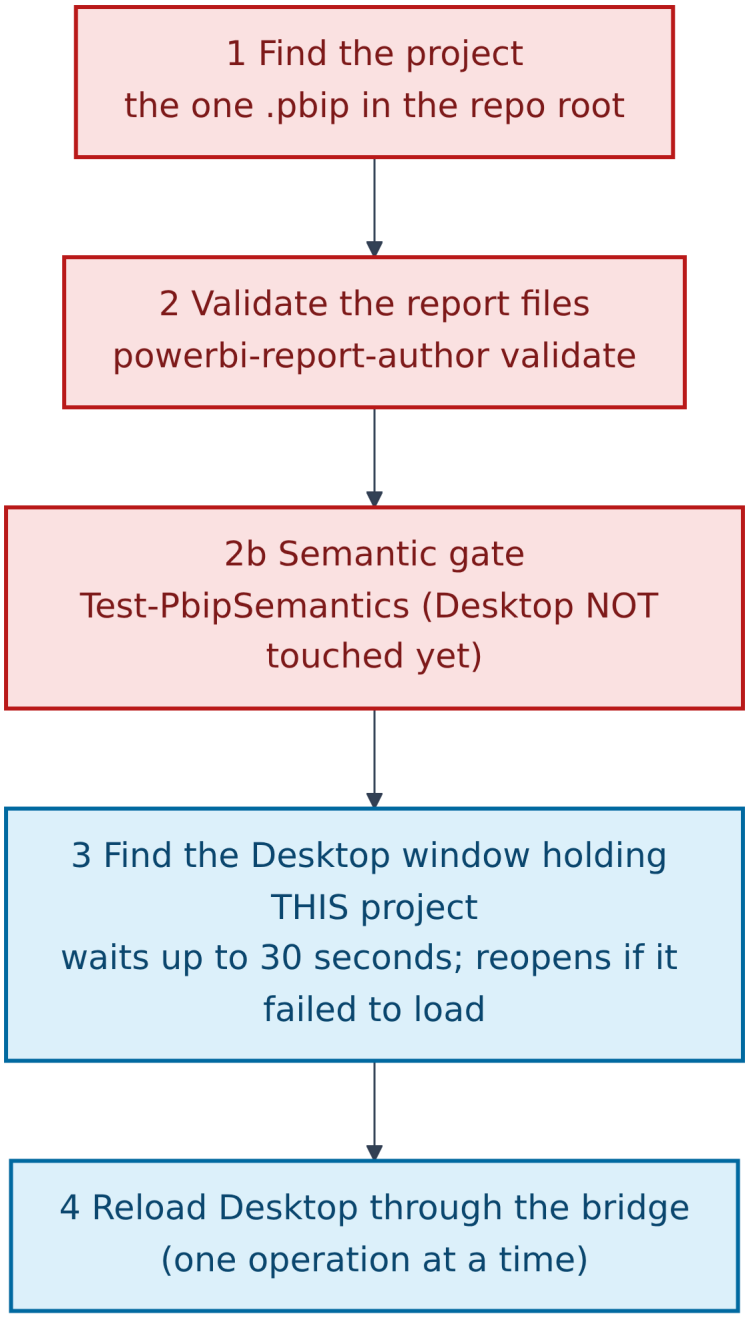


Figure 9.2 — Verify loop, part 1: before anything is drawn.

Walkthrough of Figure 9.2:

1. **Find the project:** the one `.pbip` file in the repository root. The scripts never assume it is called `Template`, because the template is meant to be renamed.
2. **Validate the report files** with `powerbi-report-author validate`. Invalid files are refused here, which "saves a Desktop cycle and gives a far better error".
    - **2b. Semantic gate:** `Test-PbipSemantics` runs before Desktop is touched.
3. **Find the Desktop window that has *this* project open.** It waits up to 30 seconds for a slow load. If the project failed to load, it captures the error dialog, closes that spent window, and reopens the project once. It only ever closes a window it started, or one that was stuck on an error.

4. **Reload Desktop through the bridge.** Bridge calls go strictly one at a time; overlapping calls fail with “Cancelled”.

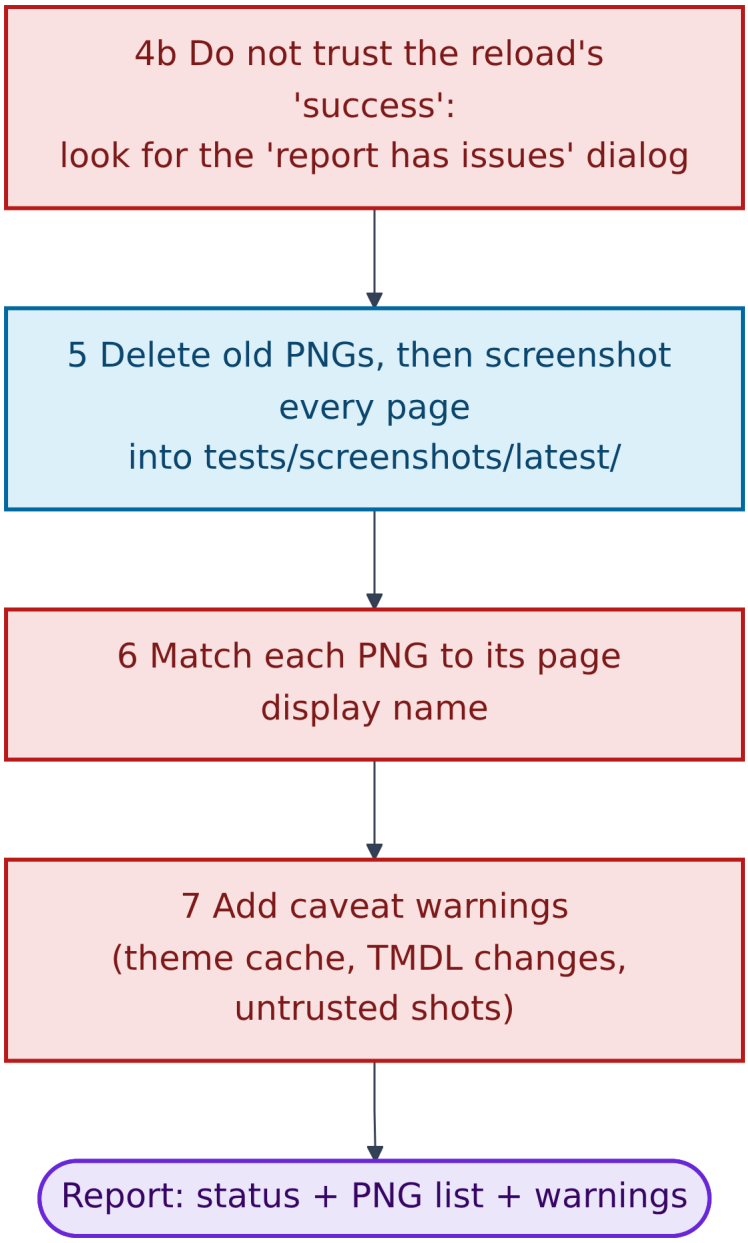


Figure 9.3 — Verify loop, part 2: capture and report.

Walkthrough of Figure 9.3:

- **4b. Do not trust the reload’s “success”.** `powerbi-desktop reload` reports success even when Desktop raises “Your report has issues that could not be resolved”, so the script looks for that dialog itself.

5. **Delete old PNGs, then screenshot every page** into `tests/screenshots/latest/` (2× scale by default). This folder is ignored by Git.
6. **Match each PNG to its page display name**, so findings say “Sales Overview”, not a 20-character id.

7. **Add caveat warnings**, such as “theme colours may be stale” or “screenshots are untrusted”.

- **Result:** a status (Figure 9.4), the list of PNGs, and any warnings.

Run it yourself:

```
powershell -NoProfile -File scripts\Invoke-PbipVerify.ps1 -Mode Review
```

Useful options: `-Mode Confirm` (second pass, after a fix), `-NoReopen` (do not reopen automatically), `-LoadTimeoutSeconds 30`, `-Scale 2`.

What the result means

Status	What it means	What you do
`loaded`	The project loaded and every page was screenshotted.	Open the PNGs and review them against the visual-verify skill.
`validation_failed`	A report or semantic check failed. Desktop was not touched.	Fix the listed errors in the source files, then run again.
`load_failed`	Desktop rejected the project and showed an error dialog.	Read the captured dialog text; fix the named file and line.
`loaded_with_issues`	Desktop loaded only after dropping broken parts. Screenshots are UNTRUSTED.	Fix the reported filter expression; the instance is closed so nothing degraded is saved.
`not_running`	Power BI Desktop is not running (after an automatic reopen attempt).	Open Template.pbip in Desktop, then run again.
`no_bridge`	Desktop is running but offers no bridge connection.	Turn on the Desktop Bridge setting (Section 07) and restart Desktop.
`other_project`	Desktop is running, but with a different project open.	Open this project; the message names what is open instead.
`bridge_unavailable`	The powerbi-report-author CLI is not installed, so the report files cannot be validated.	Install both npm CLIs (Section 07).
`error`	The project could not be found.	Run from inside the repository folder.

Green = success. Amber = skipped: the loop fails open and lets work continue, but nothing was visually verified. Red = a defect that blocks completion.

Figure 9.4 — Result statuses and what to do. Read the row for the status the loop printed.

Walkthrough of Figure 9.4:

1. `loaded` (green) is the only success. Now open the PNGs and review them.
2. **Red statuses** are defects in your files. Fix them before calling the page done.
    - `loaded_with_issues` is the most dangerous: the page *looks* normal, but broken parts were silently dropped, so the screenshots are marked UNTRUSTED. The script closes that window so the damaged version cannot be saved over your files.
3. **Amber statuses** mean the loop **failed open**: it could not reach Power BI, so it skipped rather than block you. Nothing was visually verified. AGENTS.md is explicit: in that case, “say the report was not visually verified rather than claiming it renders correctly”.

The stop hook: when the loop runs by itself

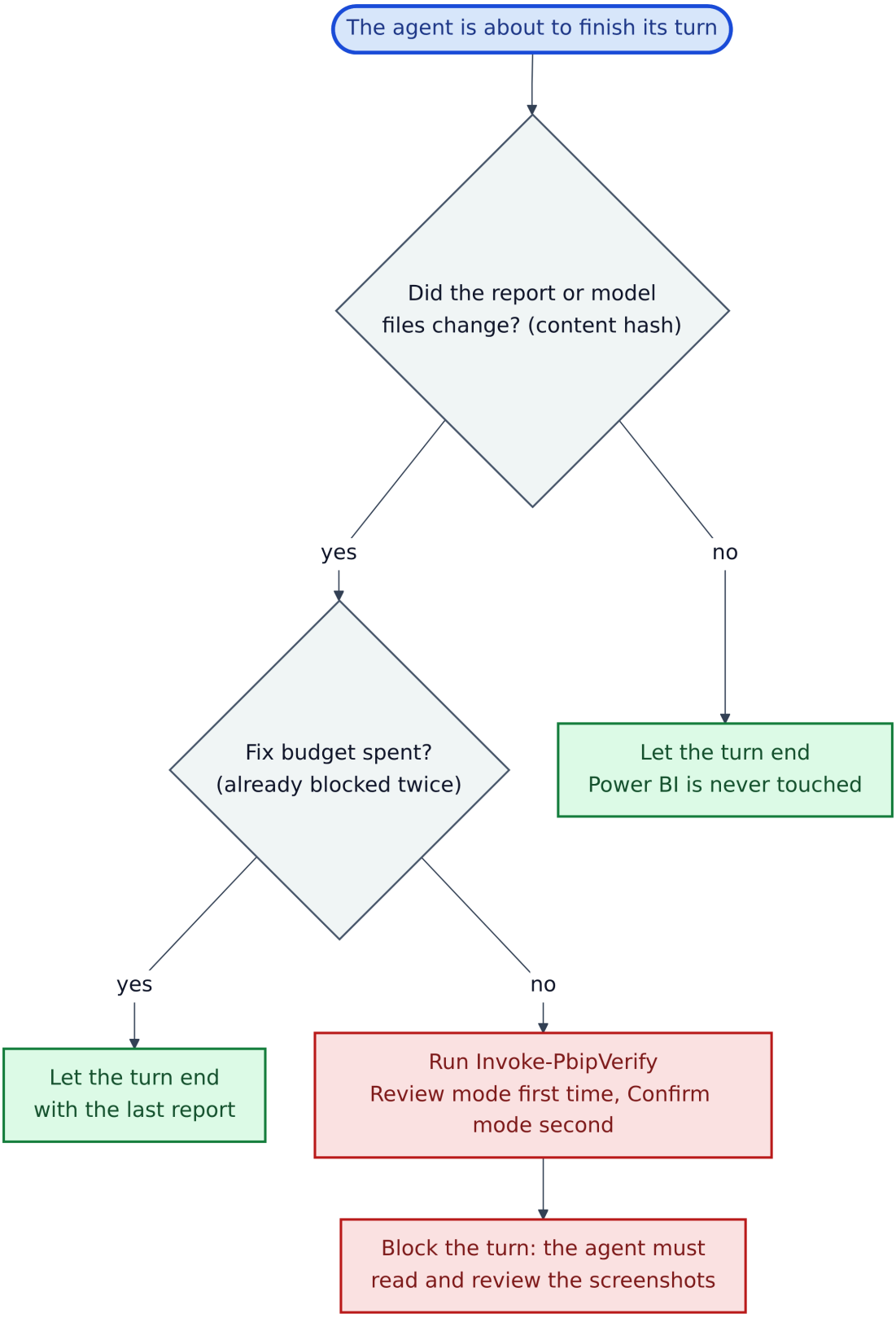


Figure 9.5 — `scripts\hooks\pbip-stop-hook.ps1`**.** This wrapper runs the loop when an agent is about to finish.

Walkthrough of Figure 9.5:

1. **Did anything change?** It compares a fingerprint (a “content hash”) of the report and model files with the last verified one. If nothing changed, Power BI is never touched.
2. **Is the fix budget spent?** The budget is **one fix round**: at most two blocks, “round 1 = review + fix, round 2 = confirmation + report”. After that, the turn may end.
3. **Otherwise it runs the verify loop**: Review mode the first time, Confirm mode the second.
4. **It blocks the turn**, so the agent must read and review the screenshots before finishing. Its bookkeeping lives in `.github/.pbip-state/` (ignored by Git; never edit it).

When Power BI shows an error window

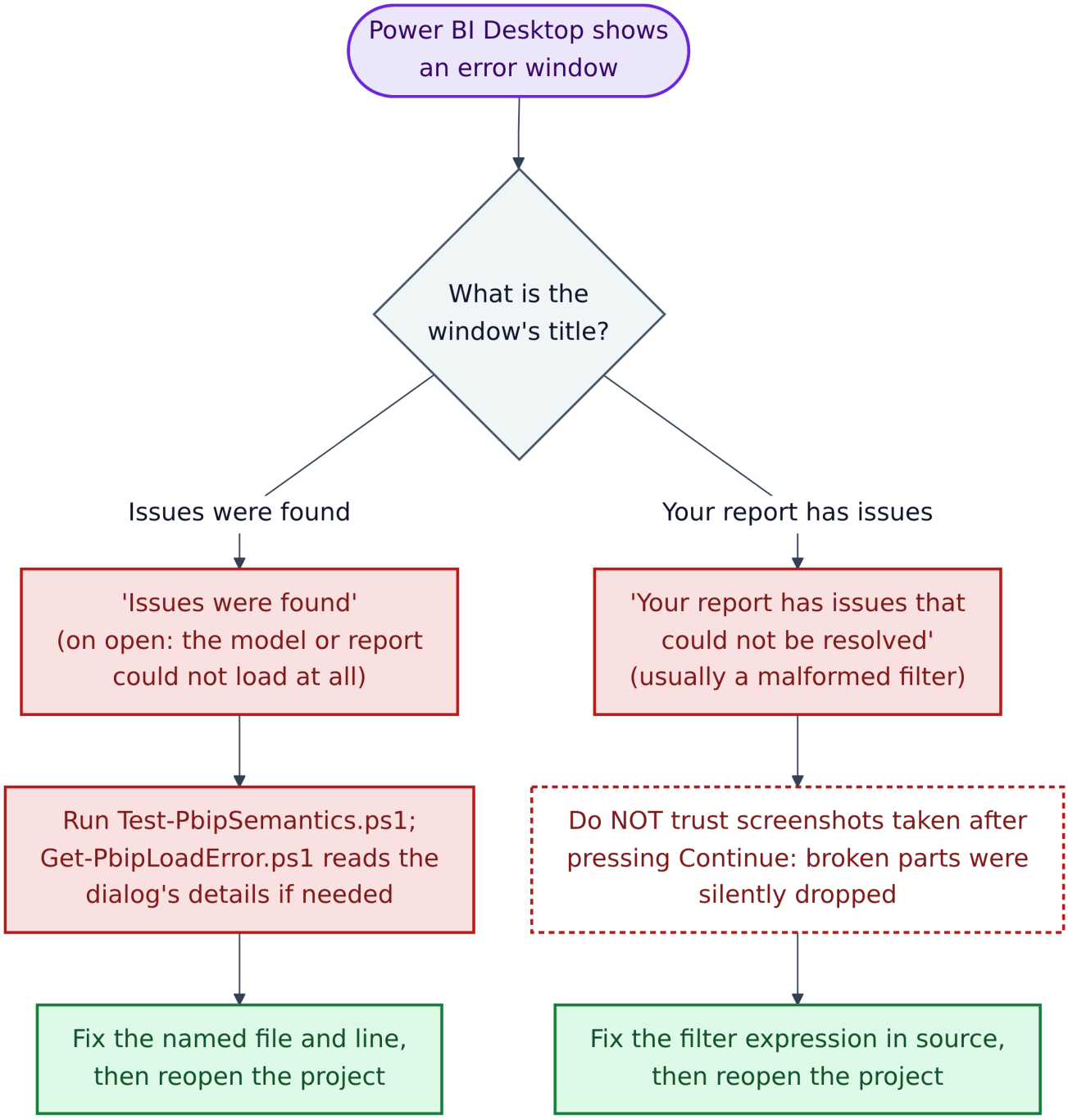


Figure 9.6 — Two error windows, two different fixes.

Walkthrough of Figure 9.6:

1. **“Issues were found”** appears on open when the model or report cannot load at all. The window title often stays *Untitled*. Run `Test-PbipSemantics.ps1` to find the cause in the source. If that finds nothing, `scripts\Get-PbipLoadError.ps1` reads the dialog through Windows UI Automation: its text, the “Copy details to clipboard” diagnostic (which names the file and line), and a cropped screenshot.

2. **“Your report has issues that could not be resolved”** usually means one malformed filter expression. Its list of about 49 issues usually comes from a single bad expression key. **Do not press *Continue* and trust the result**: *Continue* silently drops the broken parts, so a broken filter simply stops filtering.
3. In both cases, fix the source file, then reopen the project. `copilot-instructions.md` lists the usual causes: a measure missing from TMDL, a resource not registered in `report.json`, a property not allowed by the declared schema version, or a UTF-8 BOM.

Gotchas that make screenshots lie

From `copilot-instructions.md` and the README:

Gotcha	What happens	What to do
Theme cache	After editing a theme, a reload still shows the **old colours**	Rename the theme file with a new suffix and update `report.json`, or close and reopen Desktop
Model changes	A visual shows stale or missing measure values after a TMDL change	Reopen instead of reload: `powerbi-desktop open "<project>.pbip"`
First open of a new report	Desktop refuses to load and stays on *Untitled*	Fix the cause; never use *continue with errors*
Wrong window	An idle *Untitled* window says “connected” but fails with `REPORT_DIR_REQUIRED`	Open the project in that window; the scripts match by file path
One operation at a time	Reload and screenshot together give “Cancelled”	Let each call finish first
Empty visuals on a fresh project	“Some of the tables have incomplete or no data”	Run `scripts\Invoke-PbipRefresh.ps1` (it clicks *Refresh*), then save in Desktop

Reviewing the PNGs

Use `.github/skills/powerbi-visual-verify/SKILL.md`. It asks you to score the relevant checklist rows as **Pass / Needs Work / N/A**, cite the row and visible evidence for every defect, recommend a fix **in the source files, never in the image**, and report by **page display name** in this table:

Page	Row	Status	Visual evidence	Threshold/standard	Recommended source fix

10. The theme pipeline: from `Theme.json` to the live report

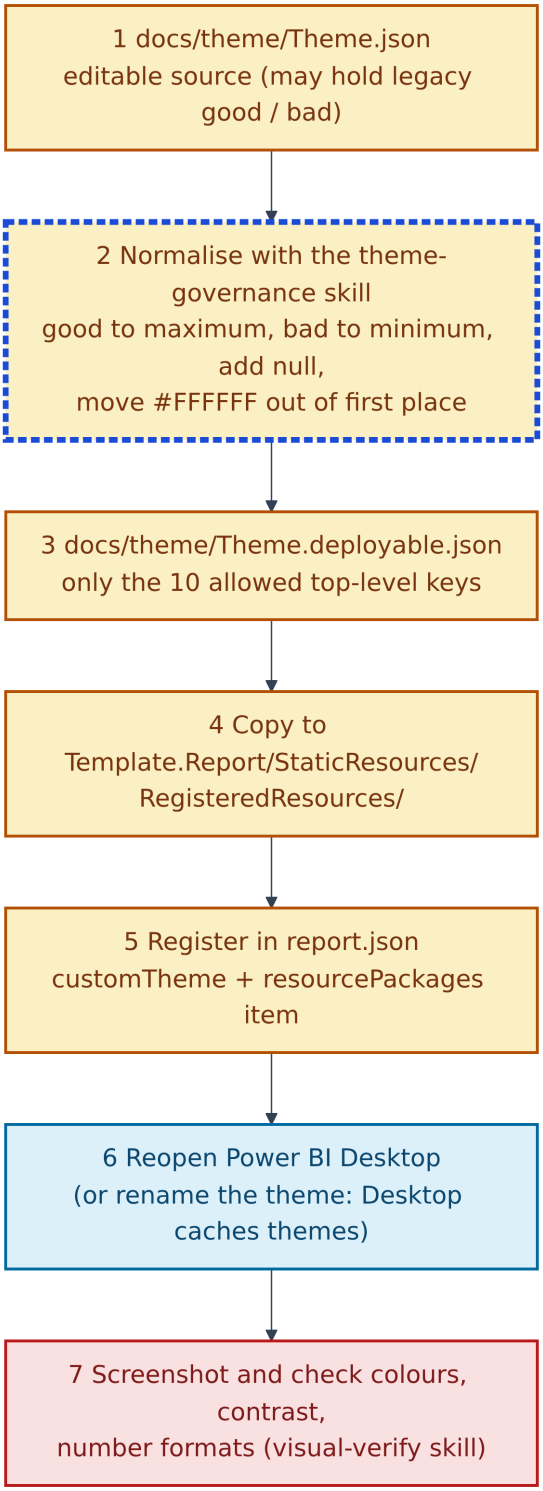


Figure 10.1 — The seven-step theme pipeline. A **theme** is a JSON file of colours and fonts. Microsoft's documentation describes it as a JSON file that sets a report's design, with colours that "come from data colors in the theme".

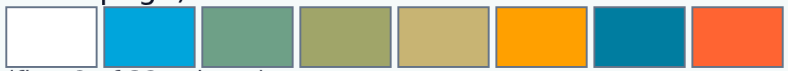
Walkthrough of Figure 10.1:

1. **`docs/theme/Theme.json`** is the editable source, named `PGEColors`. It may contain older fields (`good`, `bad`) that Power BI will not accept in a registered theme.
2. **Normalise** it using the skill `.github/skills/powerbi-theme-governance/SKILL.md` (Amber + AI). The skill's rules: `good` becomes `maximum`, `bad` becomes `minimum`, make sure `null` exists, and move `#FFFFFF` (white) out of first place in `dataColors`.
3. **`docs/theme/Theme.deployable.json`** is the result. It may only use ten top-level keys: `name`, `dataColors`, `background`, `foreground`, `tableAccent`, `maximum`, `minimum`, `null`, `textClasses`, `visualStyles`. "Any other key causes import failure."
4. **Copy** it into `Template.Report/StaticResources/RegisteredResources/`. In this repository the two copies are byte-for-byte identical today.
5. **Register** it in `Template.Report/definition/report.json`: under `themeCollection > customTheme` (name `PGEColors`, including `reportVersionAtImport`), and as an item in the `RegisteredResources` package with `"type": "CustomTheme"`.
6. **Reopen Power BI Desktop.** Desktop caches themes, so a plain reload can show the old colours. Either reopen, or rename the file with a new suffix and update `report.json`.
7. **Screenshot and check** with the visual-verify skill: a theme is applied on purpose (checklist row 51), no white-on-white or low-contrast series (rows 67, 71, 75), and repeated numbers are formatted the same way (row 72).

What promotion changed in this repository

BEFORE: docs/theme/Theme.json (source)

First 8 dataColors (series 1 is **white**: invisible on a white page)



(first 8 of 32 colours)

Top-level keys

```
name
dataColors
background
foreground
tableAccent
textClasses
good
bad
visualStyles
```

AFTER: Theme.deployable.json

First 8 dataColors (white moved to the end)



(first 8 of 32 colours)

Top-level keys

```
name
dataColors
background
foreground
tableAccent
maximum
minimum
null
textClasses
visualStyles
```

good (#008001) became maximum; bad (#D2222D) became minimum; null (#94A3B8) was added. Red = not allowed in a deployable theme; green = added during promotion.

Figure 10.2 — Before and after. Real values from the two files in `docs/theme/`.

Walkthrough of Figure 10.2:

1. **The colour order.** In the source, the first series colour is white (`#FFFFFF`). On a white page the first line or bar would be invisible. The deployable copy moves white to the end of the 32 colours.
2. **The semantic colours.** `good` (`#008001`, green) became `maximum`; `bad` (`#D2222D`, red) became `minimum`; `null` (`#94A3B8`, grey) was added. Red highlighting marks keys that are not allowed; green marks keys added.
3. **Everything else** (`name`, `background`, `foreground`, `tableAccent`, `textClasses`, `visualStyles`) is unchanged.

Two themes at once

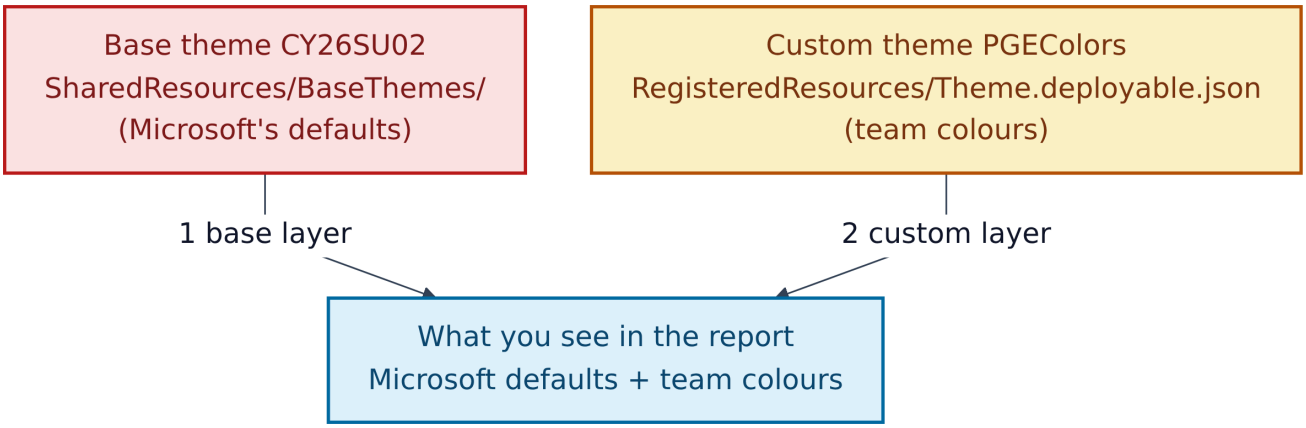


Figure 10.3 — Two theme layers. `report.json` registers both.

Walkthrough of Figure 10.3:

1. **The base theme `CY26SU02`** is a Microsoft file in `SharedResources/BaseThemes/` (Red: do not edit).
2. **The custom theme `PGEColors`** is the team's file in `RegisteredResources/` (Amber).
3. The report you see combines Microsoft's defaults with the team's colours and fonts.

Do and don't

Do	Don't
Edit colours in `docs/theme/Theme.json`	Register `Theme.json` directly in `report.json`
Re-run the promotion rules after every edit	Leave `#FFFFFF` as the first data colour
Keep `reportVersionAtImport` in `report.json` (omitting it gives "Required properties are missing")	Add keys such as `visualContainerHeader`, `axisLine` or `gridlines` to the deployable file
Reopen Desktop, or rename the theme, before judging colours	Trust colours after a plain reload
Keep IBCS green and red meanings (Section 12)	Use brand colours for "good" and "bad"

A small detail for careful readers: `docs/theme/Theme.json` starts with an invisible byte-order mark (BOM). Python's standard JSON reader rejects such a file unless you open the file with `encoding='utf-8-sig'`. Section 16 lists this.

11. Testing and automation: AI tests, GitHub Actions and branches

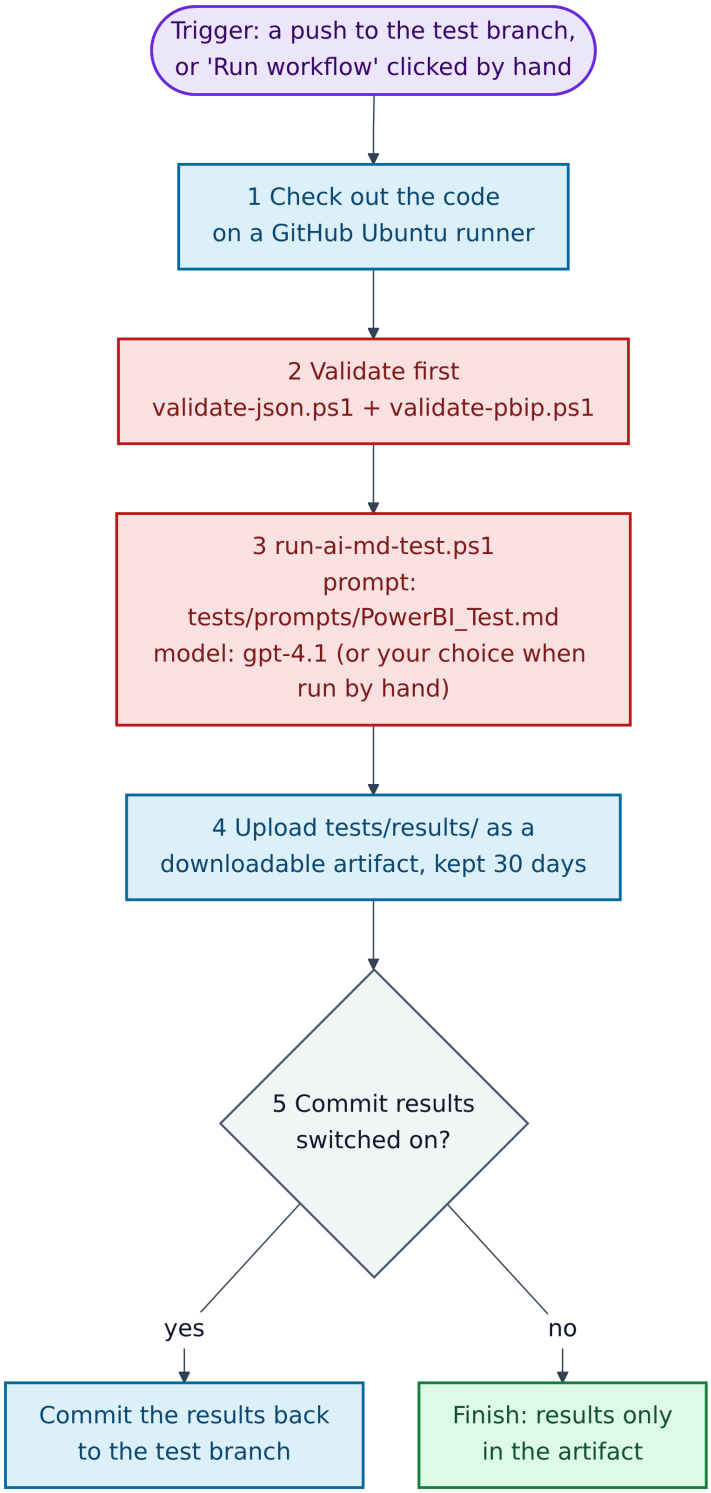


Figure 11.1 — `.github/workflows/ai-testing.yml`, step by step. GitHub Actions is GitHub's automation service. GitHub defines a **workflow** as "a configurable automated process that will run one or more jobs", written in a YAML file in `.github/workflows/`. A **runner** is "a server that runs your workflows when they're triggered".

Walkthrough of Figure 11.1:

- **Trigger.** The workflow runs on every push to the `test` branch, or when someone clicks *Run workflow* (a `workflow_dispatch` trigger). A manual run can choose the AI model (default `gpt-4.1`) and whether to include Best Practice Analyzer rules.

1. **Check out the code** on a GitHub-hosted Ubuntu (Linux) runner, using the `test` environment.
2. **Validate first:** `validate-json.ps1` and `validate-pbip.ps1`, run with PowerShell 7 (`pwsh`). If either fails, the workflow stops.
3. **Run the AI test:** `scripts/run-ai-md-test.ps1` with the prompt `tests/prompts/PowerBI_Test.md` (Figure 11.2).
4. **Upload** `tests/results/` as a downloadable artifact named `ai-markdown-pbip-test-<run id>`, kept for 30 days.
5. **Commit results switched on?** Only when the run is on the `test` branch *and* the repository variable `AI_TEST_COMMIT_RESULTS` equals `true` does the workflow commit the results back to `test` (with `[skip ci]` in the message so it does not trigger itself again). Otherwise the results live only in the artifact.

Inside the AI test runner

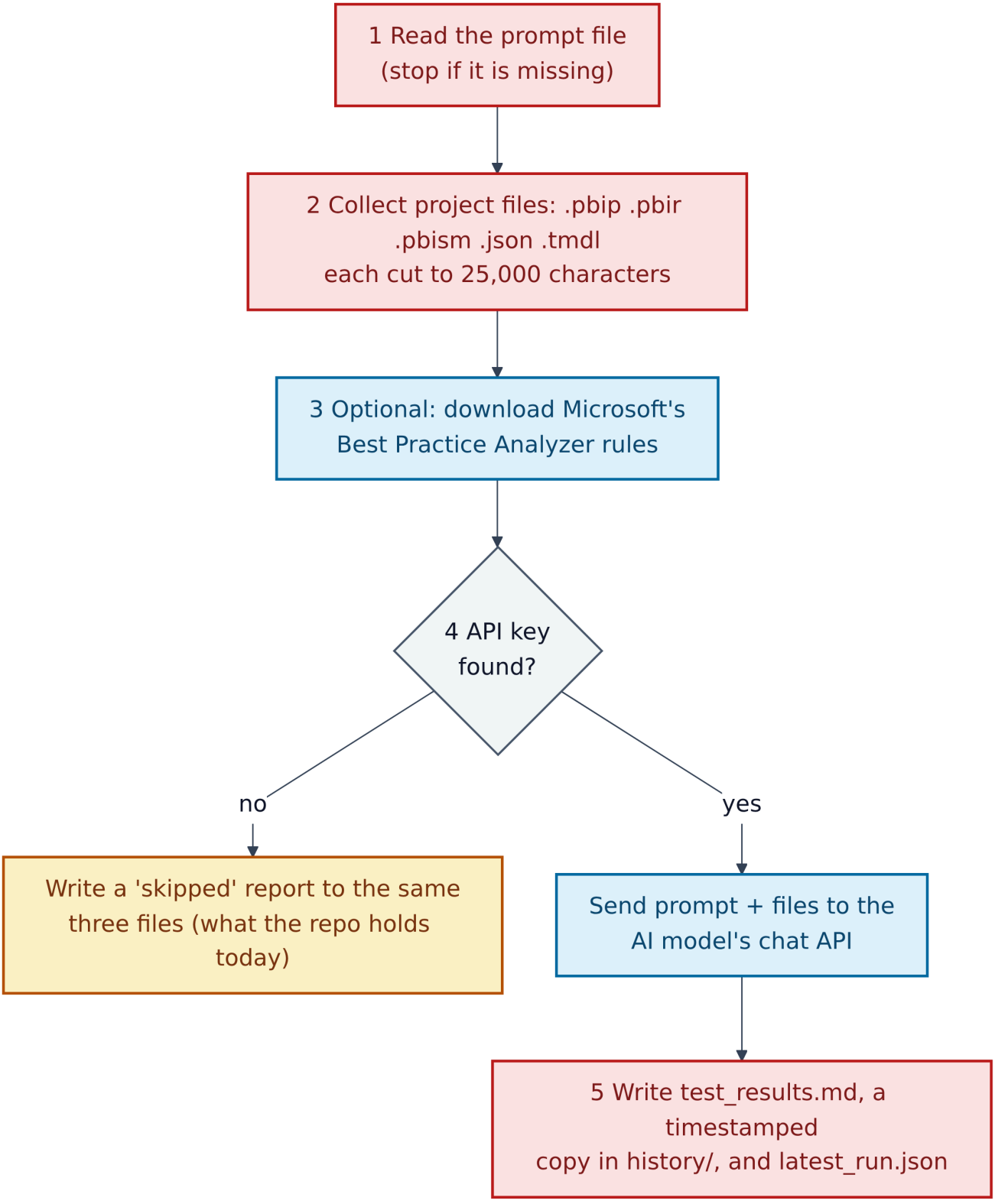


Figure 11.2 — `scripts/run-ai-md-test.ps1`.

Walkthrough of Figure 11.2:

1. **Read the prompt file.** It stops if the file is missing.

2. **Collect the project files** (`.pbip`, `.pbir`, `.pbism`, `.json`, `.tmdl`), skipping `tests/results/`, `.git/` and the Power BI data cache. Each file is cut to 25,000 characters (`-MaxFileChars`).
3. **Optional:** with `-IncludeBpaRules`, download Microsoft's Best Practice Analyzer rules from the `microsoft/Analysis-Services` repository on GitHub and add them to the prompt. If the download fails, it carries on without them.
4. **API key found?** The script looks in `GITHUB_MODELS_API_KEY`, then `AZURE_OPENAI_API_KEY`.
    - **No key:** it writes a "skipped" report. That is exactly what this repository's `tests/results/` holds today: two runs on 17 April 2026, both skipped with the reason "No AI API key was found…". Add `-FailIfNotConfigured` to make a missing key an error instead.
    - **Key found:** it sends the prompt and files to the chat API (by default `https://models.inference.ai.azure.com/chat/completions`, GitHub Models).
5. **Write the outputs:** `tests/results/test_results.md` (latest), `tests/results/history/test_results_<UTC time>.md` (permanent history), and `tests/results/latest_run.json` (details of the run).

Run it locally (PowerShell, from the repository root):

```
./scripts/run-ai-md-test.ps1 -PromptFile tests/prompts/PowerBI_Test.md -OutputDir tests/results -O
utputFileName test_results.md -IncludeBpaRules
```

Who does what in an automated test

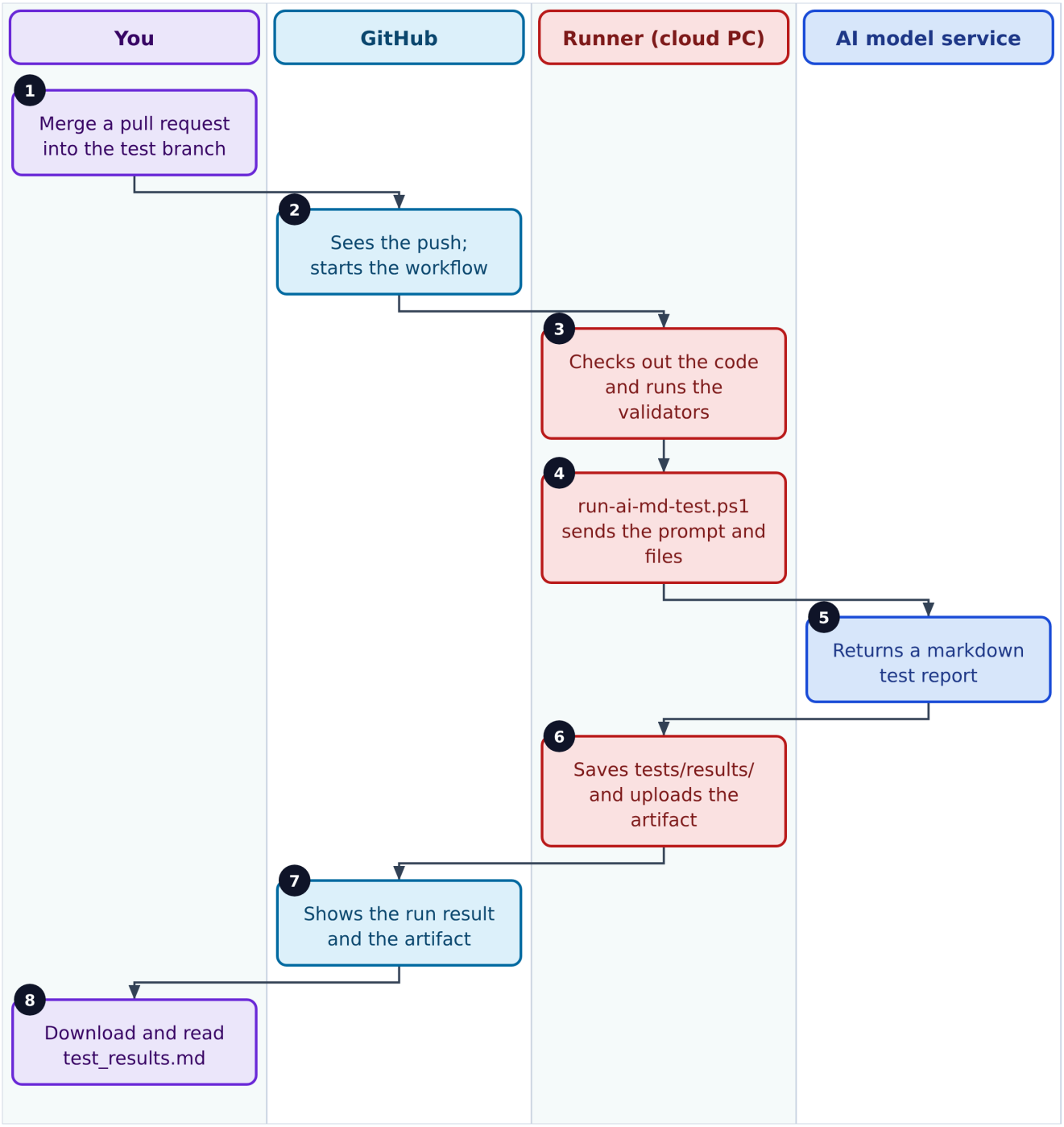


Figure 11.3 — Swimlane of one automated test run.

Walkthrough of Figure 11.3:

1. You merge a pull request into `test`. Merging is a push.
2. GitHub sees the push and starts the workflow.
3. A runner (a temporary cloud computer) checks out the code and runs the validators.
4. The runner's script sends the prompt and project files to the AI model.
5. The AI model service returns a markdown test report.
6. The runner saves `tests/results/` and uploads it as an artifact.
7. GitHub shows the run's result (green tick or red cross) and the artifact.

8. You download and read `test_results.md`.

What the AI test checks

`tests/prompts/PowerBI_Test.md` is what the repository calls a “Measure Killer-style” audit prompt: it looks for unused and broken parts of the model by reading the files, and its hard requirements forbid running Python, Node or any custom script. In eight phases it:

1. finds and checks the project files;
2. lists every table, column, measure and relationship;
3. lists every page and visual and what they use;
4. finds visuals that point at measures or columns that do not exist, and measures or columns that nothing uses;
5. writes the report with severity markers (critical, high, medium, low, info) in a fixed layout;
6. returns a short chat summary;
7. adds a Best Practice Analyzer (BPA) assessment;
8. gives one combined verdict.

Settings to add on GitHub

Setting	Kind	Where	Purpose
`GITHUB_MODELS_API_KEY` or `AZURE_OPENAI_API_KEY`	Secret (at least one)	*Settings > Secrets and variables > Actions*	Lets the AI test call a model
`AI_TEST_COMMIT_RESULTS`	Variable, value `true`	Same page, *Variables* tab	Commits results back to `test`
`test`	Environment	*Settings > Environments*	The workflow’s job runs in this environment

GitHub’s documentation notes that secrets are not passed to workflows triggered from forks (except the built-in `GITHUB_TOKEN`), and that a secret that is not set reads as an empty string. That is why a fork without secrets produces a “skipped” report rather than an error.

`docs/CICD-SETUP.md` also describes **Fabric deployment secrets** (`FABRIC_TENANT_ID`, `FABRIC_CLIENT_ID`, `FABRIC_CLIENT_SECRET`) and **workspace variables** (`PBI_WORKSPACE_ID_DEV`, `_TEST`, `_PROD`, optional `PBI_DEPLOYMENT_PIPELINE_ID`) for a deployment workflow called `cd.yml`. That workflow is not in this repository (Section 16).

The recommended branch and environment set-up

From `docs/CICD-SETUP.md` (see also Figure 3.3):

1. **Branches:** create and protect `dev`, `test` and `main`. Feature work goes to `dev`, release candidates to `test`, production to `main`.
2. **GitHub environments:** `dev`, `test`, `prod`, with required reviewers for `test` and `prod`, and no self-approval for production.
3. **Branch protection:** require a pull request, passing status checks, at least one reviewer, and dismiss old approvals when new commits arrive.
4. **Optional hardening:** secret scanning, model linting, screenshot artifacts, release notes.

Python data-value testing

`tests/prompts/Python_Data_Value_Test.md` is an “instruction-only” prompt. It does **not** write a script. It asks Copilot to read the model and visuals, find the measure behind a “Sales by Branch” visual, and then give you a blueprint for writing your own Python checker. The example question is “sales of branch `ABV` in March 2026”, with command-line options `--branch`, `--month`, `--year` and `--dry-run`. Results should be saved under `tests/results/`. No validator script ships with the template; teams write their own (Section 13).

12. The custom-visual (.pbiviz) starter kit

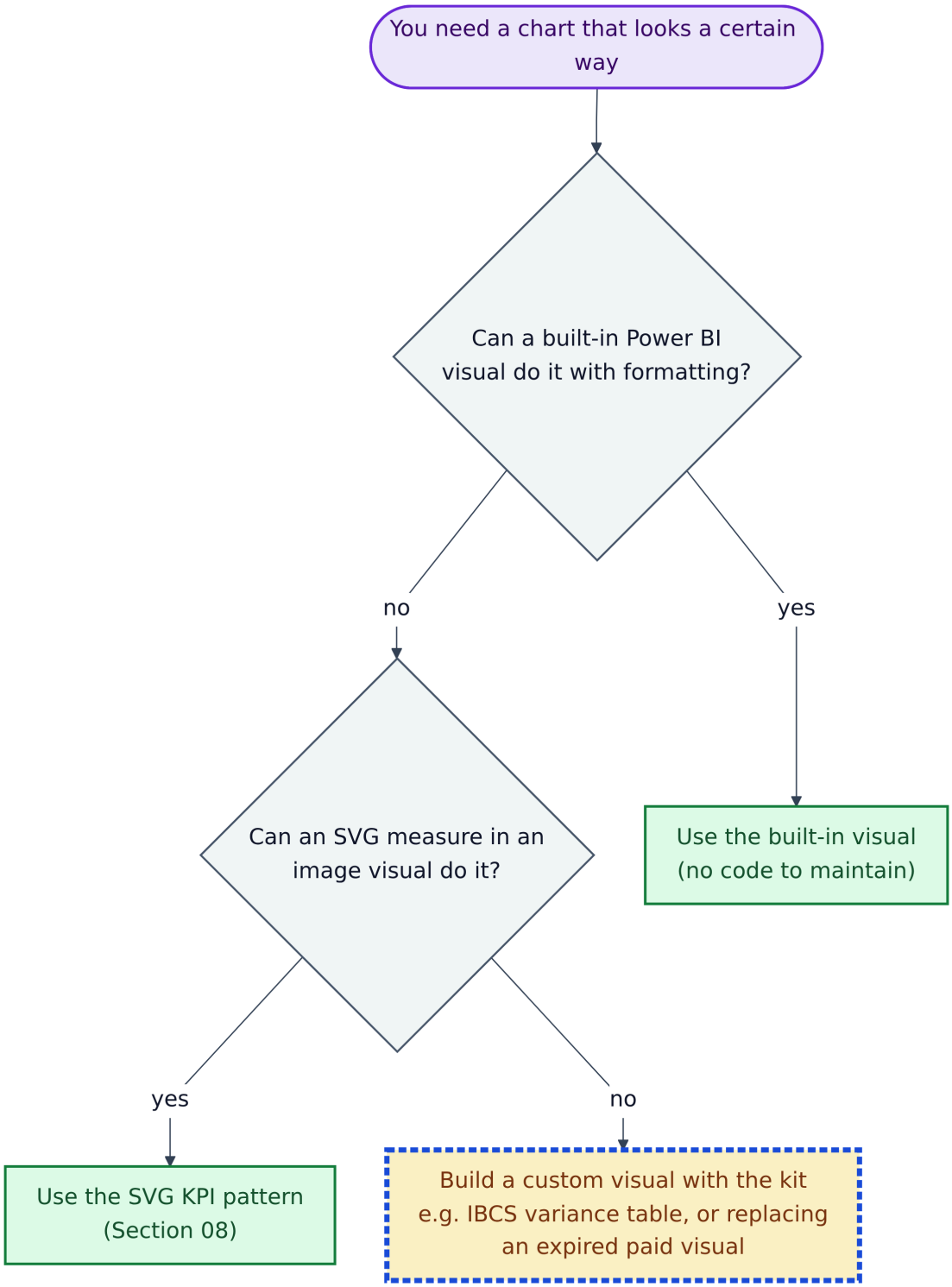


Figure 12.1 — Do you really need a custom visual? A **custom visual** is a chart type your team writes in code (TypeScript), packaged as a `.pbiviz` bundle. The skill `.github/skills/powerbi-pbiviz-custom-visuals/SKILL.md` warns: “A custom visual is code to maintain. Say so if the request does not need one.”

Walkthrough of Figure 12.1:

1. **Can a built-in visual do it with formatting?** Use it.
2. **Can an SVG measure in an image visual do it?** Use the SVG KPI pattern from Section 08.
3. **Neither?** Build one with this kit. Typical reasons: an **IBCS** finance layout (IBCS is a set of business-chart design standards, e.g. actuals solid dark, variances green and red), or replacing a paid marketplace visual whose licence expired.

Before building, the skill says the agent must ask three questions in one round: what the visual should show and why (with any reference screenshot), how interactive it should be (default: fully interactive), and the author name and email for `pbiviz.json` (packaging refuses without them).

The ten-step workflow

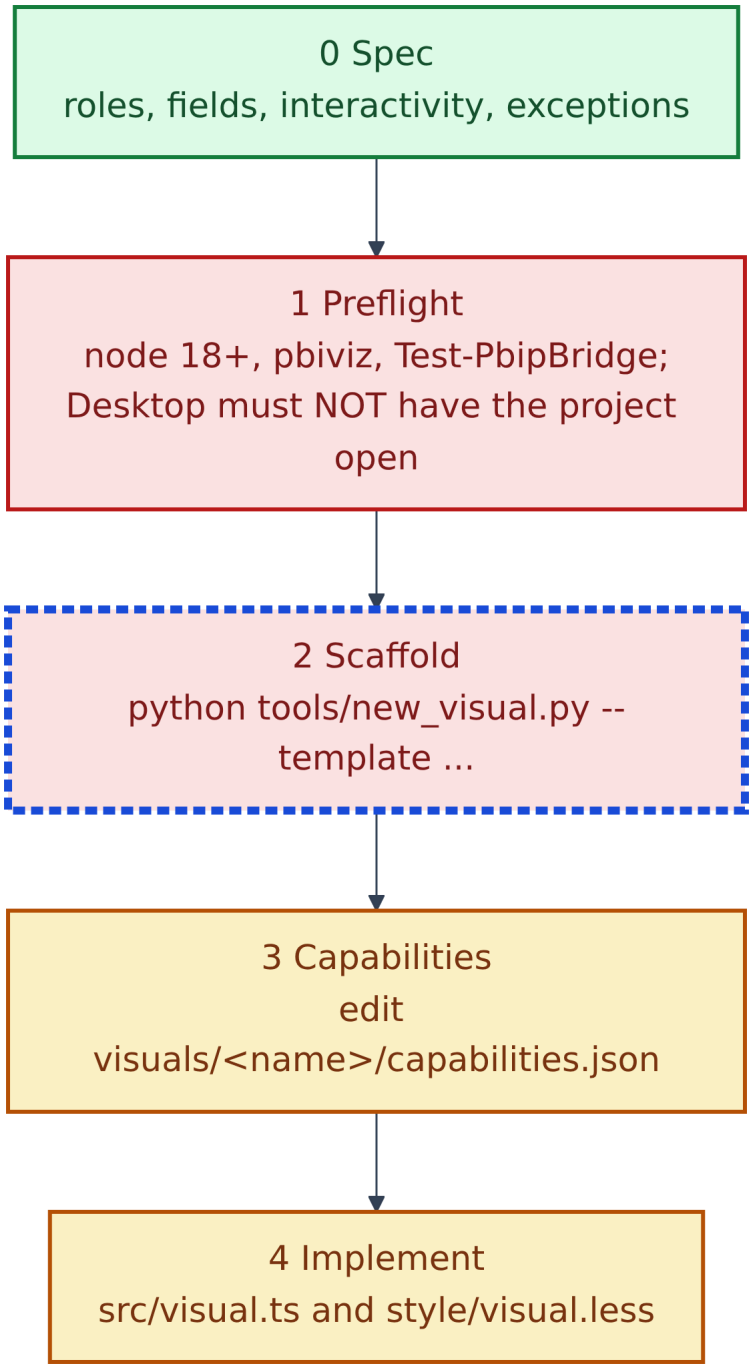


Figure 12.2 — Workflow, part 1: plan and write.

Walkthrough of Figure 12.2:

0. **Spec.** Copy `specs/TEMPLATE.md`; for each visual record its data roles and fields, Format-pane options, computed logic, interactivity, size and exceptions.

1. **Preflight.** `node --version` (18 or later), `pbiviz --version` (install with `npm i -g powerbi-visuals-tools`), and `scripts\Test-PbipBridge.ps1`. Power BI Desktop must **not** have the project open while syncing.

2. **Scaffold** a new visual from a template:

```
python .github\skills\powerbi-pbiviz-custom-visuals\assets\tools\new_visual.py --name finVar
ianceTable --template varianceTable --display "Variance Table" --description "IBCS statement
table" --author-name "<name>" --author-email "<email>"
```

This creates `visuals/<name>/` in *your dashboard repository* (not in this template), gives it a permanent GUID, records it in `tools/visuals.json`, and runs `npm install`.

3. **Capabilities.** Edit `capabilities.json`: the data roles, how the data is shaped, and the Format-pane options. Microsoft describes this file as telling Power BI "what kind of data the visual accepts" and what to put in the properties pane.

4. **Implement** the drawing code in `src/visual.ts` and styles in `style/visual.less`. Edit shared code only in `visuals/shared/`; `src/shared/` is overwritten on every build.

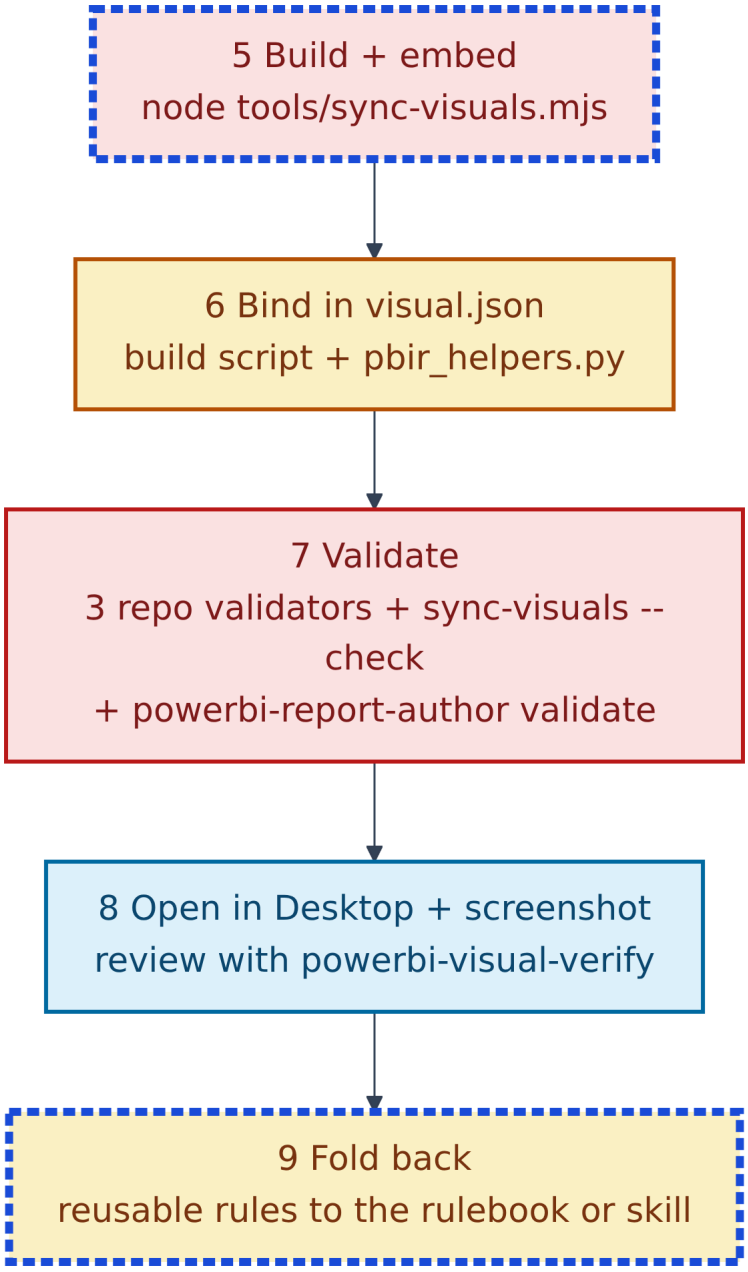


Figure 12.3 — Workflow, part 2: build, bind and prove.

Walkthrough of Figure 12.3:

5. **Build and embed** with `node tools\sync-visuals.mjs` (or `--only <name>`). A TypeScript or lint error stops the build here.
6. **Bind** the visual on a page by writing its `visual.json`, preferably from the page's Python build script using `tools/pbir_helpers.py`, and check it with `check_against_capabilities(...)`.
7. **Validate** with the three repository validators, `node tools\sync-visuals.mjs --check`, and `powerbi-report-author validate`.
8. **Open in Desktop and screenshot** every page; review with the visual-verify skill. Extra checks: no empty boxes, no licence overlays, no clipped labels, totals match the agreed numbers, and clicking and expanding work.
9. **Fold back** reusable rules into the rulebook or the skill.

The rule that breaks most often: three things must line up

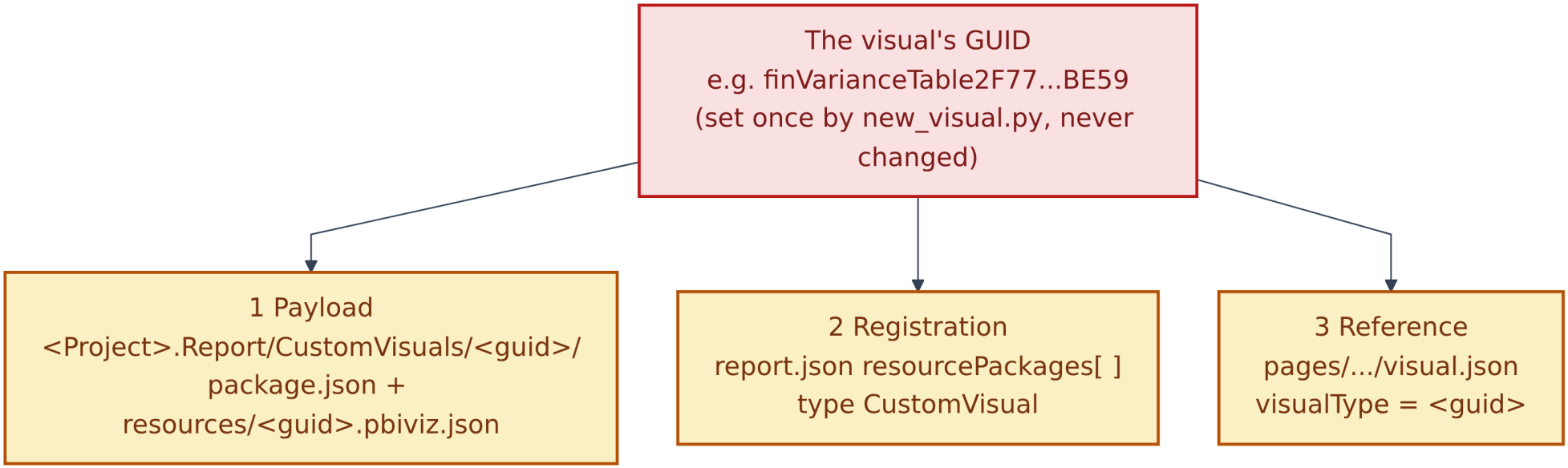


Figure 12.4 — The GUID triad, from `references/PBIP-EMBEDDING.md`. A **GUID** is a long unique identifier, like a serial number.

Walkthrough of Figure 12.4:

1. **Payload:** the built bundle sits in `<Project>.Report/CustomVisuals/<guid>/`.
2. **Registration:** `report.json` lists the GUID in `resourcePackages` with type `CustomVisual`.
3. **Reference:** each page's `visual.json` names the GUID as its `visualType`.

"Get one wrong and the visual renders as an empty box with **no error dialog**." `sync-visuals.mjs` does steps 1 and 2 and checks step 3; `scripts/validate-pbip.ps1` also checks it. The GUID never changes once a report uses it; rename a visual only by its `displayName`.

What `sync-visuals.mjs` does for each visual

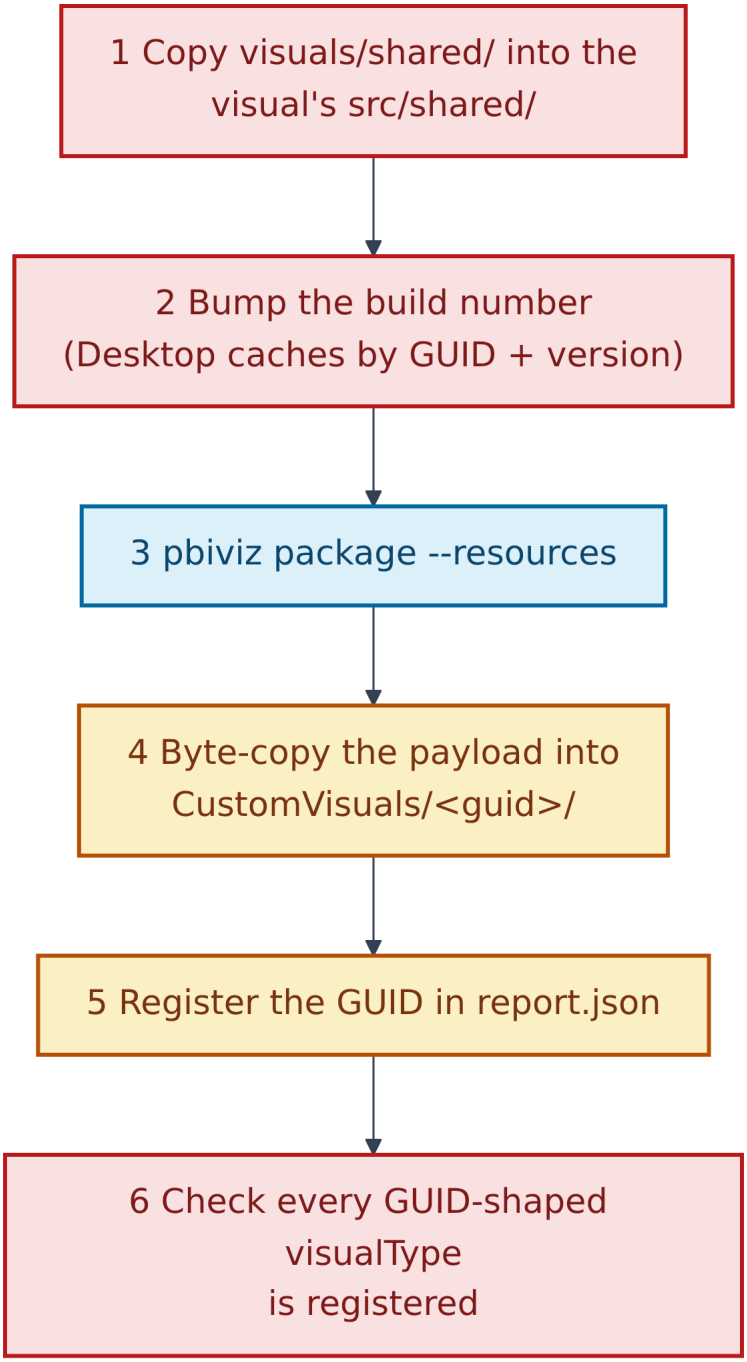


Figure 12.5 — The six steps of `sync-visuals.mjs`.

Walkthrough of Figure 12.5:

1. Copy the shared library into the visual's `src/shared/`.
2. Increase the build number, because Desktop caches bundles by GUID and version.
3. Run `pbiviz package --resources`.
4. Copy the payload byte for byte into `CustomVisuals/<guid>/` (never re-save it through a JSON tool).
5. Register the GUID in `report.json`.

6. Check every GUID-shaped `visualType` is registered.

After a sync, **close and reopen** Desktop; a reload still serves the cached bundle.

What is in the kit

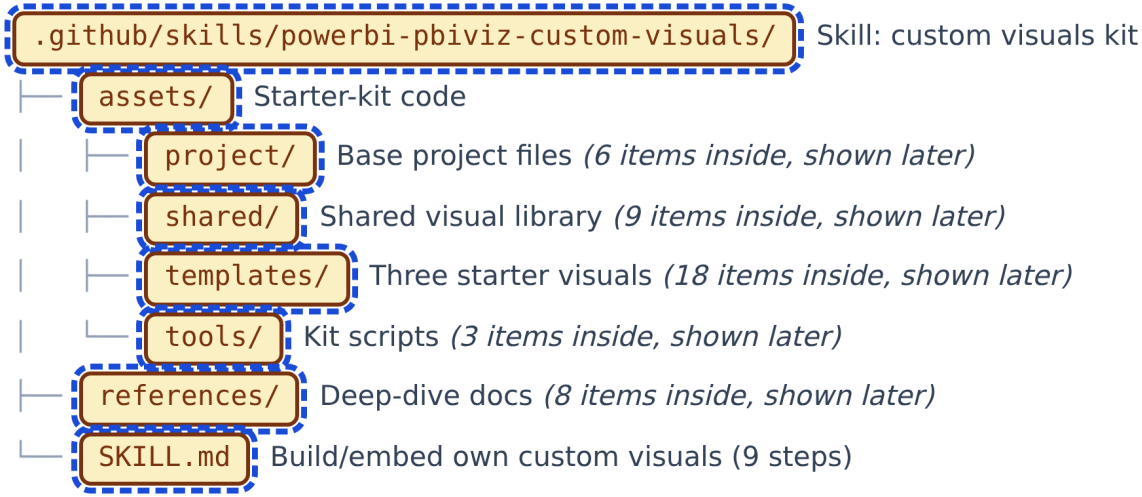


Figure 12.6 — The kit folder, two levels deep. The four `assets/` sub-folders and the eight reference documents are listed in full in Section 14.

Walkthrough of Figure 12.6:

1. **`assets/project/`** — the files every visual project starts with: `package.json` (pinned versions: `powerbi-visuals-api` 5.11.1, `powerbi-visuals-utils-formattingmodel` 6.0.4, TypeScript 5.5.4), `pbiviz.json` (placeholders such as `__GUID__`), `tsconfig.json`, `eslint.config.mjs` and a 20×20 icon.
2. **`assets/shared/`** — a shared TypeScript library: colour tokens (`tokens.ts` + `base.less`), safe element builders (`dom.ts`, no `innerHTML`), number formats (`format.ts`), clicks and tooltips (`interaction.ts`), shapes (`paths.ts`), Format-pane settings (`props.ts`), scales (`scale.ts`) and text fitting (`text.ts`).
3. **`assets/templates/`** — three working starter visuals: `varianceTable` (IBCS statement table), `varianceChart` (IBCS period chart with a variance strip) and `blank` (a minimal skeleton showing every required pattern).
4. **`assets/tools/`** — `new_visual.py`, `sync-visuals.mjs`, `pbir_helpers.py`.
5. **`references/`** — eight deep-dive documents, from data roles to troubleshooting.
6. **`SKILL.md`** — the procedure itself.

All kit files are Red + AI: the agent copies them into your dashboard repository; you change the copies, not these originals.

The ten hard rules (from SKILL.md, in plain words)

1. A GUID never changes once bound; rename with `displayName` only.
2. `visual.json` data keys must match the data-role names, and its options must match `capabilities.json`.
3. Close Desktop while syncing; reopen afterwards.
4. Copy built payloads byte for byte.
5. No `innerHTML`, no internet access (`privileges: []`), no vendor branding.
6. Visuals hide Power BI’s container chrome; the SVG background draws cards and titles.
7. Table-style visuals that add up numbers in code only use additive measures; ratios are calculated from their parts.
8. Never copy a paid visual’s pop-ups, logo or patent strip.
9. Interactive by default, and always signal rendering start and finish (needed for PDF and PowerPoint export).
10. A visual is never done because it built; only a reviewed Desktop screenshot counts.

Related scoped rule file

`.github/instructions/power-bi-custom-visuals-development.instructions.md` loads automatically for `.ts`, `.tsx`, `.js`, `.jsx`, `.less` and `.css` files. It is a general React and D3.js reference for visual internals; the skill says to use the skill, not that file, for anything touching the PBIP project.

13. The Python email-report and alert guides

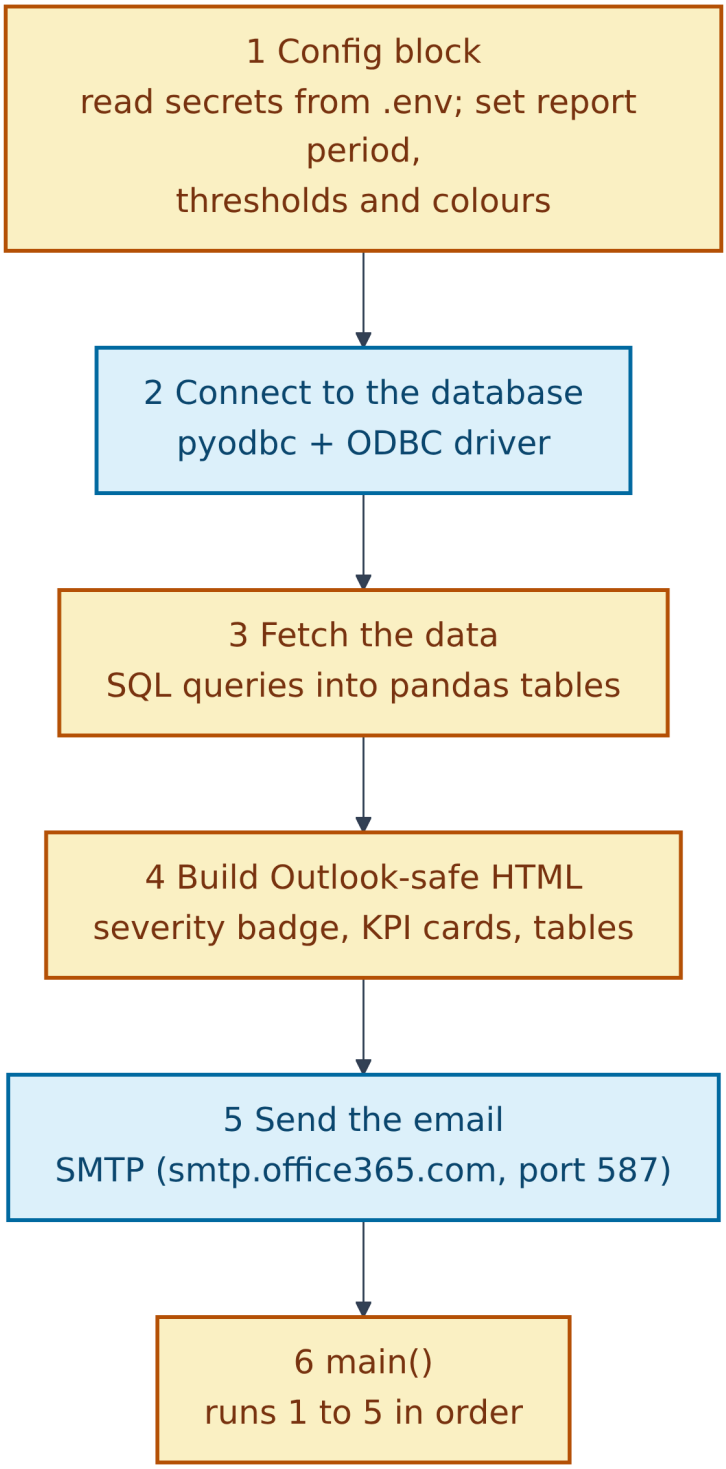


Figure 13.1 — How the alert-email script is organised, from `python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md`. The `python/` folder holds **instructions only**: no Python file ships with the template. You (or an agent) follow these guides to write a script in your own project.

Walkthrough of Figure 13.1:

1. **Config block.** Load secrets from `.env` (Section 07) and set the report period, thresholds, labels and colours.
2. **Connect to the database** with `pyodbc` and an ODBC driver (a standard Windows database connector).
3. **Fetch the data** with SQL queries into `pandas` tables (pandas is Python's spreadsheet-like data library).
4. **Build the email as HTML** that Outlook displays correctly: a severity badge, KPI cards, a summary table, a detail table, a note strip and a footer.
5. **Send the email** through SMTP (the email-sending protocol). The guide's example uses `smtp.office365.com` on port 587.
6. `main()` runs the steps in order.

The guide's `requirements.txt` lists `pyodbc`, `pandas` and `python-dotenv`. **Real-world comparison:** a night guard who checks the building, writes a short colour-coded note, and emails it to the manager.

How the alert decides its colour

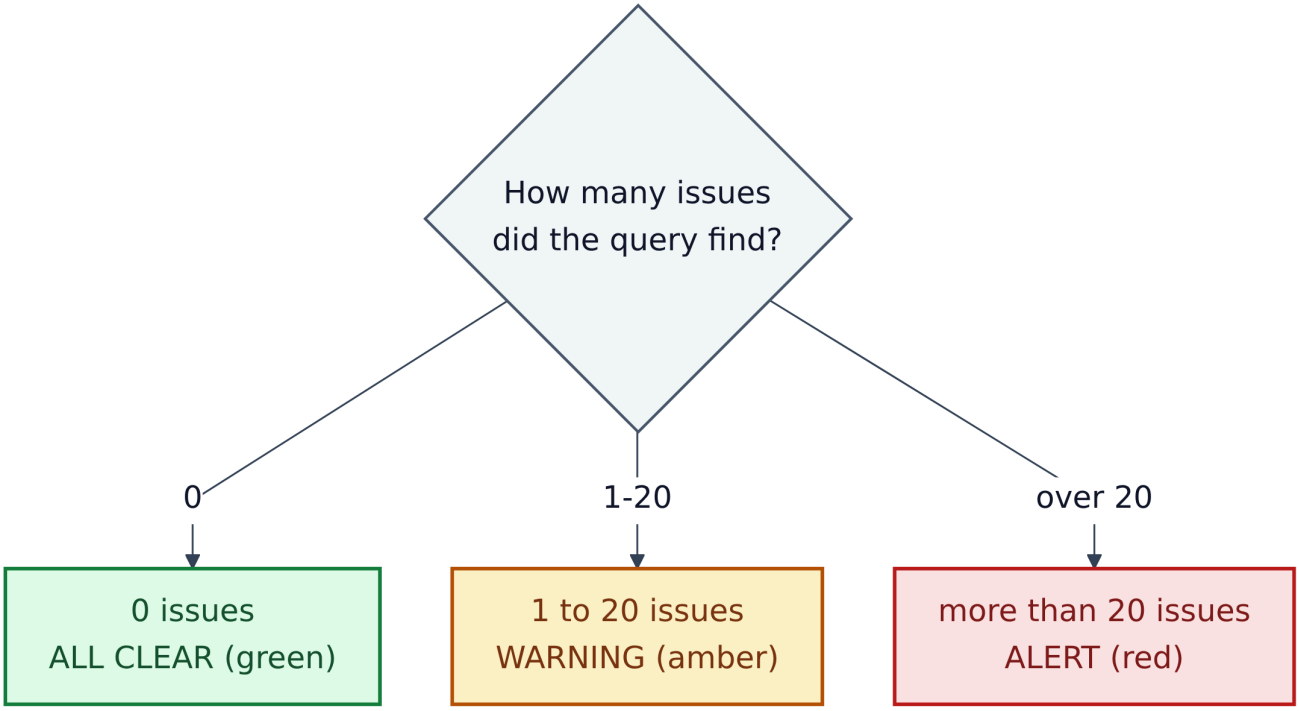


Figure 13.2 — Severity rule (section 7.1 of the alert guide). These colours describe the email's urgency, not the file-safety legend.

Walkthrough of Figure 13.2:

1. **0 issues:** "ALL CLEAR", green.
2. **1 to 20 issues:** "WARNING", amber.
3. **More than 20 issues:** "ALERT", red.

The guide says to classify the alert *before* building the HTML, because the level sets the badge colour and the email subject line.

Outlook-safe HTML

Do NOT use (Outlook breaks it)	Use instead
`background: shorthand`	`background-color: inline`
`linear-gradient(...)`	`a flat background-color`
`border-radius:`	`skip it (Outlook ignores it)`
`box-shadow:`	`skip it (not supported)`
`display:flex / display:grid`	`<table> layout only`
`<div> for coloured backgrounds`	`<td bgcolor="...">`
`max-width: on <table>`	`width="720" attribute`

Source: python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md, section 7.2 (Outlook-Safe HTML Rules).

Figure 13.3 — What Outlook breaks, and what to use instead.

Walkthrough of Figure 13.3:

1. Desktop Outlook ignores or breaks many modern web-page styles, such as rounded corners, shadows, gradients and flexible layouts.
2. The guide therefore builds every layout from `<table>` elements with inline `background-color` and fixed widths (720 pixels).

Making Python numbers match Power BI

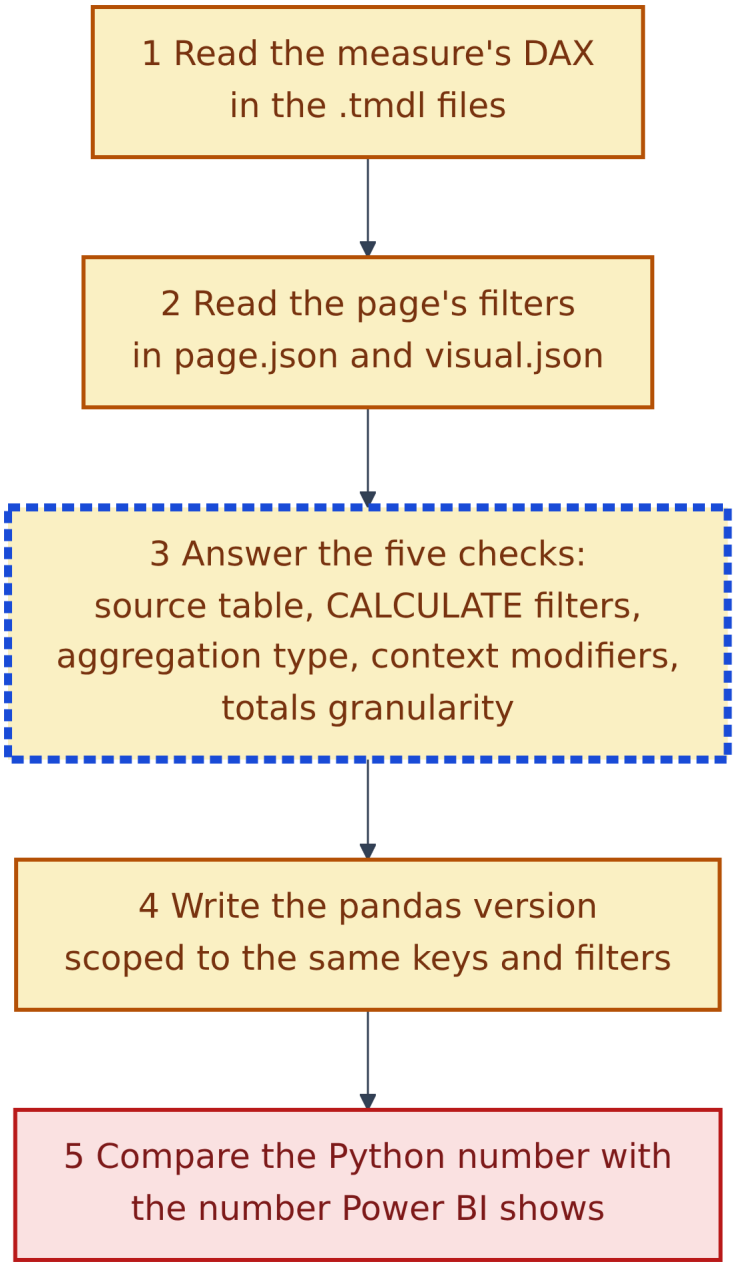


Figure 13.4 — The DAX-to-Python validation protocol, from `python/email_reporting/emal_style_instructions.md`. The guide calls it “MANDATORY” and says to read it “completely before writing ANY Python or HTML”.

Walkthrough of Figure 13.4:

1. **Read the measure’s DAX** in the model’s `.tmdl` files. Treat the semantic model and page JSON as “the source of truth”.
2. **Read the page’s filters** in `page.json` and each `visual.json`, and build a set of the valid keys the page allows.
3. **Answer the five checks for every measure:**
    - **Source table**: which fact table does the measure use? Never mix tables with different scopes.

    - **CALCULATE filters**: copy every filter inside `CALCULATE`. Missing one gives wrong totals.
    - **Aggregation type**: `SUM`, `DISTINCTCOUNT`, `SUMX` and `AVERAGEX` behave differently in totals.
    - **Context modifiers**: `ALL(...)` removes filters, so Python must not filter that table either.
    - **Granularity in totals**: summing a per-customer number in a per-customer-per-location table counts it more than once.
4. **Write the pandas version**, scoped to the same keys and filters. Use helpers such as `safe_div` (never divide by zero) and `fmt` / `fmt_pct` for display.
5. **Compare** the Python number with the number Power BI shows. They must match before the email goes out.

Real-world comparison: translating a recipe from one language to another. If you skip one ingredient (a filter), the dish (the total) tastes different, even though every step you did translate is correct.

The three Python files at a glance

File	What it is	Key contents
`python/README.md`	"Build-your-own" validator instructions	Read the PBIP files, find a "Sales by Branch" visual, generate SQL from its measure, support `--dry-run`; example command `python python/<your_script_name>.py --branch ABV --month March --year 2026`
`python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md`	Template for any database-triggered alert email	12 sections: project structure, `.env` credentials, 6-phase skeleton, Outlook HTML components, format helpers, colour reference, a pre-send checklist
`python/email_reporting/emal_style_instructions.md`	Rules for emails that copy a Power BI page	Actually three documents joined together: a boilerplate "SKILL", the universal instructions (Parts A–E: validation protocol, HTML style, Python structure, charts, project reference), and "HTML Email Style & Body Instructions"

Safety reminders for these guides

- **Never hard-code passwords.** Both guides read every secret from `.env` through `python-dotenv` (Section 07).
- **Part E of the email guide** has blank placeholders for the database and SMTP server: fill them in your own copy, not in this template.
- **Preview first.** The email guide describes a preview mode that writes the HTML to a file instead of sending it. Use it before emailing real people.

14. File-by-file directory

Every one of the repository’s 176 files and folders, grouped by folder. Each group opens with a folder-tree diagram coloured with the safety legend from Section 00 (fill colour = safety; blue dashed outline = AI agent file). Under each tree, the numbered entries follow the tree from top to bottom and give:

- the **exact path** and its **safety flag**;
- **What it is:** a plain-English summary with a real-world comparison;
- **Technical role:** what the file does for the tools.

Folders that appear closed in one tree are opened in their own tree later. Each item is described once, the first time it appears.

14.1 Root files

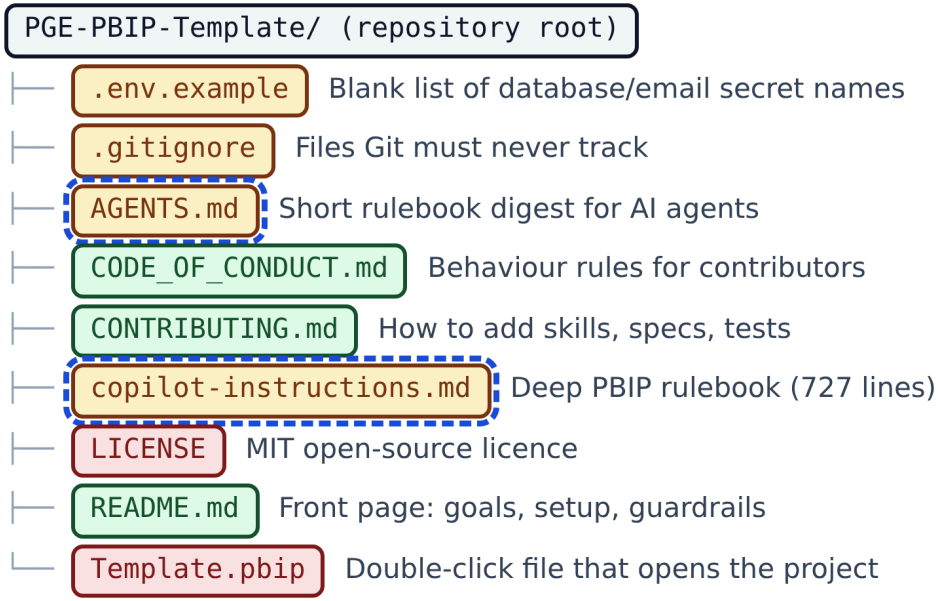


Figure 14.1 — Root files. The nine files that sit directly in the repository's top folder.

Walkthrough of Figure 14.1:

1. `.env.example` — **Amber**
   - *What it is:* A blank form listing the names of the secrets the Python guides need, with no real values. Like an empty form with labelled boxes.
   - *Technical role:* Template for a local, git-ignored .env file: DB_SERVER, DB_NAME, DB_USER, DB_PASSWORD, DB_DRIVER, SMTP_EMAIL, SMTP_PASSWORD.
2. `.gitignore` — **Amber**
   - *What it is:* The list of files Git must never save, such as your private .env and Power BI's local cache. Like a 'do not pack' list for a suitcase.
   - *Technical role:* Ignores **/.pbi local settings and cache, .env and .env.* (except .env.example), Python caches, *.tmp, *.log, tests/screenshots/, .github/.pbip-state/ and Proposals/.

3. `AGENTS.md` — **Amber** · **AI file**
    - *What it is:* A one-page summary of the house rules for any AI agent. Like the laminated safety card on a factory wall.
    - *Technical role:* Cross-agent instruction file (the AGENTS.md convention) read by Copilot and by Claude Code when no CLAUDE.md exists; mirrors the 10 guardrails, layers and finishing checklist.
4. `CODE_OF_CONDUCT.md` — **Green**
    - *What it is:* How people in this project are expected to treat each other. Like a club's code of behaviour.
    - *Technical role:* Community document adapted from the Contributor Covenant 2.0.
5. `CONTRIBUTING.md` — **Green**
    - *What it is:* Short rules for adding to the project: where skills, specs and tests go. Like the 'how to submit an article' note for a newsletter.
    - *Technical role:* Contributor guidance; notes a skill's folder name must match its frontmatter name and its description must say when to use it.
6. `copilot-instructions.md` — **Amber** · **AI file**
    - *What it is:* The full technical rulebook for building PBIP dashboards: file layouts, forbidden properties, JSON templates, SVG measure patterns. Like the detailed building code book.
    - *Technical role:* 727-line deep reference for agents; delegated to by .github/copilot-instructions.md. Not auto-loaded.
7. `LICENSE` — **Red**
    - *What it is:* The legal permission to use, copy and change this code. Like the terms printed on a ticket.
    - *Technical role:* MIT licence, copyright 2026 'Mofaji'.
8. `README.md` — **Green**
    - *What it is:* The front page of the repository: goals, guardrails, quick start, and how the verify loop and custom-visual kit work. Like the welcome leaflet in a hotel room.
    - *Technical role:* Markdown overview shown by GitHub on the repository home page; links to the deeper docs.
9. `Template.pbip` — **Red**
    - *What it is:* The file you double-click to open the project in Power BI Desktop. Like the front door of the house.
    - *Technical role:* PBIP entry file (pbipProperties 1.0.0 schema) whose artifacts list points at Template.Report; enableAutoRecovery is on.

14.2 Root folders

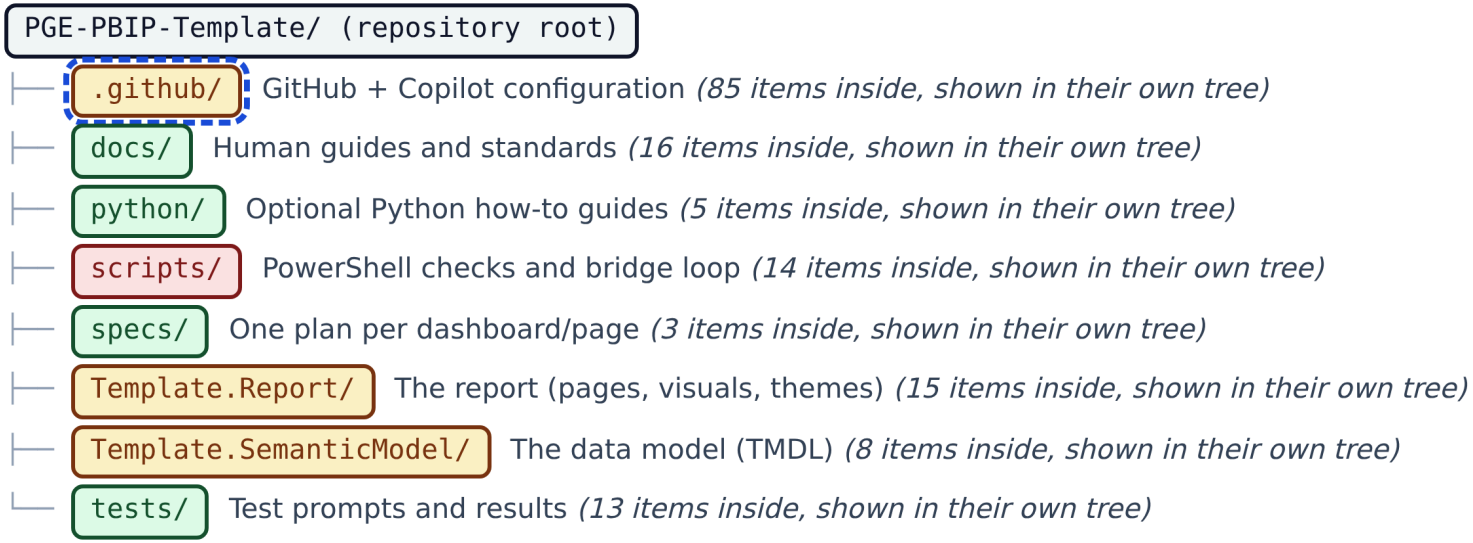


Figure 14.2 — Root folders. The nine top-level folders, each opened in its own tree below.

Walkthrough of Figure 14.2:

1. `.github/` — **Amber · AI file**
   - *What it is:* GitHub's settings folder, which also holds all the AI agent configuration. Like the control room of the building.
   - *Technical role:* Holds Copilot policy, scoped instructions, agent modes, skills, issue and PR templates, CODEOWNERS and the workflow.
2. `docs/` — **Green**
   - *What it is:* Human-readable guides and quality standards. Like the company handbook shelf.
   - *Technical role:* Documentation folder.
3. `python/` — **Green**
   - *What it is:* Optional how-to guides for Python reporting and alerts. No scripts ship here.
   - *Technical role:* Python guidance folder.
4. `scripts/` — **Red**
   - *What it is:* PowerShell scripts that check the files and drive Power BI Desktop. Like the inspection machines.
   - *Technical role:* Validation, bridge verification, AI test runner and hook wrappers.
5. `specs/` — **Green**
   - *What it is:* One plan per dashboard or page, written before building. Like architects' drawings.
   - *Technical role:* Specs folder.

6. `Template.Report/` — **Amber**
    - *What it is:* The report itself: pages, visuals and themes, as text files. Like the plated meal.
    - *Technical role:* PBIP report folder in PBIR format.
7. `Template.SemanticModel/` — **Amber**
    - *What it is:* The data model: tables, relationships and measures, as text files. Like the kitchen and pantry.
    - *Technical role:* PBIP semantic model folder in TMDL format.
8. `tests/` — **Green**
    - *What it is:* Test prompts and their results. Like the lab.
    - *Technical role:* Testing folder.

14.3 .github: top level

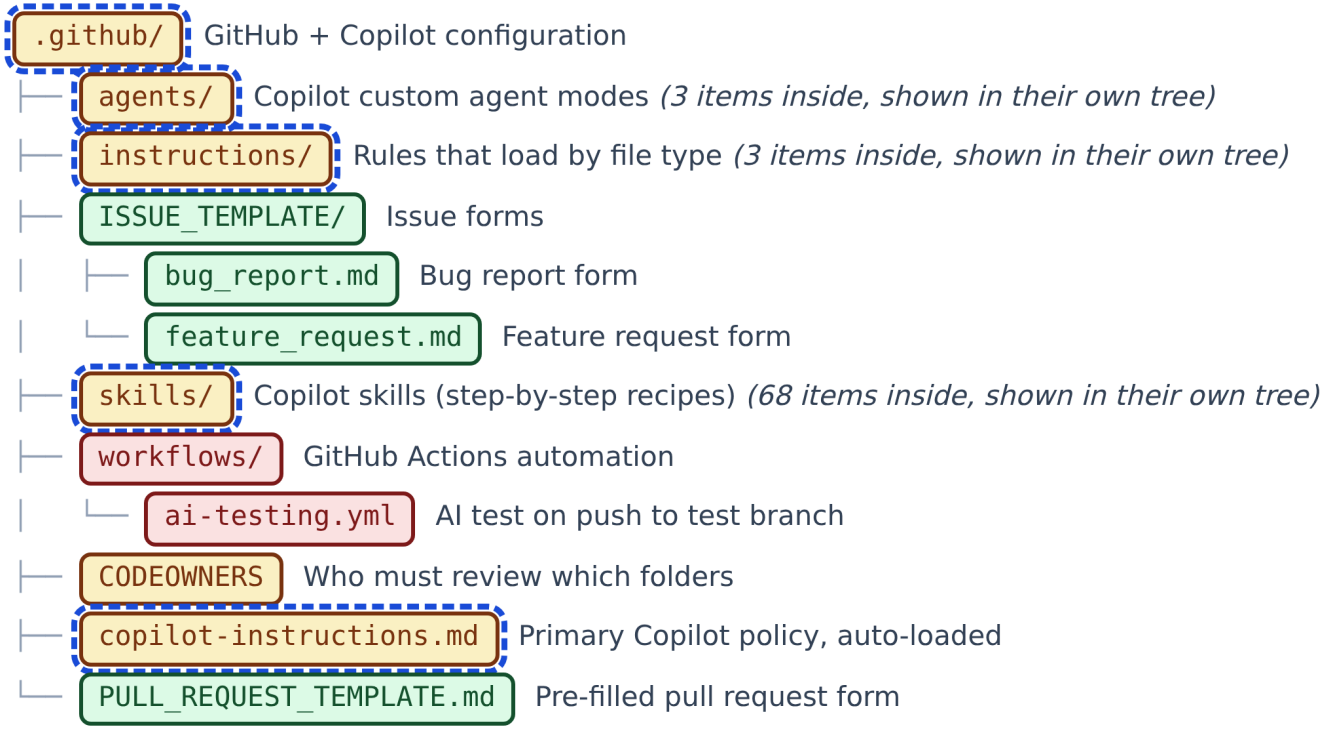


Figure 14.3 — .github: top level. GitHub's own configuration, plus the folders the AI agent reads.

Walkthrough of Figure 14.3:

1. `.github/ISSUE_TEMPLATE/` — **Green**
   - *What it is:* Ready-made forms for reporting problems or asking for features. Like the forms at a help desk.
   - *Technical role:* GitHub issue-template folder.
2. `.github/ISSUE_TEMPLATE/bug_report.md` — **Green**
   - *What it is:* The form for reporting something broken. Like a repair request slip.
   - *Technical role:* Issue template with description, steps to reproduce, expected behaviour, context.

3. `.github/ISSUE_TEMPLATE/feature_request.md` — **Green**
    - *What it is:* The form for asking for something new. Like a suggestion-box card.
    - *Technical role:* Issue template with problem, proposed solution, context.
4. `.github/workflows/` — **Red**
    - *What it is:* Automation recipes GitHub runs for you. Like timers on a kitchen appliance.
    - *Technical role:* GitHub Actions workflow folder.
5. `.github/workflows/ai-testing.yml` — **Red**
    - *What it is:* Runs the AI test automatically when code reaches the test branch. Like an automatic inspection at the factory gate.
    - *Technical role:* Workflow: push to test or workflow_dispatch; validate-json, validate-pbip, run-ai-md-test.ps1; artifact 30 days; optional commit back.
6. `.github/CODEOWNERS` — **Amber**
    - *What it is:* Says which team must review changes to which folders. Like a sign-off sheet naming the inspector for each room.
    - *Technical role:* GitHub CODEOWNERS file; owners are auto-requested for review. Handles such as @your-org/bi-modelers are placeholders to replace.
7. `.github/copilot-instructions.md` — **Amber** · **AI file**
    - *What it is:* GitHub Copilot's main rulebook for this repository, loaded automatically every time. Like the manager's standing orders.
    - *Technical role:* Primary Copilot policy (wins conflicts); 12 guardrails, a map of where things live, checklist list, theme governance and validation commands.
8. `.github/PULL_REQUEST_TEMPLATE.md` — **Green**
    - *What it is:* The form every pull request starts with. Like a delivery note with tick boxes.
    - *Technical role:* Pre-fills new pull requests with a summary section and a three-item checklist (docs, tests, structure).

14.4 .github/agents and .github/instructions

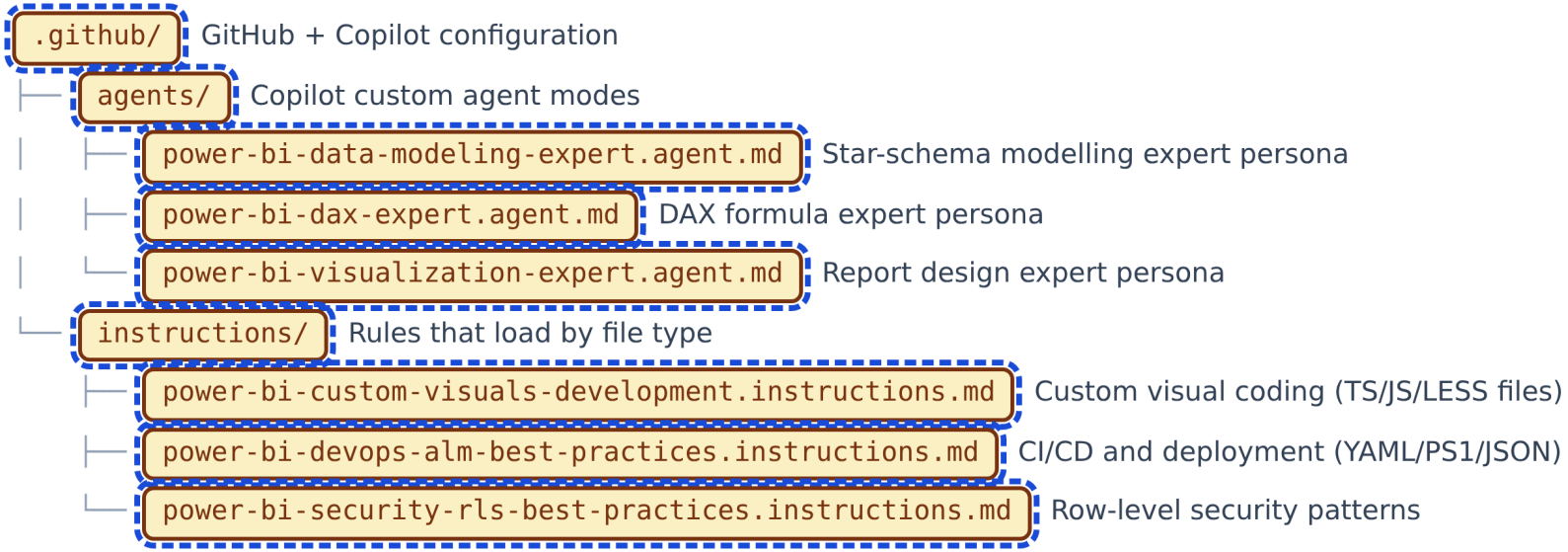


Figure 14.4 — .github/agents and .github/instructions. Expert modes and file-type rules for GitHub Copilot.

Walkthrough of Figure 14.4:

1. `.github/agents/` — **Amber · AI file**
   - *What it is:* Three 'expert hats' Copilot can put on. Like calling in a specialist consultant.
   - *Technical role:* Copilot custom agents (.agent.md): instructions plus a tool list for one role.
2. `.github/agents/power-bi-data-modeling-expert.agent.md` — **Amber · AI file**
   - *What it is:* Turns Copilot into a data-model expert (star schemas, relationships). Like a structural engineer.
   - *Technical role:* Custom agent (model gpt-4.1) with modelling guidance and repository override rules (measure homes, schema constraints).
3. `.github/agents/power-bi-dax-expert.agent.md` — **Amber · AI file**
   - *What it is:* Turns Copilot into a DAX formula expert. Like a specialist accountant for formulas.
   - *Technical role:* Custom agent with DAX best practices, anti-patterns and debugging; repository override rules for _Measures and _SVG_Measures.
4. `.github/agents/power-bi-visualization-expert.agent.md` — **Amber · AI file**
   - *What it is:* Turns Copilot into a report-design expert. Like an interior designer for dashboards.
   - *Technical role:* Custom agent with visual design, interactivity, accessibility and performance guidance; override rules for PBIP visual JSON.

5. `.github/instructions/` — **Amber** · **AI file**
    - *What it is:* Rules that switch on only for certain file types. Like safety rules that apply only in the workshop.
    - *Technical role:* Scoped Copilot instructions with applyTo glob patterns.
6. `.github/instructions/power-bi-custom-visuals-development.instructions.md` — **Amber** · **AI file**
    - *What it is:* Coding advice for writing custom charts. Like a carpentry manual for one kind of furniture.
    - *Technical role:* applyTo **/*.{ts,tsx,js,jsx,less,css}; React, D3.js, testing and performance patterns for Power BI visuals.
7. `.github/instructions/power-bi-devops-alm-best-practices.instructions.md` — **Amber** · **AI file**
    - *What it is:* Advice on releasing reports safely through test and production. Like a moving company's checklist.
    - *Technical role:* applyTo **/*.{yml,yaml,ps1,json,pbix,pbir}; deployment pipelines, REST API, Azure DevOps, environments, Key Vault; overridden by repo PBIP structure.
8. `.github/instructions/power-bi-security-rls-best-practices.instructions.md` — **Amber** · **AI file**
    - *What it is:* Advice on making sure each person sees only the data they should. Like a key-card system for rooms.
    - *Technical role:* applyTo **/*.{pbix,dax,md,txt,json,cs,ps1,psm1,psd1}; row-level security, dynamic RLS, embedded security, monitoring.

14.5 .github/skills, part 1

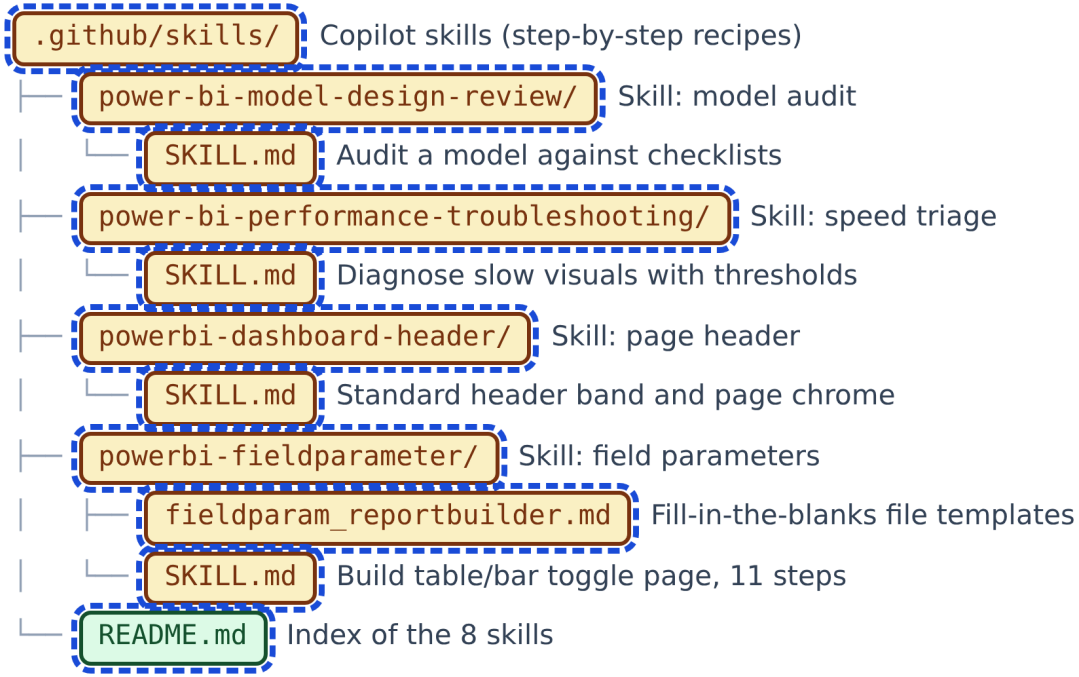


Figure 14.5 — .github/skills, part 1. The skills index and the first four skills.

Walkthrough of Figure 14.5:

1. `.github/skills/` — **Amber · AI file**
   - *What it is:* Step-by-step recipes the AI uses when a task matches. Like recipe cards in a kitchen drawer.
   - *Technical role:* Copilot agent skills: one folder per skill, each with a SKILL.md whose description routes tasks to it.
2. `.github/skills/power-bi-model-design-review/` — **Amber · AI file**
   - *What it is:* Folder for the ‘model health check’ skill.
   - *Technical role:* Skill folder.
3. `.github/skills/power-bi-model-design-review/SKILL.md` — **Amber · AI file**
   - *What it is:* A recipe for auditing an existing data model. Like a home inspection checklist.
   - *Technical role:* Skill: score checklist rows Pass/Needs Work/N/A, prioritise findings, output a Priority/Row/Status table.
4. `.github/skills/power-bi-performance-troubleshooting/` — **Amber · AI file**
   - *What it is:* Folder for the ‘why is it slow?’ skill.
   - *Technical role:* Skill folder.

5. `.github/skills/power-bi-performance-troubleshooting/SKILL.md` — **Amber** · **AI file**
    - *What it is:* A recipe for finding why a report is slow. Like a mechanic's diagnostic routine.
    - *Technical role:* Skill: thresholds (<100 ms excellent, DAX >1000 ms review, queue >500 ms review), evidence-first triage, before/after timings.
6. `.github/skills/powerbi-dashboard-header/` — **Amber** · **AI file**
    - *What it is:* Folder for the page-header skill.
    - *Technical role:* Skill folder.
7. `.github/skills/powerbi-dashboard-header/SKILL.md` — **Amber** · **AI file**
    - *What it is:* A recipe for a consistent header band across pages: logo, title, filters, buttons. Like a letterhead template.
    - *Technical role:* Skill based on a client reference page; conflicts with rule 10 on visual titles and says to resolve per project in the spec.
8. `.github/skills/powerbi-fieldparameter/` — **Amber** · **AI file**
    - *What it is:* Folder for the field-parameter page skill.
    - *Technical role:* Skill folder.
9. `.github/skills/powerbi-fieldparameter/fieldparam_reportbuilder.md` — **Amber** · **AI file**
    - *What it is:* Fill-in-the-blanks file templates for the same page. Like a form letter with blanks.
    - *Technical role:* Boilerplate with {PLACEHOLDER} values for every TMDL, visual, button, slicer and bookmark file.
10. `.github/skills/powerbi-fieldparameter/SKILL.md` — **Amber** · **AI file**

- *What it is:* A recipe for a page where users switch what a table and bar chart show. Like a TV remote that changes channels.
- *Technical role:* 11-step skill: two field-parameter tables, table + clustered bar chart, two slicers, four buttons, two bookmarks; ID generation in PowerShell.

11. `.github/skills/README.md` — **Green** · **AI file**

- *What it is:* The index card listing the eight skills. Like the contents page of the recipe box.
- *Technical role:* Lists the 8 skills and points to docs/checklists/.

14.6 .github/skills, part 2

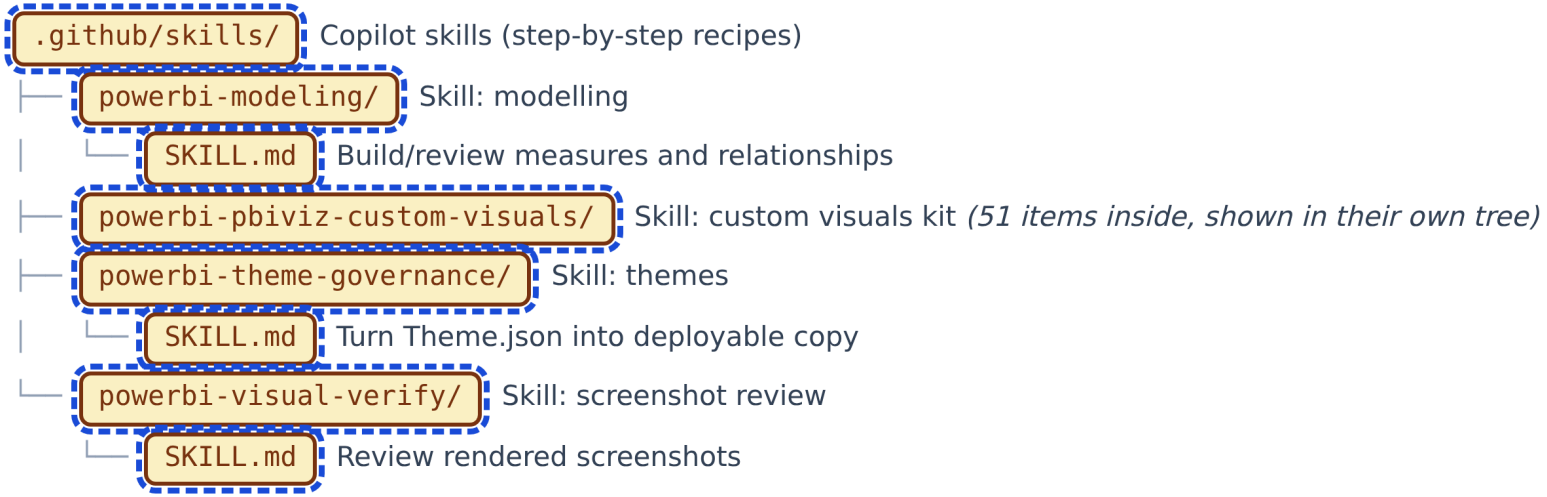


Figure 14.6 — .github/skills, part 2. Three more skills, and the custom-visual kit, which gets its own trees.

Walkthrough of Figure 14.6:

1. `.github/skills/powerbi-modeling/` — **Amber** · **AI file**
   - *What it is:* Folder for the modelling skill.
   - *Technical role:* Skill folder.
2. `.github/skills/powerbi-modeling/SKILL.md` — **Amber** · **AI file**
   - *What it is:* A recipe for building and reviewing measures and relationships. Like a builder's daily checklist.
   - *Technical role:* Skill: repository rules first, then relevant checklist rows, output Row/Area/Status/Evidence/Action table.
3. `.github/skills/powerbi-theme-governance/` — **Amber** · **AI file**
   - *What it is:* Folder for the theme skill.
   - *Technical role:* Skill folder.
4. `.github/skills/powerbi-theme-governance/SKILL.md` — **Amber** · **AI file**
   - *What it is:* A recipe for turning the editable theme into a safe, loadable one. Like proof-reading before printing.
   - *Technical role:* Skill: good->maximum, bad->minimum, add null, move #FFFFFF, keep 10 allowed root keys.
5. `.github/skills/powerbi-visual-verify/` — **Amber** · **AI file**
   - *What it is:* Folder for the screenshot-review skill.
   - *Technical role:* Skill folder.
6. `.github/skills/powerbi-visual-verify/SKILL.md` — **Amber** · **AI file**
   - *What it is:* A recipe for checking screenshots of the finished pages. Like a quality inspector's sheet.
   - *Technical role:* Skill: score rows, cite evidence, fix source not images, report by page display name; theme-specific checks.

14.7 Custom-visual kit: project files and tools

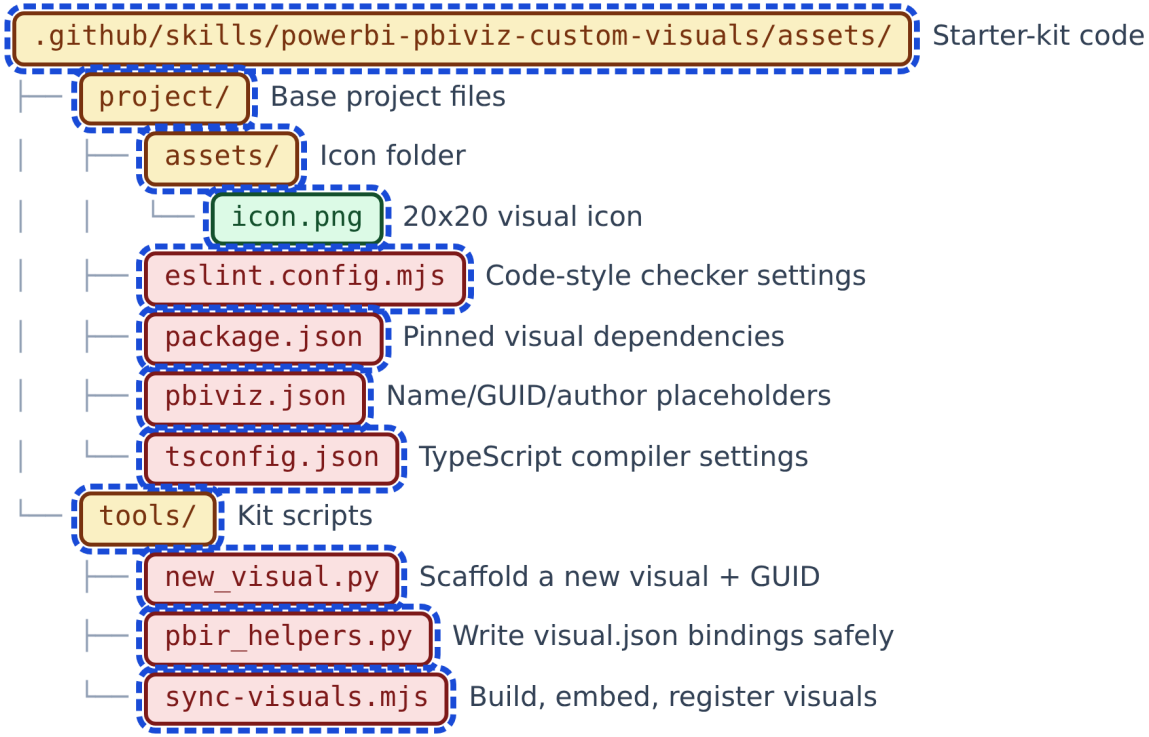


Figure 14.7 — Custom-visual kit: project files and tools. The base project every visual starts from, and the three kit scripts.

Walkthrough of Figure 14.7:

1. `.github/skills/powerbi-pbiviz-custom-visuals/assets/` — **Amber · AI file**
   - *What it is:* The starter-kit code the skill copies into a dashboard repository.
   - *Technical role:* Assets folder: project/, shared/, templates/, tools/.
2. `.github/skills/powerbi-pbiviz-custom-visuals/assets/project/` — **Amber · AI file**
   - *What it is:* The files every new visual project starts with. Like the blank chassis of a car.
   - *Technical role:* Base pbiviz project files copied by new_visual.py.

3. `.github/skills/powerbi-pbiviz-custom-visuals/assets/project/assets/` — **Amber** · **AI file**
    - *What it is:* Folder for the visual's icon.
    - *Technical role:* Icon folder referenced by pbiviz.json.
4. `.github/skills/powerbi-pbiviz-custom-visuals/assets/project/assets/icon.png` — **Green** · **AI file**
    - *What it is:* The tiny 20 x 20 picture shown for the visual in Power BI. Like a shop's sign.
    - *Technical role:* PNG icon referenced as assets.icon in pbiviz.json.
5. `.github/skills/powerbi-pbiviz-custom-visuals/assets/project/eslint.config.mjs` — **Red** · **AI file**
    - *What it is:* Settings for the code-style checker. Like a spell-checker's rule list.
    - *Technical role:* ESLint config using eslint-plugin-powerbi-visuals recommended rules; ignores node_modules, dist, .vscode, .tmp.
6. `.github/skills/powerbi-pbiviz-custom-visuals/assets/project/package.json` — **Red** · **AI file**
    - *What it is:* The shopping list of code libraries the visual needs, with exact versions. Like a parts list.
    - *Technical role:* npm manifest: powerbi-visuals-api 5.11.1, formattingmodel 6.0.4, TypeScript 5.5.4, ESLint; scripts pbiviz/start/package/lint.
7. `.github/skills/powerbi-pbiviz-custom-visuals/assets/project/pbiviz.json` — **Red** · **AI file**
    - *What it is:* The visual's identity card with blanks for name, GUID and author. Like an unfilled passport.
    - *Technical role:* pbiviz manifest with **NAME**, **GUID**, **AUTHOR_NAME** etc. placeholders; apiVersion 5.11.1.
8. `.github/skills/powerbi-pbiviz-custom-visuals/assets/project/tsconfig.json` — **Red** · **AI file**
    - *What it is:* Settings for turning TypeScript into JavaScript. Like the translator's style sheet.
    - *Technical role:* TypeScript compiler options (ES2022, outDir .tmp/build/, src/visual.ts).
9. `.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/` — **Amber** · **AI file**
    - *What it is:* Scripts that create, build and wire up visuals. Like power tools.
    - *Technical role:* new_visual.py, sync-visuals.mjs, pbir_helpers.py.
10. `.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/new_visual.py` — **Red** · **AI file**

- *What it is:* Creates a new visual project from a template and gives it a permanent ID. Like a birth certificate office.

- *Technical role:* Python scaffolder: copies template, mints GUID, writes tools/visuals.json, installs shared lib and sync tooling, runs npm install (or –node-modules-from).

11. `.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/pbir_helpers.py` — **Red** · **AI file**

- *What it is:* Helpers for writing a page's visual files correctly. Like a form-filling assistant.

- *Technical role:* Builders custom_visual, col, meas, props, literal encoders s/b/d/i, categorical_filter, check_against_capabilities; schema visualContainer 2.12.0.

12. `.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/sync-visuals.mjs` — **Red** · **AI file**

- *What it is:* Builds every visual and plugs it into the report. Like a factory line that also installs the product.

- *Technical role:* Node script: sync shared code, bump build, pbiviz package, embed payload, register in report.json, verify references; –only, –check, –pbiviz-too, –project.

14.8 Custom-visual kit: shared library

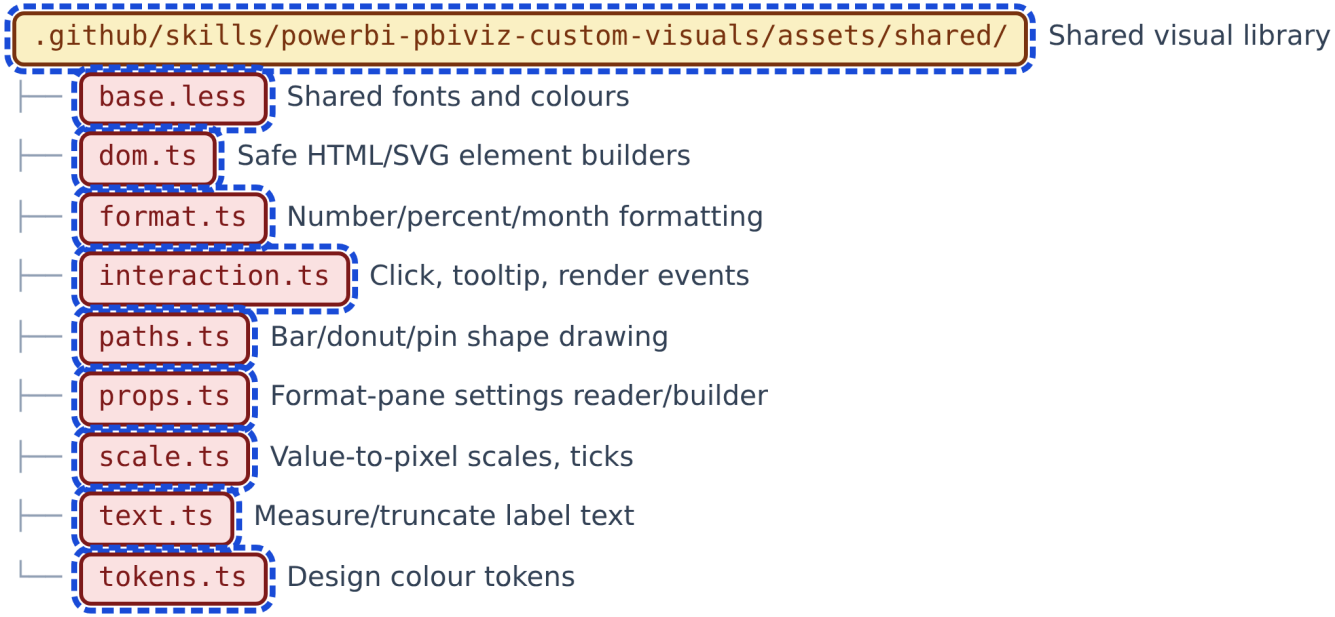


Figure 14.8 — Custom-visual kit: shared library. Code shared by every custom visual.

Walkthrough of Figure 14.8:

1. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/` — **Amber** · **AI file**
   - *What it is:* Code shared by every custom visual. Like a shared toolbox in a workshop.
   - *Technical role:* Shared TypeScript/LESS library synced into each visual's src/shared/ at build time.
2. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/base.less` — **Red** · **AI file**
   - *What it is:* Shared fonts and colours for every visual's styling. Like a paint colour card.
   - *Technical role:* LESS variables mirroring tokens.ts plus base html/body and empty-state styles.
3. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/dom.ts` — **Red** · **AI file**
   - *What it is:* Safe helpers for creating page elements. Like pre-cut building blocks.
   - *Technical role:* createElement/createElementNS helpers (el, svg, svgText, clear, append); no innerHTML, which the lint step forbids.
4. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/format.ts` — **Red** · **AI file**
   - *What it is:* Turns raw numbers into readable labels such as thousands and percentages. Like a price-tag printer.
   - *Technical role:* fmtValue, fmtDelta, fmtPct, fmtDeltaPct, fmtDeltaPp, fmtMonth; units K/M/none.

5. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/interaction.ts` — **Red** · **AI file**
    - *What it is:* Makes visuals respond to clicks and hovering. Like the buttons on a remote control.
    - *Technical role:* Interactions class wrapping selection manager, tooltip service and render events; CLEAR_CATCHER; markOpacity.
6. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/paths.ts` — **Red** · **AI file**
    - *What it is:* Draws shapes such as rounded bars and donut rings. Like stencils.
    - *Technical role:* SVG path builders: roundedTopBar, donutArc, pinTeardrop.
7. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/props.ts` — **Red** · **AI file**
    - *What it is:* Reads and builds the visual's settings panel. Like the control panel wiring.
    - *Technical role:* PropSpec-driven readCard/buildCard/buildModel for the Format pane; literal-encoding notes.
8. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/scale.ts` — **Red** · **AI file**
    - *What it is:* Converts numbers into positions on the screen. Like a ruler that maps money to centimetres.
    - *Technical role:* linearScale, bandScale, ticksByStep, niceStep, niceMax.
9. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/text.ts` — **Red** · **AI file**
    - *What it is:* Measures and shortens labels so they fit. Like trimming a caption to fit a frame.
    - *Technical role:* measureText via an offscreen canvas; truncateToWidth.
10. `.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/tokens.ts` — **Red** · **AI file**

- *What it is:* The master list of colours and fonts for the visuals. Like a brand guide.
- *Technical role:* Design tokens T: neutral greys, IBCS scenario colours (ac, pm), variance colours (pos, neg), swappable accent.

14.9 Custom-visual kit: templates, blank

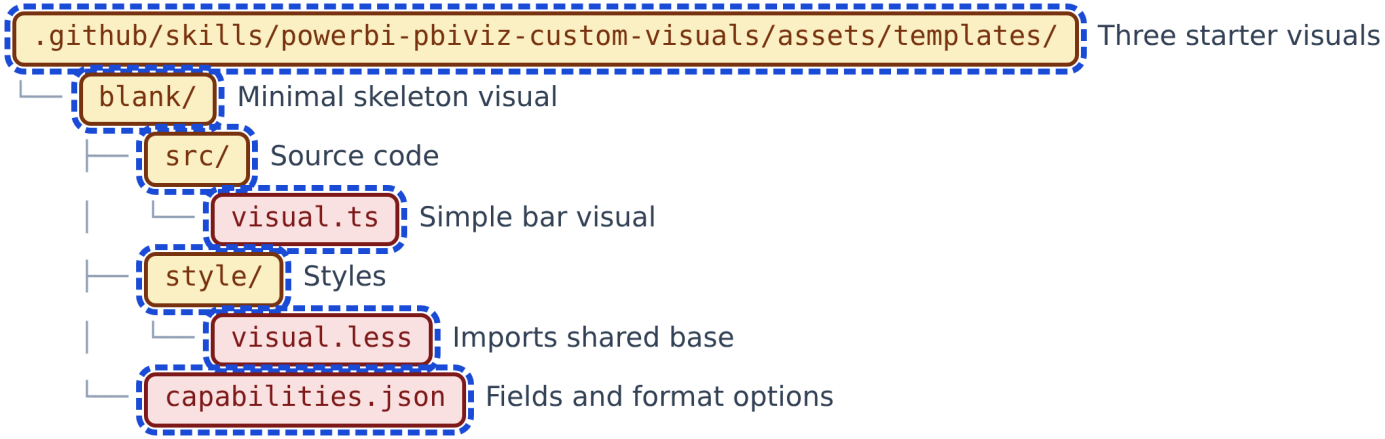


Figure 14.9 — Custom-visual kit: templates, blank. The templates folder and the minimal ‘blank’ starter visual.

Walkthrough of Figure 14.9:

1. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/` — **Amber** · **AI file**
   - *What it is:* Three ready-to-adapt example visuals. Like model homes.
   - *Technical role:* Starter templates copied by new_visual.py –template.
2. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/` — **Amber** · **AI file**
   - *What it is:* The simplest example visual: one category, one number, horizontal bars.
   - *Technical role:* Template folder.
3. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/src/` — **Amber** · **AI file**
   - *What it is:* Source-code folder.
   - *Technical role:* Template source folder.
4. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/src/visual.ts` — **Red** · **AI file**
   - *What it is:* The drawing code for the simple bar visual. Like the engine.
   - *Technical role:* Minimal IVisual showing every required pattern: role parsing, Format pane, interactions, highlights, render events.
5. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/style/` — **Amber** · **AI file**
   - *What it is:* Style folder.
   - *Technical role:* Template style folder.
6. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/style/visual.less` — **Red** · **AI file**
   - *What it is:* The visual’s style sheet. Like its paint job.
   - *Technical role:* Imports shared/base.less; svg display block.

7. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/capabilities.json` — **Red** · **AI file**
    - *What it is:* Tells Power BI which fields this visual accepts and which settings it has. Like the sockets on a device.
    - *Technical role:* Roles category (Grouping) and measure (Measure), categorical mapping (top 500), display object, supportsHighlight, privileges [].

14.10 Custom-visual kit: templates, varianceChart

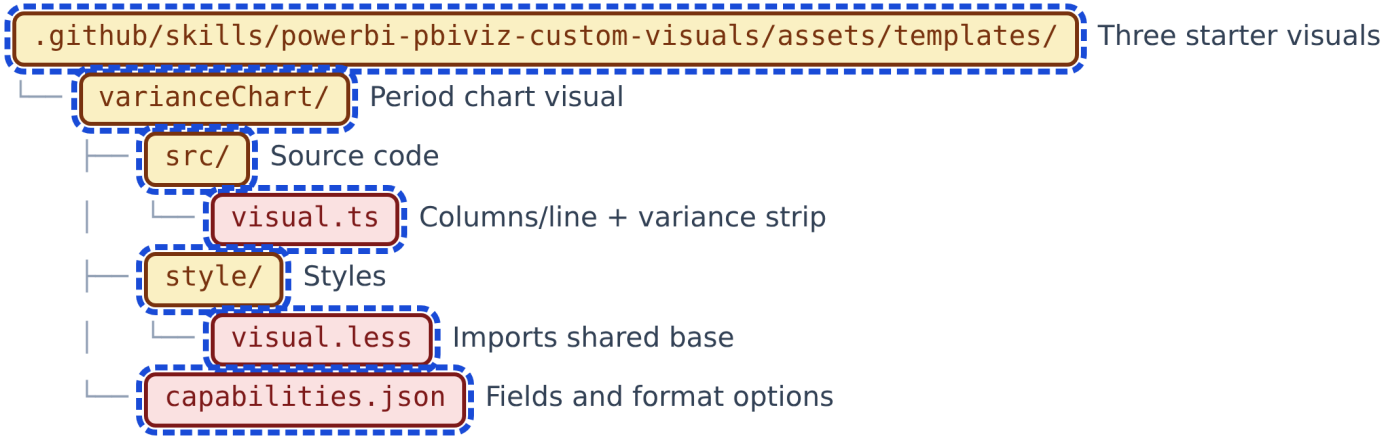


Figure 14.10 — Custom-visual kit: templates, varianceChart. The IBCS period-chart starter visual (same six-item layout).

Walkthrough of Figure 14.10:

1. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/` — **Amber · AI file**
   - *What it is:* A finance chart of values over time with a strip showing change from the previous period.
   - *Technical role:* IBCS period-chart template folder.
2. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/src/` — **Amber · AI file**
   - *What it is:* Source-code folder.
   - *Technical role:* Template source folder.
3. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/src/visual.ts` — **Red · AI file**
   - *What it is:* The drawing code for the period chart and its variance strip.
   - *Technical role:* IBCS chart: AC columns/line/area with data labels; green/red variance strip; percentage-point variances for ratios.
4. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/style/` — **Amber · AI file**
   - *What it is:* Style folder.
   - *Technical role:* Template style folder.
5. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/style/visual.less` — **Red · AI file**
   - *What it is:* The period chart's style sheet.
   - *Technical role:* Imports shared/base.less; svg display block.
6. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/capabilities.json` — **Red · AI file**
   - *What it is:* Fields and settings for the period chart. Like the sockets on a device.
   - *Technical role:* Roles category (Period) and measure (Value AC); chart type column/line/area, value format, variance options.

14.11 Custom-visual kit: templates, varianceTable

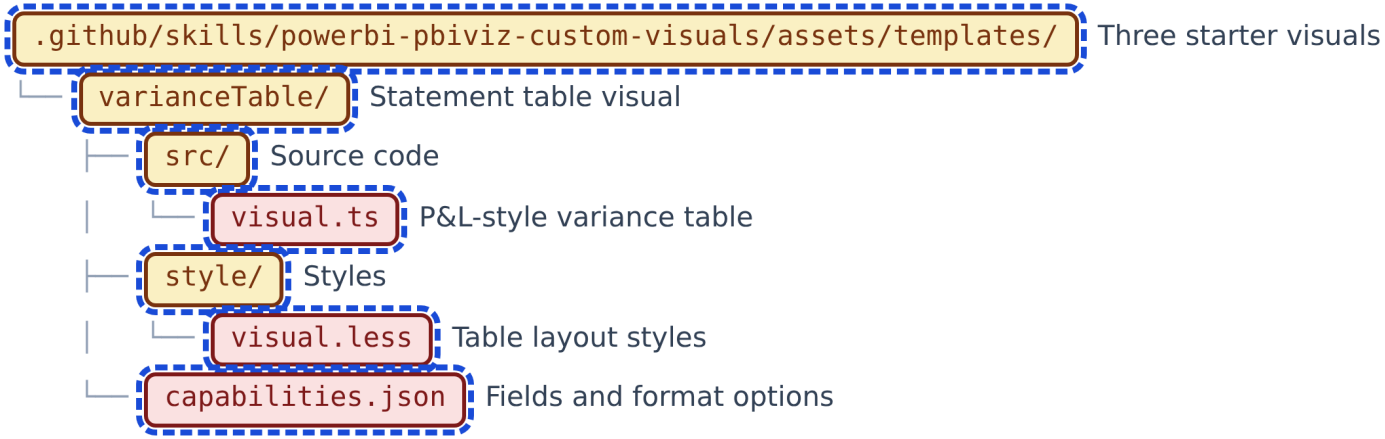


Figure 14.11 — Custom-visual kit: templates, varianceTable. The IBCS statement-table starter visual (same six-item layout).

Walkthrough of Figure 14.11:

1. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/` — **Amber · AI file**
   - *What it is:* A finance statement table, such as a profit-and-loss, with variance bars.
   - *Technical role:* IBCS statement-table template folder.
2. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/src/` — **Amber · AI file**
   - *What it is:* Source-code folder.
   - *Technical role:* Template source folder.
3. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/src/visual.ts` — **Red · AI file**
   - *What it is:* The drawing code for the statement table. Like an accountant's ledger that draws itself.
   - *Technical role:* Two layouts (variance / cross-tab), running P&L results, ratio rows, expand/collapse, cross-filtering; additive measures only.
4. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/style/` — **Amber · AI file**
   - *What it is:* Style folder.
   - *Technical role:* Template style folder.
5. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/style/visual.less` — **Red · AI file**
   - *What it is:* The statement table's style sheet.
   - *Technical role:* Table layout styles (rows, columns, tabular numbers) on top of shared/base.less.
6. `.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/capabilities.json` — **Red · AI file**
   - *What it is:* Fields and settings for the statement table.
   - *Technical role:* Roles rows, rowOrder, period, ac, pm (table mapping); statement, display and colour objects.

14.12 Custom-visual kit: SKILL.md and references

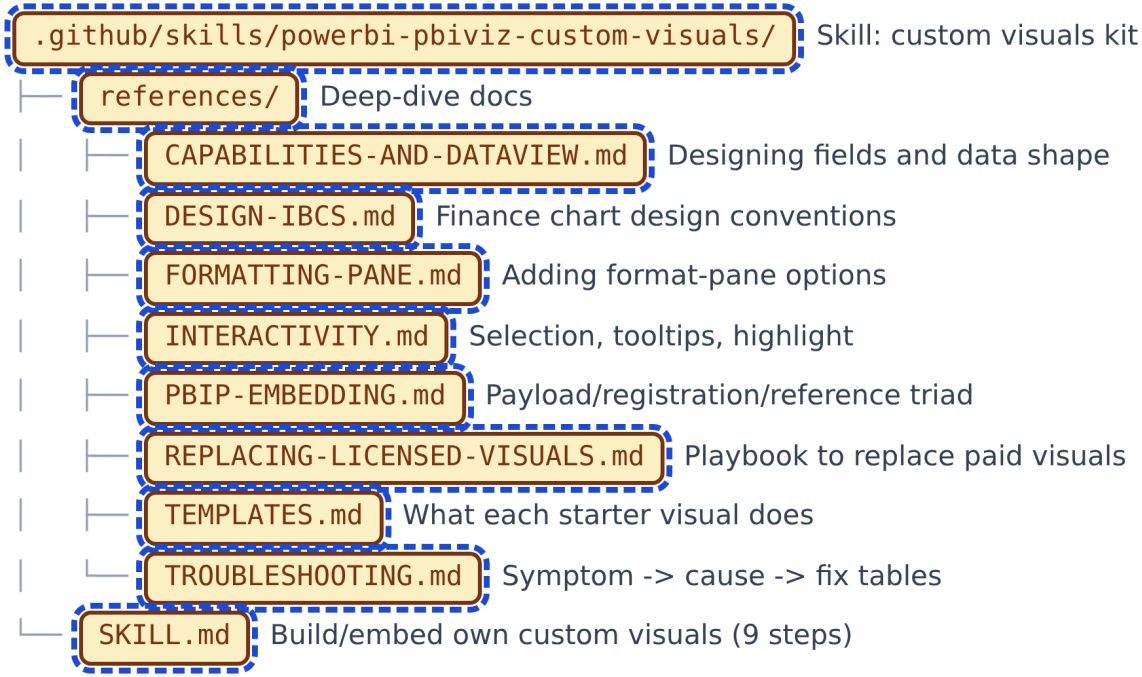


Figure 14.12 — Custom-visual kit: SKILL.md and references. The skill procedure and its eight reference documents.

Walkthrough of Figure 14.12:

1. `.github/skills/powerbi-pbiviz-custom-visuals/` — **Amber · AI file**
   - *What it is:* Folder for the custom-visual kit. Like a complete workshop for building your own chart types.
   - *Technical role:* Skill folder containing SKILL.md, assets/ and references/.
2. `.github/skills/powerbi-pbiviz-custom-visuals/references/` — **Amber · AI file**
   - *What it is:* Deep-dive manuals for the kit. Like the appendix of a workshop manual.
   - *Technical role:* Eight reference documents linked from SKILL.md.
3. `.github/skills/powerbi-pbiviz-custom-visuals/references/CAPABILITIES-AND-DATAVIEW.md` — **Amber · AI file**
   - *What it is:* How to decide which fields a visual takes and how the data arrives. Like a socket-and-plug guide.
   - *Technical role:* dataRoles, categorical vs table mappings, conditions, data reduction, dates, highlights, privileges.

4. `.github/skills/powerbi-pbiviz-custom-visuals/references/DESIGN-IBCS.md` — **Amber** · **AI file**
    - *What it is:* Design rules so custom visuals match the rest of the report. Like a style guide.
    - *Technical role:* Chrome suppression, tokens, IBCS semantics, number formats, typography, empty states.
5. `.github/skills/powerbi-pbiviz-custom-visuals/references/FORMATTING-PANE.md` — **Amber** · **AI file**
    - *What it is:* How to add settings users can change in the Format pane. Like adding knobs to a control panel.
    - *Technical role:* Declaring objects in capabilities, PropSpec reading/building, writing from visual.json, conventions.
6. `.github/skills/powerbi-pbiviz-custom-visuals/references/INTERACTIVITY.md` — **Amber** · **AI file**
    - *What it is:* How to make clicks, tooltips and highlighting work. Like wiring the doorbell.
    - *Technical role:* Selection IDs, standard update() shape, local expand state, page interactions, keyboard accessibility.
7. `.github/skills/powerbi-pbiviz-custom-visuals/references/PBIP-EMBEDDING.md` — **Amber** · **AI file**
    - *What it is:* How a built visual is stored inside the report so it opens anywhere. Like packing furniture inside the house.
    - *Technical role:* Payload / registration / reference triad, GUID identity, version cache, Desktop-closed rule, repo layout.
8. `.github/skills/powerbi-pbiviz-custom-visuals/references/REPLACING-LICENSED-VISUALS.md` — **Amber** · **AI file**
    - *What it is:* Step-by-step plan for replacing a paid chart whose licence expired. Like a renovation plan.
    - *Technical role:* Playbook steps 0-7: get to PBIR, inventory, map roles, profile data, decide what not to copy, build, verify, report.
9. `.github/skills/powerbi-pbiviz-custom-visuals/references/TEMPLATES.md` — **Amber** · **AI file**
    - *What it is:* What each starter visual does and how to bind it. Like model-home brochures.
    - *Technical role:* Roles, objects, behaviour, sizing and example bindings for varianceTable, varianceChart, blank.
10. `.github/skills/powerbi-pbiviz-custom-visuals/references/TROUBLESHOOTING.md` — **Amber** · **AI file**
    - *What it is:* A symptom-to-fix table for when things go wrong. Like a car's fault-finding chart.
    - *Technical role:* Tables for build, embedding, binding, rendering and model-side failures.
11. `.github/skills/powerbi-pbiviz-custom-visuals/SKILL.md` — **Amber** · **AI file**
    - *What it is:* The recipe for building, embedding and checking your own chart types. Like the workshop manual.
    - *Technical role:* Skill: 3 mandatory questions, workflow 0-9, 10 hard rules, starter kit table, pinned versions, definition of done.

14.13 Template.Report: definition

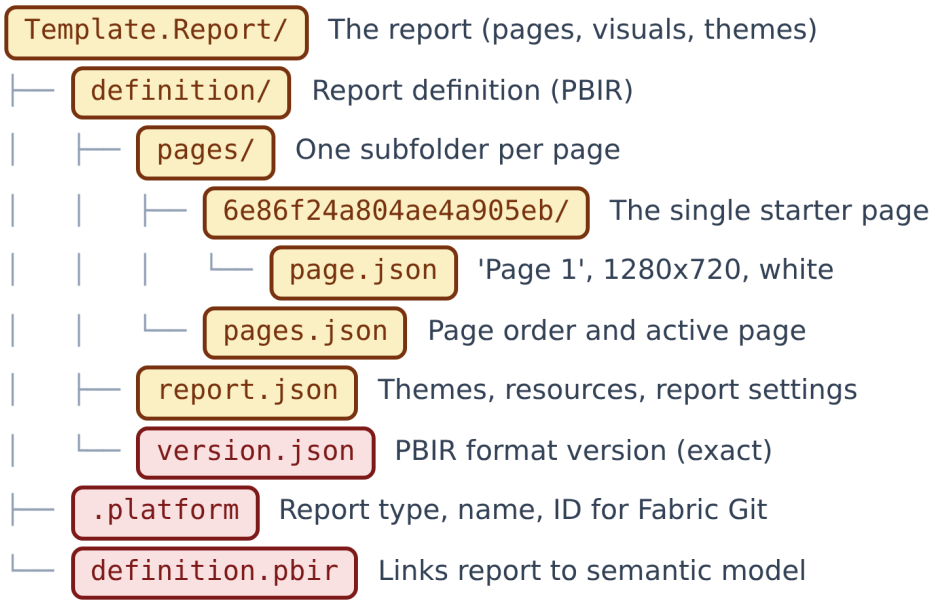


Figure 14.13 — Template.Report: definition. The report's pointer file, ID card and definition files.

Walkthrough of Figure 14.13:

1. `Template.Report/definition/` — **Amber**
   - *What it is:* The report's detailed definition files. Like the report's filing drawer.
   - *Technical role:* PBIR definition folder.
2. `Template.Report/definition/pages/` — **Amber**
   - *What it is:* One folder per report page, plus the page order. Like pages in a binder.
   - *Technical role:* Pages folder.
3. `Template.Report/definition/pages/6e86f24a804ae4a905eb/` — **Amber**
   - *What it is:* The folder for the only page, named by a machine-made ID.
   - *Technical role:* Page folder (20-character hex id).
4. `Template.Report/definition/pages/6e86f24a804ae4a905eb/page.json` — **Amber**
   - *What it is:* Describes the blank starter page: name 'Page 1', size, white surround. Like a blank canvas on an easel.
   - *Technical role:* page 2.1.0 schema: displayName Page 1, FitToPage, 1280x720, outspace #FFFFFF.
5. `Template.Report/definition/pages/pages.json` — **Amber**
   - *What it is:* The order of pages and which one opens first. Like a table of contents.
   - *Technical role:* pagesMetadata: pageOrder [6e86f24a804ae4a905eb], activePageName.

6. `Template.Report/definition/report.json` — **Amber**
    - *What it is:* Report-wide settings and the list of themes and resources. Like the report's master settings page.
    - *Technical role:* report 3.2.0 schema: themeCollection (CY26SU02 + PGEColors), resourcePackages, section alignment, report settings.
7. `Template.Report/definition/version.json` — **Red**
    - *What it is:* A tiny file stating the report format version. Must be exactly right. Like a format stamp.
    - *Technical role:* versionMetadata 1.0.0 schema with version 2.0.0; copilot-instructions.md gives its exact required content.
8. `Template.Report/.platform` — **Red**
    - *What it is:* The report's ID card for Microsoft Fabric's Git sync. Like a passport number.
    - *Technical role:* Fabric platform file: type Report, displayName Template, logicalId GUID.
9. `Template.Report/definition.pbir` — **Red**
    - *What it is:* Tells the report where its data model is. Like a forwarding address.
    - *Technical role:* Report definition properties (version 4.0); datasetReference.byPath ../Template.SemanticModel.

14.14 Template.Report: StaticResources



Figure 14.14 — Template.Report: StaticResources. Themes and images used by the report.

Walkthrough of Figure 14.14:

1. `Template.Report/StaticResources/` — **Amber**
   - *What it is:* Pictures and themes the report uses. Like the report's storeroom.
   - *Technical role:* StaticResources folder.
2. `Template.Report/StaticResources/RegisteredResources/` — **Amber**
   - *What it is:* Your own themes and background pictures, each listed in report.json. Like your own shelf in the storeroom.
   - *Technical role:* RegisteredResources package folder.
3. `Template.Report/StaticResources/RegisteredResources/Theme.deployable.json` — **Amber**
   - *What it is:* The team colour theme the report actually uses. Like the paint actually on the walls.
   - *Technical role:* PGEColors custom theme, registered as type CustomTheme; identical to docs/theme/Theme.deployable.json.
4. `Template.Report/StaticResources/SharedResources/` — **Red**
   - *What it is:* Resources supplied by Microsoft. Like the landlord's fixtures.
   - *Technical role:* SharedResources package folder.
5. `Template.Report/StaticResources/SharedResources/BaseThemes/` — **Red**
   - *What it is:* Microsoft's base themes.
   - *Technical role:* BaseThemes folder.
6. `Template.Report/StaticResources/SharedResources/BaseThemes/CY26SU02.json` — **Red**
   - *What it is:* Microsoft's default theme underneath the team theme. Like the primer coat under the paint.
   - *Technical role:* Base theme CY26SU02 (940 lines) registered under SharedResources.

14.15 Template.SemanticModel

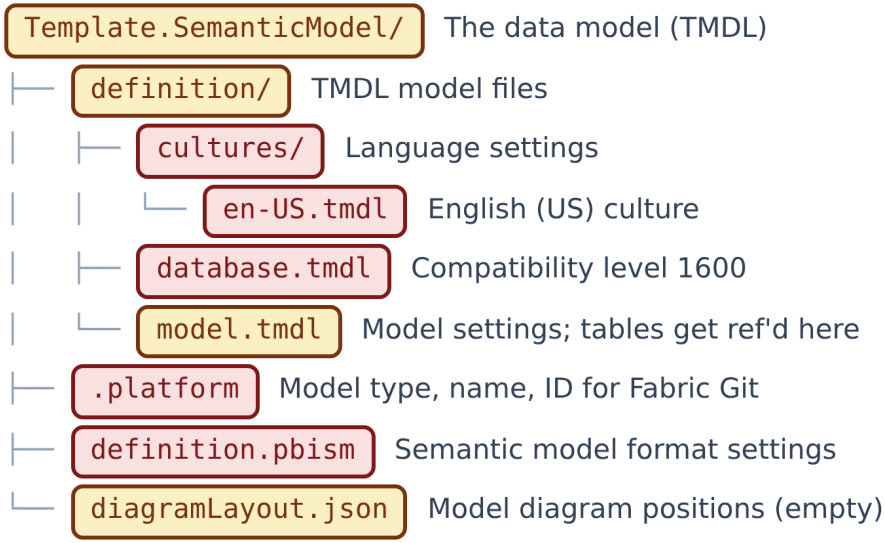


Figure 14.15 — Template.SemanticModel. The data model, as TMDL text files.

Walkthrough of Figure 14.15:

1. `Template.SemanticModel/definition/` — **Amber**
   - *What it is:* The TMDL files describing the model.
   - *Technical role:* TMDL definition folder; tables/ is added when tables exist.
2. `Template.SemanticModel/definition/cultures/` — **Red**
   - *What it is:* Language settings folder.
   - *Technical role:* Cultures folder.
3. `Template.SemanticModel/definition/cultures/en-US.tmdl` — **Red**
   - *What it is:* Declares English (US) as the model language. Like setting the language on a phone.
   - *Technical role:* cultureInfo en-US.
4. `Template.SemanticModel/definition/database.tmdl` — **Red**
   - *What it is:* Sets the model engine's compatibility level. Like the voltage rating on an appliance.
   - *Technical role:* database compatibilityLevel 1600.
5. `Template.SemanticModel/definition/model.tmdl` — **Amber**
   - *What it is:* Model-wide settings; tables get listed here as they are added. Like the pantry's index.
   - *Technical role:* model Model: culture en-US, powerBI_V3 data source version, time-intelligence annotation 1, PBI_ProTooling DevMode, ref cultureInfo en-US.

6. `Template.SemanticModel/.platform` — **Red**
    - *What it is:* The model's ID card for Microsoft Fabric's Git sync.
    - *Technical role:* Fabric platform file: type SemanticModel, displayName Template, logicalId GUID.
7. `Template.SemanticModel/definition.pbism` — **Red**
    - *What it is:* Says which model format version this is. Like an edition number.
    - *Technical role:* semanticModel definitionProperties 1.0.0 schema, version 4.2.
8. `Template.SemanticModel/diagramLayout.json` — **Amber**
    - *What it is:* Remembers where tables sit in Power BI's model diagram. Like a seating plan. Empty for now.
    - *Technical role:* Diagram layout (All tables) with no nodes.

14.16 docs: guides

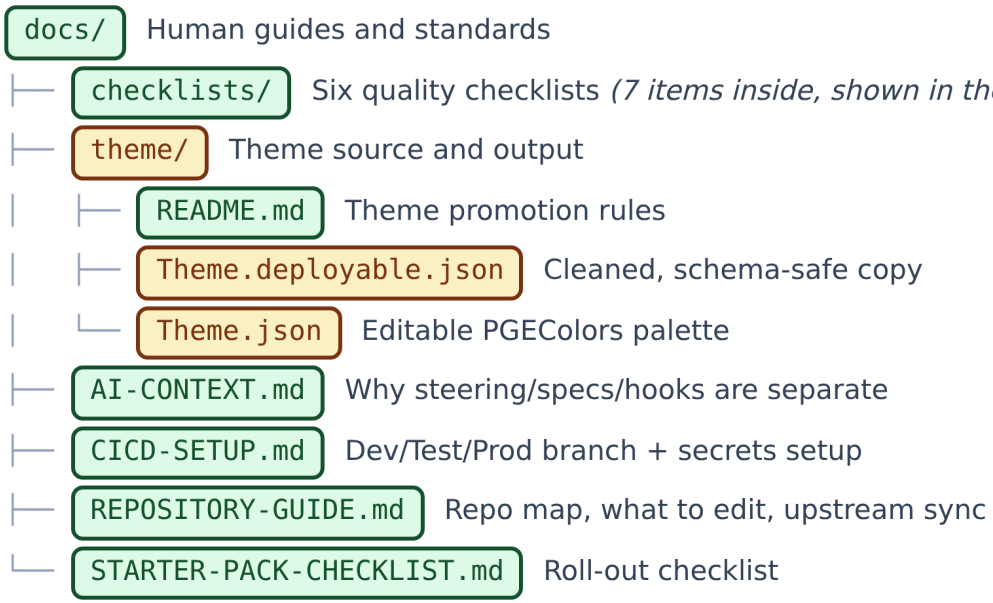


Figure 14.16 — docs: guides. Human guides, the theme source, and the checklists folder.

Walkthrough of Figure 14.16:

1. `docs/theme/` — **Amber**
   - *What it is:* The team theme's editable source and its cleaned-up copy.
   - *Technical role:* Theme source and deployable output folder.
2. `docs/theme/README.md` — **Green**
   - *What it is:* Explains how the editable theme becomes the deployable one. Like a recipe card for the paint mix.
   - *Technical role:* Promotion rules and checklist alignment for Theme.json -> Theme.deployable.json.
3. `docs/theme/Theme.deployable.json` — **Amber**
   - *What it is:* The cleaned theme, safe to load into a report. Like paint mixed and ready to use.
   - *Technical role:* Deployable theme: maximum/minimum/null, white moved last, 10 allowed root keys.
4. `docs/theme/Theme.json` — **Amber**
   - *What it is:* The editable team colour theme (PGEColors). Like the designer's paint swatch book.
   - *Technical role:* Source theme with legacy good/bad keys and #FFFFFF first; starts with a UTF-8 BOM.

5. `docs/AI-CONTEXT.md` — **Green**
    - *What it is:* Explains why rules, plans and automatic checks are kept separate. Like the ‘why we do it this way’ chapter.
    - *Technical role:* Steering / specs / hooks model, Markdown vs JSON/TMDL/YAML, which layer a rule belongs to.
6. `docs/CICD-SETUP.md` — **Green**
    - *What it is:* How to set up dev, test and prod branches, environments and secrets. Like instructions for setting up three rooms.
    - *Technical role:* CI/CD guide: branches, environments, Fabric secrets, workspace variables, branch protection, AI and Python test stages; refers to a cd.yml not present.
7. `docs/REPOSITORY-GUIDE.md` — **Green**
    - *What it is:* A map of what to reuse, what to customise and how to get template updates. Like a floor plan with ‘staff only’ signs.
    - *Technical role:* Repo map table, files not to edit, workflow for adding an existing PBIP, validation commands, upstream sync.
8. `docs/STARTER-PACK-CHECKLIST.md` — **Green**
    - *What it is:* A to-do list for turning the template into a complete team platform. Like a move-in checklist.
    - *Technical role:* Checkbox lists: governance, secrets, model, report, testing, deployment, agentic enablement.

14.17 docs/checklists

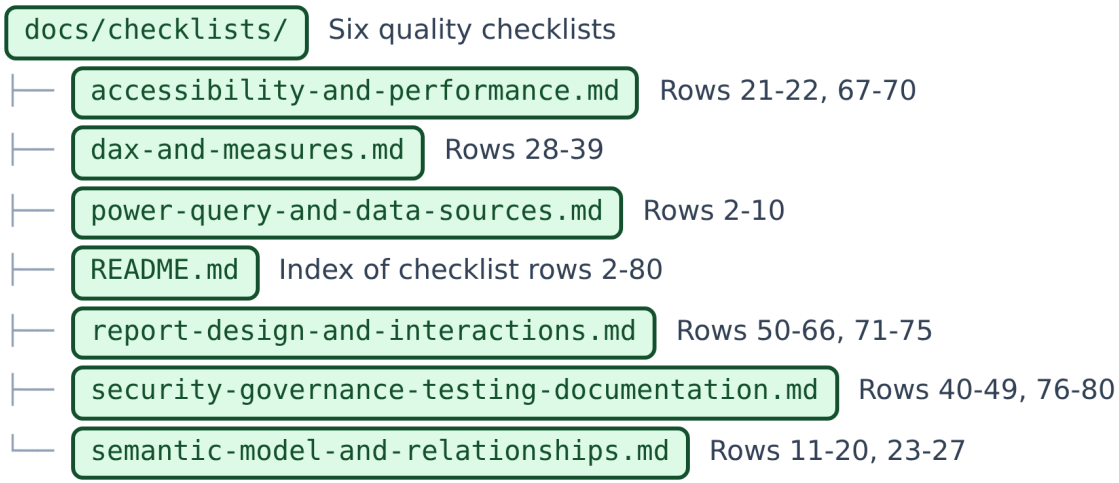


Figure 14.17 — docs/checklists. Six focused quality checklists and their index.

Walkthrough of Figure 14.17:

1. `docs/checklists/` — **Green**
   - *What it is:* Six focused quality checklists. Like inspection forms for each trade.
   - *Technical role:* Checklist folder (rows 2-80 of a modelling checklist workbook).
2. `docs/checklists/accessibility-and-performance.md` — **Green**
   - *What it is:* Checks for speed and for readability by everyone. Like a road test and an eye test.
   - *Technical role:* Rows 21-22, 67-70: visual density, slicer cardinality, contrast, text size, Performance Analyzer, query reduction.
3. `docs/checklists/dax-and-measures.md` — **Green**
   - *What it is:* Checks for well-written calculations. Like an accountant's review.
   - *Technical role:* Rows 28-39: measures vs columns, naming, explicit measures, calculation groups, date table, DIVIDE, VAR, formatting.
4. `docs/checklists/power-query-and-data-sources.md` — **Green**
   - *What it is:* Checks for how data is loaded and cleaned. Like checking deliveries at the back door.
   - *Technical role:* Rows 2-10: parameters, local files, query folding, staging, column reduction, typing, naming, documentation.
5. `docs/checklists/README.md` — **Green**
   - *What it is:* The index of the six checklists and how to use them. Like the cover sheet of an inspection pack.
   - *Technical role:* Row ranges per file; Pass / Needs Work / N/A method; skill alignment.

6. `docs/checklists/report-design-and-interactions.md` — **Green**
    - *What it is:* Checks for layout, visuals and clicks. Like a designer's review.
    - *Technical role:* Rows 50-66, 71-75: grid, theme, navigation, visual choice, labels, interactions, drillthrough, field parameters, consistency.
7. `docs/checklists/security-governance-testing-documentation.md` — **Green**
    - *What it is:* Checks for security, sign-off, testing and documentation. Like the compliance audit.
    - *Technical role:* Rows 40-49, 76-80: RLS, OLS, sensitivity labels, permissions, lineage, refresh, changelog, QA, exports, decommission.
8. `docs/checklists/semantic-model-and-relationships.md` — **Green**
    - *What it is:* Checks for a well-shaped data model. Like a structural survey.
    - *Technical role:* Rows 11-20, 23-27: star schema, cardinality, filter direction, keys, DirectQuery, Auto Date-Time, hiding, metadata.

14.18 python

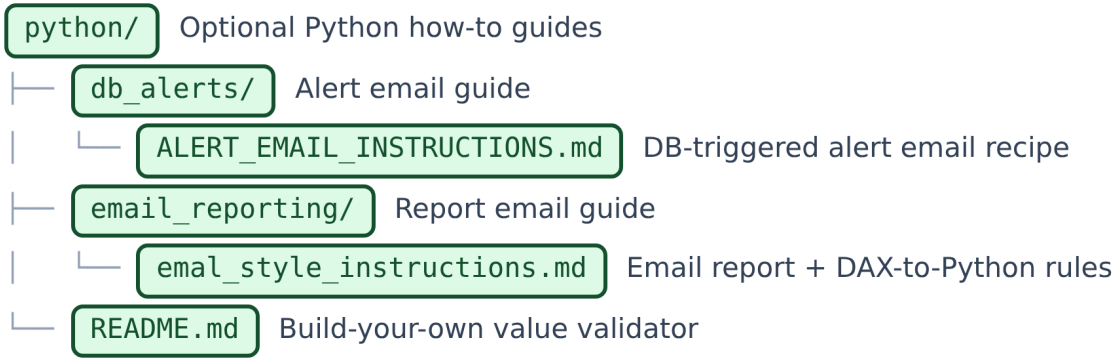


Figure 14.18 — python. Optional Python guidance; no scripts ship here.

Walkthrough of Figure 14.18:

1. `python/db_alerts/` — **Green**
   - *What it is:* The alert-email guide folder.
   - *Technical role:* Folder.
2. `python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md` — **Green**
   - *What it is:* A complete recipe for a database-triggered alert email. Like a template for a night-guard's report.
   - *Technical role:* pyodbc + pandas + python-dotenv; .env credentials; 6 phases; Outlook-safe HTML components; severity logic; checklist.
3. `python/email_reporting/` — **Green**
   - *What it is:* The report-email guide folder.
   - *Technical role:* Folder.
4. `python/email_reporting/emal_style_instructions.md` — **Green**
   - *What it is:* Rules for emails that copy a Power BI page, including how to make Python numbers match DAX. Like a translation guide.
   - *Technical role:* Three joined documents: boilerplate skill, Parts A-E (DAX-to-Python protocol, HTML, Python, charts, project ref), HTML style guide. File name has a typo ('emal').
5. `python/README.md` — **Green**
   - *What it is:* How to write your own Python checker that compares dashboard numbers with the database. Like instructions to build your own measuring tape.
   - *Technical role:* Build-your-own validator guidance: read PBIP, find Sales-by-Branch visual, generate SQL, dry-run, outputs to tests/results/.

14.19 scripts

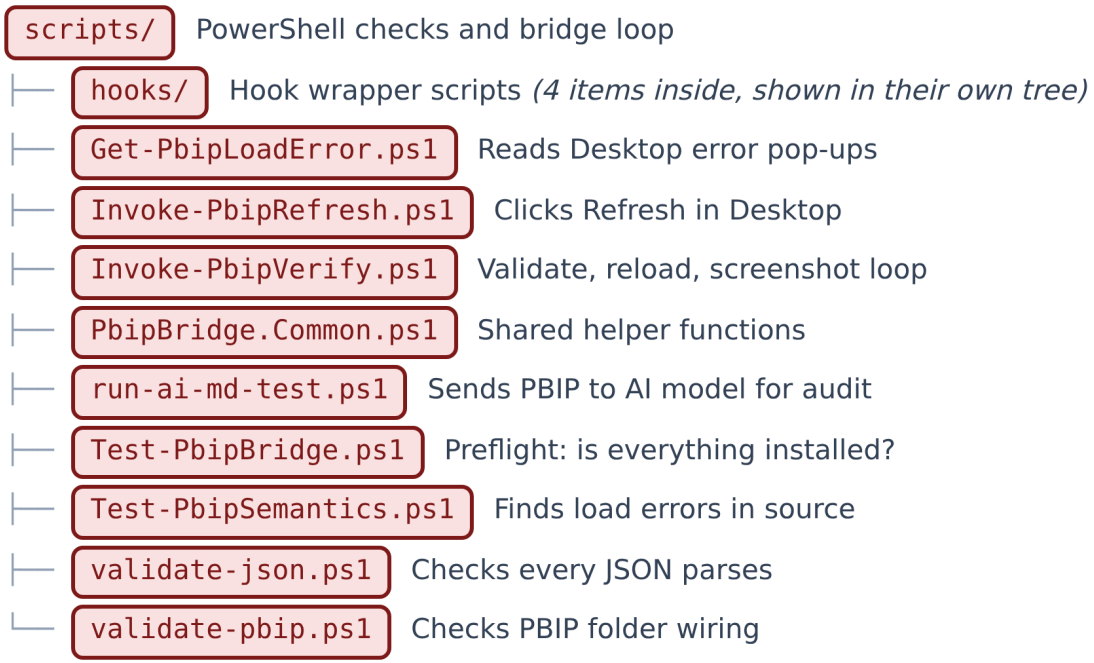


Figure 14.19 — scripts. PowerShell validation and verification scripts.

Walkthrough of Figure 14.19:

1. `scripts/Get-PbipLoadError.ps1` — **Red**
   - *What it is:* Reads Power BI's error pop-ups so they can be fixed. Like someone reading a warning light's message aloud.
   - *Technical role:* UI Automation reader for 'Issues were found' and 'Your report has issues' dialogs; root-cause collapsing; Dismiss/Continue/CloseDesktop actions.
2. `scripts/Invoke-PbipRefresh.ps1` — **Red**
   - *What it is:* Clicks Power BI's Refresh button so a new project shows data. Like pressing 'play' for someone.
   - *Technical role:* UI Automation: 'Refresh now' banner or Home ribbon Refresh, because the bridge has no refresh method.

3. `scripts/Invoke-PbipVerify.ps1` — **Red**
    - *What it is:* Reloads Power BI and takes a screenshot of every page. Like a camera that photographs every room after work.
    - *Technical role:* Verify engine: validate, semantic gate, find instance, reload, detect issues dialog, screenshot-all, page manifest; statuses and warnings.
4. `scripts/PbipBridge.Common.ps1` — **Red**
    - *What it is:* Shared helper functions used by the other bridge scripts. Like a shared toolbox.
    - *Technical role:* Project discovery, Desktop version, bridge pipe, CLI calls, instance matching, reopen, load wait, definition hash, page manifest.
5. `scripts/run-ai-md-test.ps1` — **Red**
    - *What it is:* Sends the project files and a test prompt to an AI model and saves its report. Like mailing a sample to a lab.
    - *Technical role:* AI test runner: prompt + PBIP files to a chat API; skipped mode without key; writes test_results.md, history, latest_run.json.
6. `scripts/Test-PbipBridge.ps1` — **Red**
    - *What it is:* A preflight check that everything needed for the screenshot loop is installed and open. Like a pilot's pre-flight checklist.
    - *Technical role:* Read-only: project, Desktop version >= 2.155.756.0, both CLIs, bridge pipe, matching instance, semantic and PBIR validation.
7. `scripts/Test-PbipSemantics.ps1` — **Red**
    - *What it is:* Finds, in the text, the errors Power BI would otherwise show as a pop-up. Like a spell-checker for the model.
    - *Technical role:* File-local and cross-file semantic checks on TMDL and PBIR; -Path for one file; -Quiet.
8. `scripts/validate-json.ps1` — **Red**
    - *What it is:* Checks that every JSON-style file can be read. Like checking every envelope is sealed.
    - *Technical role:* Parses *.json, *.pbip, *.pbir, *.pbism recursively, excluding dependency folders, package-lock.json and tsconfig.json.
9. `scripts/validate-pbip.ps1` — **Red**
    - *What it is:* Checks the project folders are wired together properly. Like checking every pipe connects.
    - *Technical role:* Follows .pbip -> report -> definition.pbir -> model; required files; resource items exist; custom visual GUIDs registered.

14.20 scripts/hooks

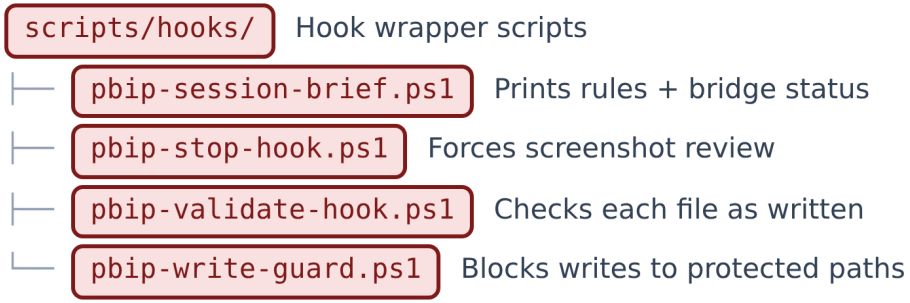


Figure 14.20 — scripts/hooks. The four hook wrapper scripts.

Walkthrough of Figure 14.20:

1. `scripts/hooks/` — **Red**
   - *What it is:* Small wrapper scripts meant to run at fixed moments. Like smoke alarms.
   - *Technical role:* Hook wrappers; fail open; read hook JSON from standard input.
2. `scripts/hooks/pbip-session-brief.ps1` — **Red**
   - *What it is:* Prints the house rules and whether Power BI's bridge is ready. Like a morning briefing.
   - *Technical role:* Session-start brief; reads the bridge named pipe; fails open and silent.
3. `scripts/hooks/pbip-stop-hook.ps1` — **Red**
   - *What it is:* Makes sure screenshots are reviewed before work is called finished. Like a supervisor's final sign-off.
   - *Technical role:* Stop wrapper: content-hash change detection, one-round fix budget, runs Invoke-PbipVerify, blocks with the report.
4. `scripts/hooks/pbip-validate-hook.ps1` — **Red**
   - *What it is:* Checks each file the moment it is written. Like a proof-reader at your shoulder.
   - *Technical role:* Post-write: JSON parse and filter checks; TMDL BOM, // comments, file-local semantics; exit 2 on failure.
5. `scripts/hooks/pbip-write-guard.ps1` — **Red**
   - *What it is:* Stops the agent writing into protected folders. Like a 'staff only' door lock.
   - *Technical role:* Blocks Write/Edit/NotebookEdit to Proposals/, **/.pbi/, tests/screenshots/, .github/.pbip-state/ with exit code 2.

14.21 specs

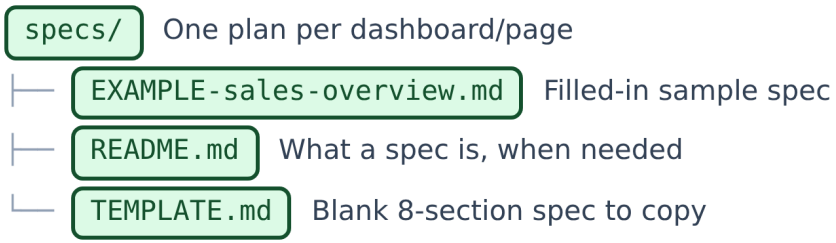


Figure 14.21 — specs. Plans for one dashboard or page at a time.

Walkthrough of Figure 14.21:

1. `specs/EXAMPLE-sales-overview.md` — **Green**
   - *What it is:* A filled-in example plan for a Sales Overview page. Like a sample completed form.
   - *Technical role:* Worked example (status built): tables, measures, relationships, page grid, card_y+30 placement, fold-back notes.
2. `specs/README.md` — **Green**
   - *What it is:* Explains what a spec is, when one is required, and the definition of done. Like the cover note on a drawing set.
   - *Technical role:* Spec purpose, when required, lifecycle, naming, difference from tests/prompts, definition of done.
3. `specs/TEMPLATE.md` — **Green**
   - *What it is:* The blank eight-part plan to copy for every new page. Like a blank order form.
   - *Technical role:* Sections: goal, data, pages, rules, build tasks, definition of done, open questions, fold-back.

14.22 tests: prompts

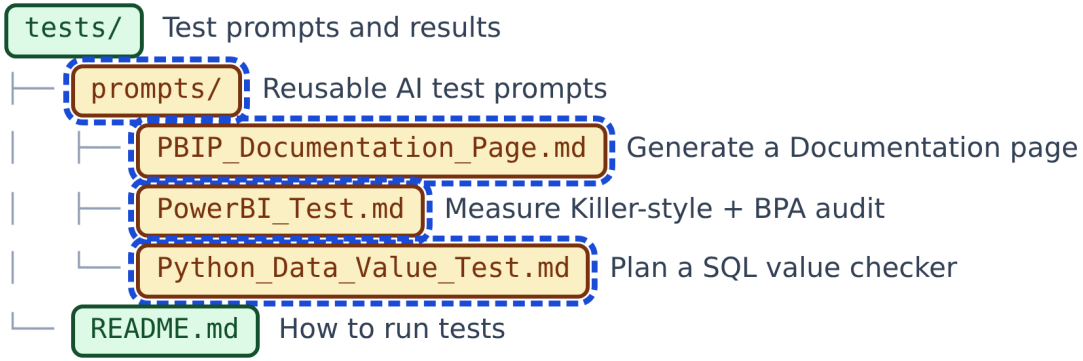


Figure 14.22 — tests: prompts. The tests folder, its guide and the three AI prompts.

Walkthrough of Figure 14.22:

1. `tests/prompts/` — **Amber** · **AI file**
   - *What it is:* Reusable instructions you give the AI to run a test or job. Like standard lab procedures.
   - *Technical role:* Execution prompts reused unchanged.
2. `tests/prompts/PBIP_Documentation_Page.md` — **Amber** · **AI file**
   - *What it is:* The AI prompt that builds a Documentation page summarising the project. Like asking for an ‘about this building’ plaque.
   - *Technical role:* 7-phase prompt: inventory, SVG canvas, register resource, add page, update pages.json, markdown summary.
3. `tests/prompts/PowerBI_Test.md` — **Amber** · **AI file**
   - *What it is:* The AI audit prompt that looks for unused or broken model parts. Like a full health check request.
   - *Technical role:* 8-phase ‘Measure Killer-style’ + BPA audit prompt with severity markers and a fixed layout.
4. `tests/prompts/Python_Data_Value_Test.md` — **Amber** · **AI file**
   - *What it is:* The AI prompt that plans (but does not write) a Python number checker. Like asking for a blueprint.
   - *Technical role:* Instruction-only blueprint for a SQL validator: Sales by Branch, –branch/–month/–year/–dry-run.
5. `tests/README.md` — **Green**
   - *What it is:* How to run the tests locally and on GitHub. Like the lab’s instruction sheet.
   - *Technical role:* Structure, local commands, Python value testing, credentials, CI integration.

14.23 tests/results

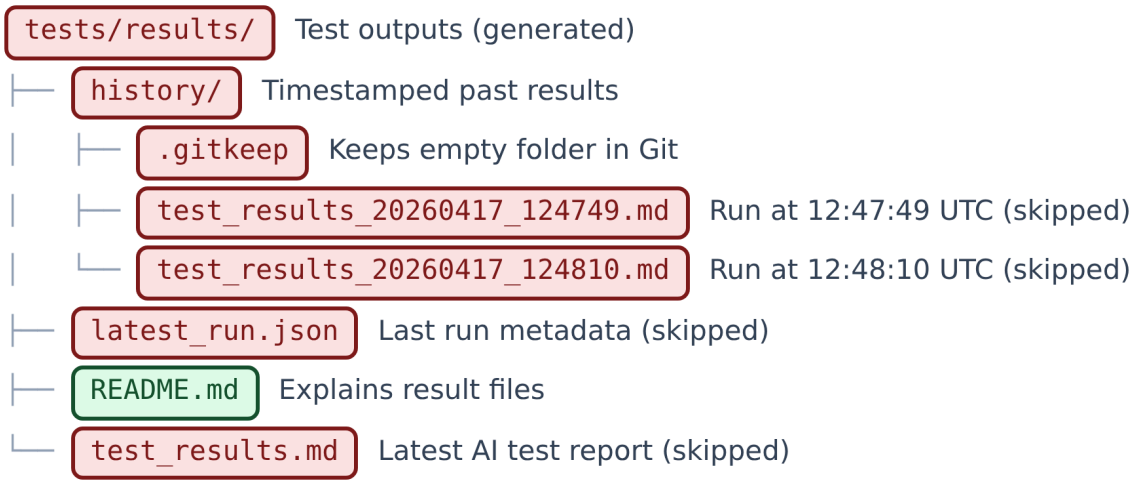


Figure 14.23 — tests/results. Generated test reports (Red: written by scripts).

Walkthrough of Figure 14.23:

1. `tests/results/` — **Red**
   - *What it is:* Where test reports are saved. Like the lab's results tray.
   - *Technical role:* Generated results folder.
2. `tests/results/history/` — **Red**
   - *What it is:* Every past test report, with a timestamp. Like an archive drawer.
   - *Technical role:* History folder.
3. `tests/results/history/.gitkeep` — **Red**
   - *What it is:* An empty file that keeps the empty folder in Git. Like a bookmark holding a place.
   - *Technical role:* Placeholder: Git does not store empty folders.
4. `tests/results/history/test_results_20260417_124749.md` — **Red**
   - *What it is:* A past test report from 17 April 2026, 12:47:49 UTC (skipped).
   - *Technical role:* Generated history copy.
5. `tests/results/history/test_results_20260417_124810.md` — **Red**
   - *What it is:* A past test report from 17 April 2026, 12:48:10 UTC (skipped).
   - *Technical role:* Generated history copy; same content as test_results.md.
6. `tests/results/latest_run.json` — **Red**
   - *What it is:* Details of the latest test run. Like the lab report's cover sheet.
   - *Technical role:* Run metadata: generatedUtc, promptFile, outputFile, reason, mode skipped, historyFile.
7. `tests/results/README.md` — **Green**
   - *What it is:* Explains the result files. Like a key to the results tray.
   - *Technical role:* Describes test_results.md, latest_run.json and history/.

8. `tests/results/test_results.md` — **Red**
    - *What it is:* The latest AI test report; today it says the test was skipped for lack of a key. Like the most recent lab report.
    - *Technical role:* Generated by run-ai-md-test.ps1 (skipped mode, 2026-04-17 12:48:10 UTC).

Coverage check: 176 of 176 items listed above.

15. “Which file do I touch?” decision trees

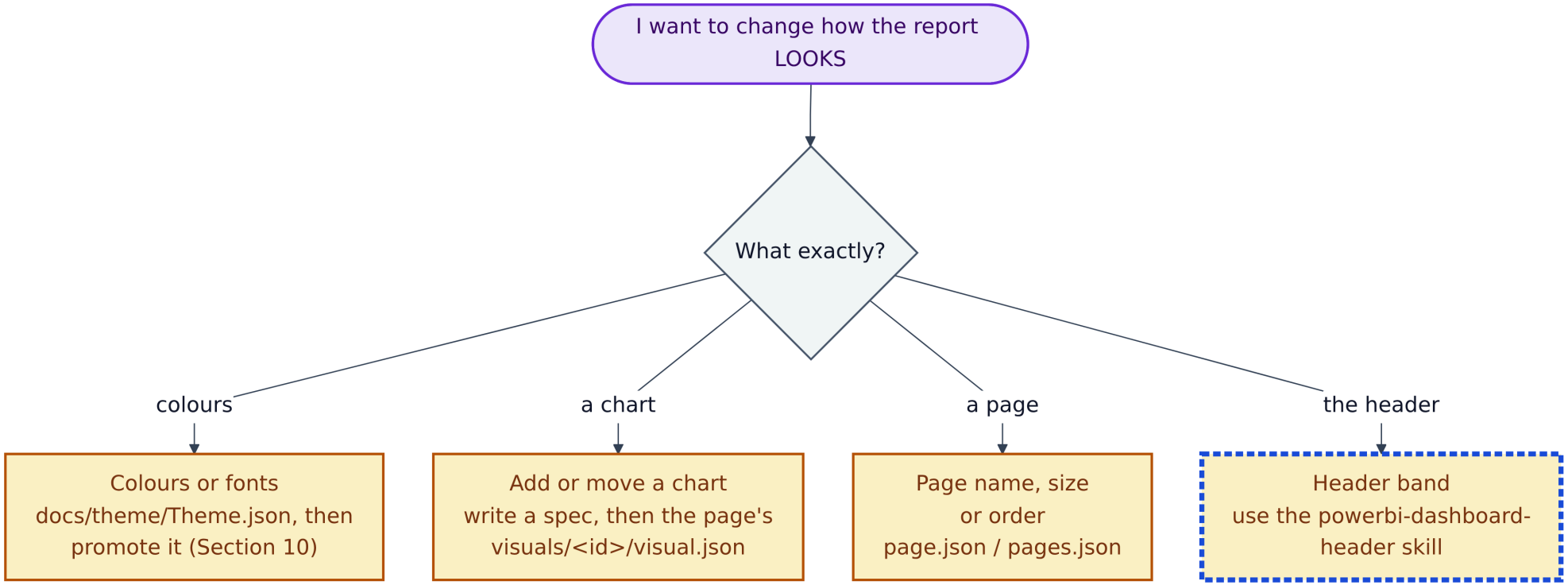


Figure 15.1 — “I want to change how the report looks.”

Walkthrough of Figure 15.1:

1. **Colours or fonts:** edit `docs/theme/Theme.json`, then promote it to the deployable copy and re-register it (Section 10). Never edit only the copy in `RegisteredResources/`; the next promotion would overwrite it.
2. **Add or move a chart:** write or update a spec first (Section 08), then change the page’s `visuals/<id>/visual.json`. Check the `card_y + 30` rule.
3. **Page name, size or order:** a page’s `page.json` (`displayName`, `width`, `height`) or `pages/pages.json` (`pageOrder`, `activePageName`).
4. **The header band:** follow the `powerbi-dashboard-header` skill, and record any exception (such as visual titles) in the spec.

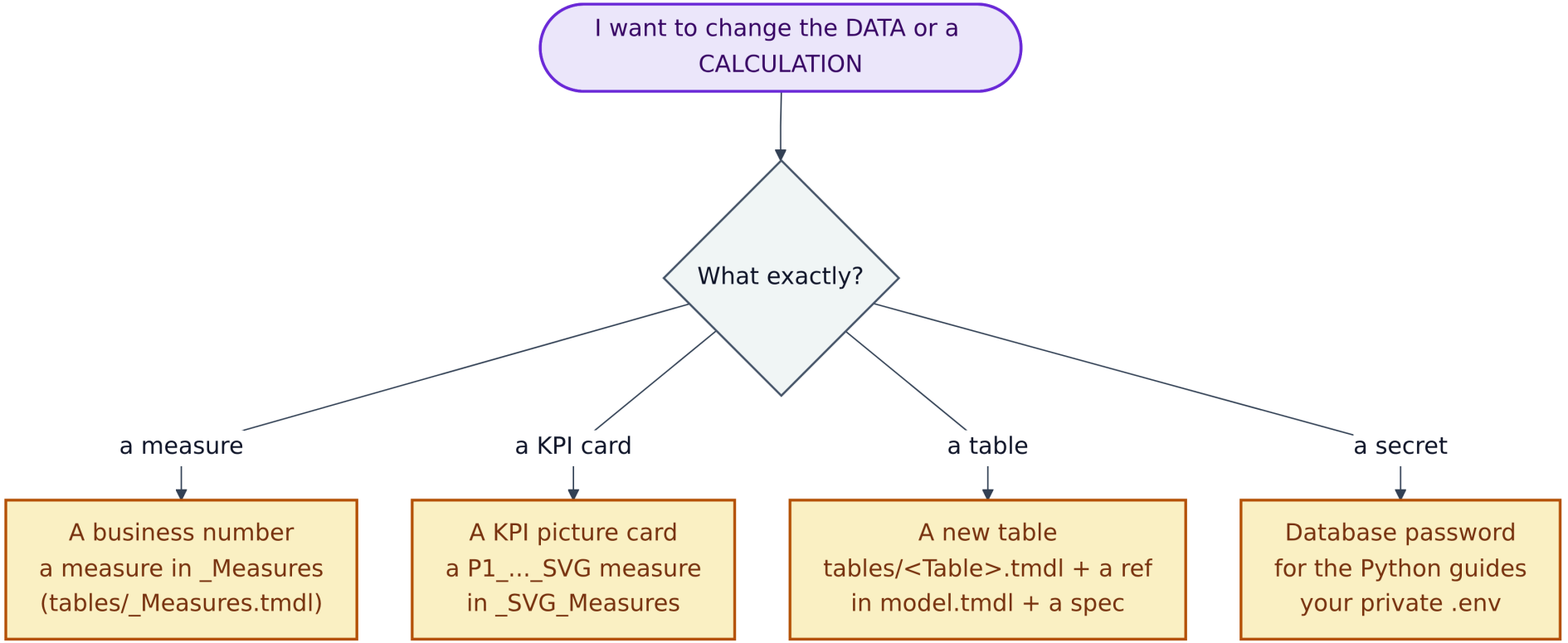


Figure 15.2 — "I want to change the data or a calculation."

Walkthrough of Figure 15.2:

1. **A business number** (e.g. "Total Sales") is a measure in the `_Measures` table, with a `///` description.
2. **A KPI picture card** is an SVG measure in `_SVG_Measures`, named `P<page>_<Name>_SVG`, shown with an image visual.
3. **A new table** needs its own `Template.SemanticModel/definition/tables/<Table>.tmdl` file, a `ref table` line in `model.tmdl`, and a spec (a new fact table always needs one).
4. **A database password for the Python guides** belongs only in your private `.env` (Section 07). Never in any file Git tracks.

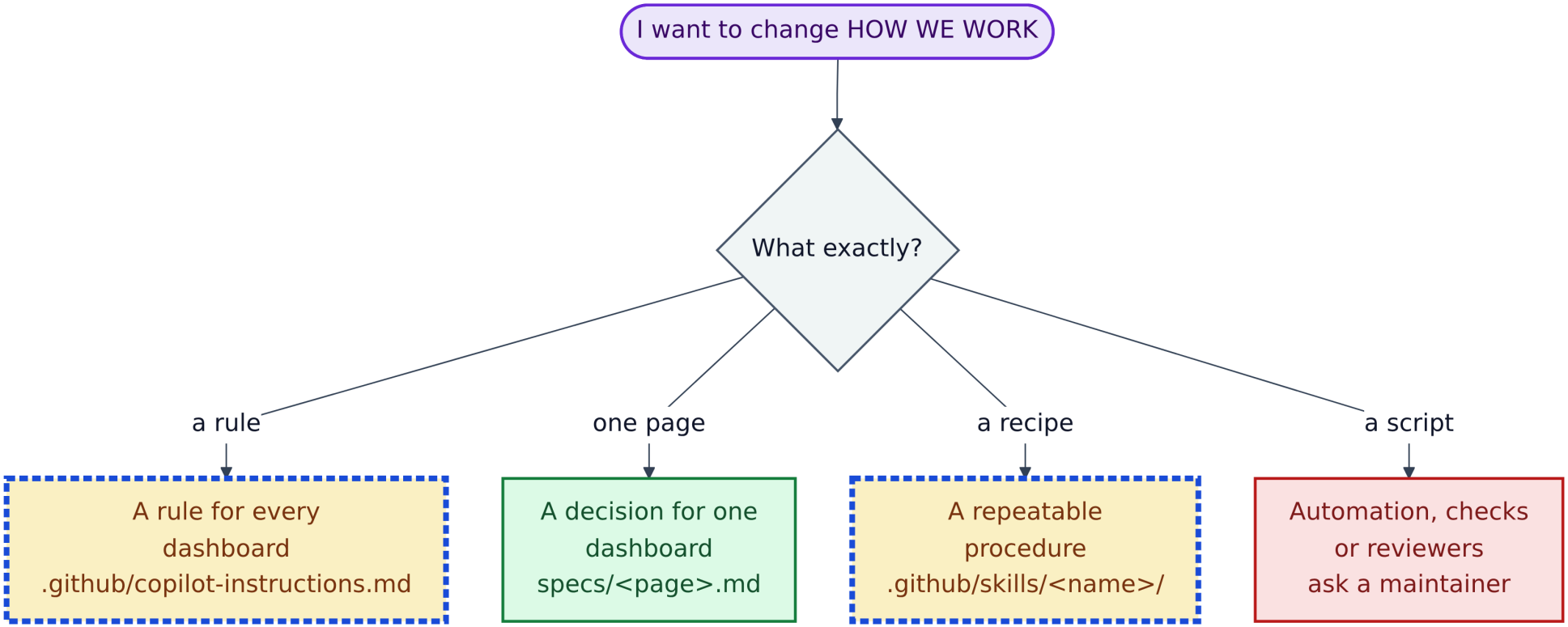


Figure 15.3 — "I want to change how we work." This mirrors Figure 6.3.

Walkthrough of Figure 15.3:

1. **A rule for every dashboard:** `.github/copilot-instructions.md` (then keep the root `copilot-instructions.md` and `AGENTS.md` aligned). Amber + AI: get it reviewed.
2. **A decision for one dashboard:** the spec, `specs/<page>.md`. Green.
3. **A repeatable procedure:** a skill, `.github/skills/<name>/SKILL.md`, whose folder name matches its `name:` and whose `description:` says when to use it (`CONTRIBUTING.md`).
4. **Automation, checks or reviewers** (`scripts/`, `.github/workflows/`, `.github/CODEOWNERS`): ask a maintainer. Red.

Quick look-up table

I want to…	Touch this	Safety	Then run
Plan a new page	copy `specs/TEMPLATE.md` to `specs/<name>.md`	Green	nothing yet
Add a measure	`Template.SemanticModel/definition/tables/_Measures.tmdl`	Amber	`Test-PbipSemantics.ps1`
Add an SVG KPI card	`tables/_SVG_Measures.tmdl` + an image `visual.json`	Amber	all three validators + verify loop
Add a table	`tables/<Table>.tmdl` + `model.tmdl`	Amber	all three validators
Add a page	new `pages/<id>/page.json` + `pages.json`	Amber	`validate-pbip.ps1`, verify loop
Add a background SVG or theme	file in `StaticResources/RegisteredResources/` + entry in `report.json`	Amber	`validate-pbip.ps1` (checks it exists)
Change team colours	`docs/theme/Theme.json` → promote	Amber	reopen Desktop, verify loop
Fix a typo in a guide	the `.md` file under `docs/` or `specs/`	Green	nothing
Change what the AI always does	`.github/copilot-instructions.md`	Amber + AI	get a review
Add a reusable AI procedure	`.github/skills/<name>/SKILL.md`	Amber + AI	get a review
Store a secret for Python	your private `.env`	Amber	`git check-ignore -v .env`
Store a secret for GitHub Actions	repository *Settings > Secrets and variables > Actions*	n/a	re-run the workflow
Change the test workflow	`.github/workflows/ai-testing.yml`	Red	ask a maintainer
Edit `version.json`, `.platform`, `definition.pbir`, `.pbism`, `database.tmdl`	**don't**	Red	n/a
Edit `tests/results/`, `tests/screenshots/`, `.github/.pbip-state/`, `**/.pbi/`	**don't**: they are generated	Red	n/a

16. Gaps found in the repository, and how to fix them

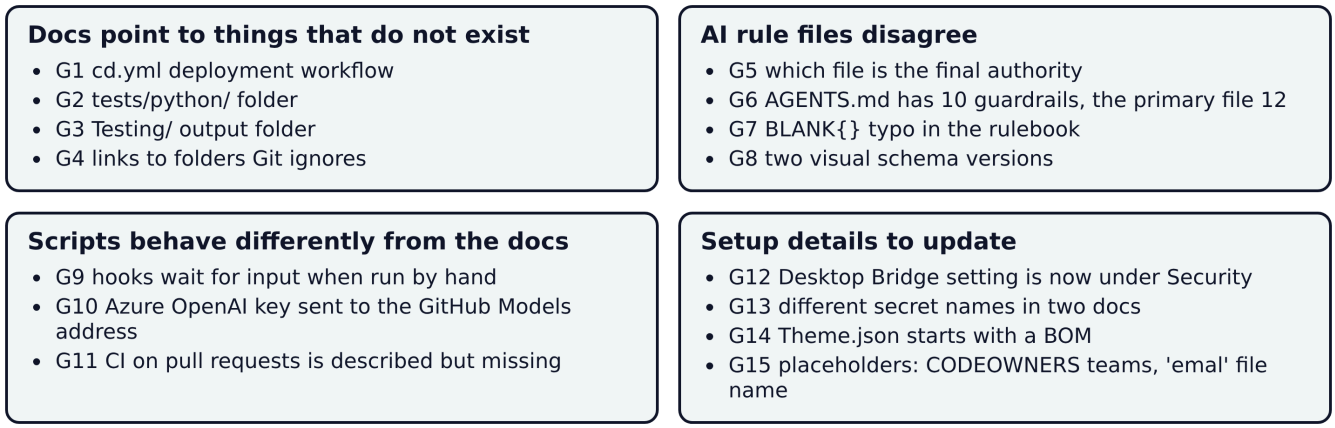


Figure 16.1 — Fifteen gaps, in four groups. While reading every file for this guide, we found places where the documents disagree with each other, point to things that do not exist, or describe behaviour the scripts do not have. None of them stops the template working, but each can confuse a beginner.

Walkthrough of Figure 16.1:

1. **Docs point to things that do not exist** (G1–G4): a missing workflow, folder or output path.
2. **AI rule files disagree** (G5–G8): the files the agent reads give different answers to the same question.
3. **Scripts behave differently from the docs** (G9–G11): something the docs promise that the code does not do.
4. **Setup details to update** (G12–G15): small facts that have drifted or placeholders still to fill.

This guide changes no repository files. The fixes below are written as steps for you, or a maintainer, to carry out in a separate change.

How to apply any fix safely

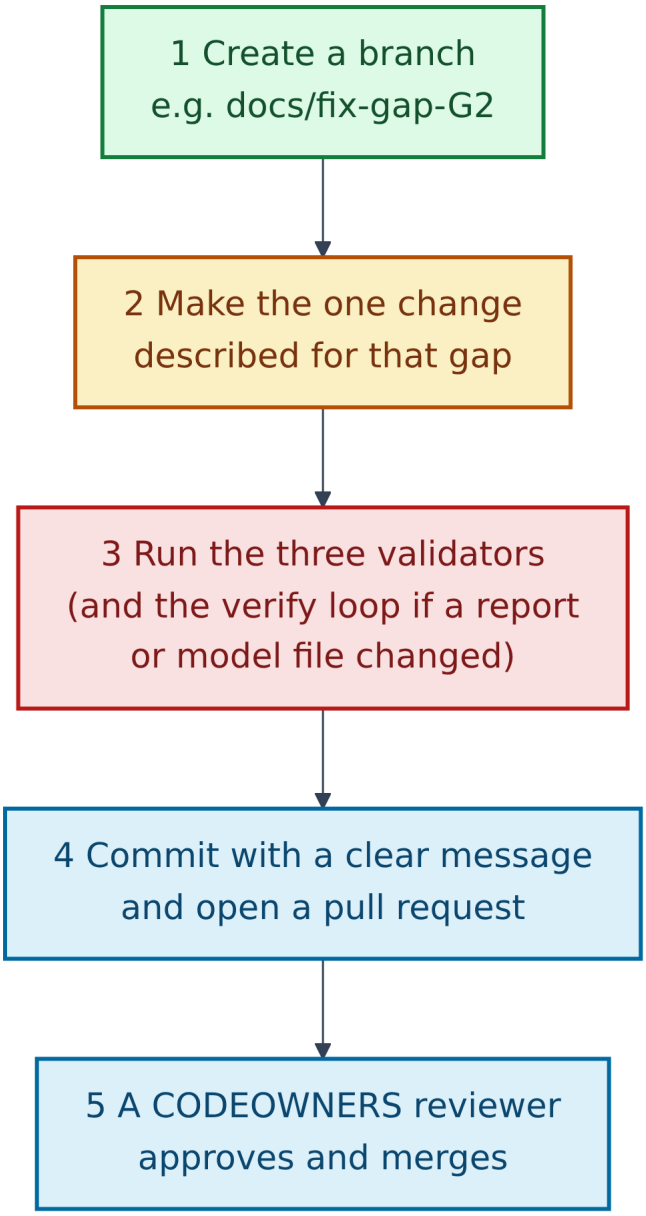


Figure 16.2 — The same five steps for every fix.

Walkthrough of Figure 16.2:

1. **Create a branch** named after the gap, e.g. `docs/fix-gap-G2`.
2. **Make only the change** described for that gap.
3. **Run the three validators** (Section 09). If you changed a report or model file, also run the verify loop.
4. **Commit and open a pull request**, naming the gap in the description.
5. **A reviewer listed in `CODEOWNERS` approves and merges.** Files marked Amber + AI change the agent's behaviour for everyone, so they always need a review.

Group 1: docs point to things that do not exist

G1. `docs/CICD-SETUP.md` describes a `cd.yml` deployment workflow that is not in the repository

What we found: the guide says deployments run "using `workflow_dispatch` in `.github/workflows/cd.yml`", but `.github/workflows/` only contains `ai-testing.yml`.

Plain fix, step by step:

1. Decide with your team whether you need automated deployment to Power BI workspaces yet.
2. **If not:** edit `docs/CICD-SETUP.md` and mark sections 2–6 as "planned, not yet included", so nobody looks for `cd.yml`.
3. **If yes:** ask a platform engineer to write `.github/workflows/cd.yml` with a `workflow_dispatch` trigger and the inputs the guide names (`target_environment`, `git_ref`), using the secrets and variables listed in sections 3–4 of that guide. Test it against the `dev` workspace first.

G2. Three files point to a `tests/python/` folder that does not exist

What we found: `docs/CICD-SETUP.md` and `tests/README.md` refer to `tests/python/README.md` and `tests/python/<your_script_name>.py`, and `tests/prompts/Python_Data_Value_Test.md` suggests "file location options in `tests/python/`". The guidance actually lives in `python/README.md`, whose example command uses `python/<your_script_name>.py`.

Plain fix, step by step:

1. Pick one home for Python validators. `python/` already exists, so it is the smaller change.
2. In `docs/CICD-SETUP.md` (section 9) and `tests/README.md`, replace `tests/python/` with `python/`.
3. In `tests/prompts/Python_Data_Value_Test.md`, change the suggested location the same way.
4. Search the repository for `tests/python` (VS Code: `Ctrl+Shift+F`) and confirm nothing is left.

G3. The AI audit prompt writes to `Testing/test_results.md`, not `tests/results/`

What we found: `tests/prompts/PowerBI_Test.md` tells Copilot to "Write final results to `Testing/test_results.md`", while the runner script, the workflow and `tests/results/README.md` all use `tests/results/test_results.md`.

Plain fix, step by step:

1. Open `tests/prompts/PowerBI_Test.md`.
2. Replace both mentions of `Testing/test_results.md` (hard requirement 4 and the start of Phase 5) with `tests/results/test_results.md`.
3. Save, then run the prompt once in Copilot Chat and confirm the report appears in `tests/results/`.

G4. `docs/REPOSITORY-GUIDE.md` links to folders that Git never stores

What we found: the "Files users should not edit directly" list links to `tests/screenshots/`, `.github/.pbip-state/` and `Proposals/`. All three are in `.gitignore`, so on GitHub these links lead to "page not found".

Plain fix, step by step:

1. Open `docs/REPOSITORY-GUIDE.md`.
2. In that list, keep the folder names in code formatting but remove the links, e.g. change `[`Proposals/`](../Proposals/)` to `` `Proposals/` ``.
3. Optionally add "(created locally, never committed)" after each one.

Group 2: AI rule files disagree

G5. Which file is "the final authority"?

What we found: `README.md`, `AGENTS.md` and `.github/copilot-instructions.md` say `.github/copilot-instructions.md` wins any conflict. But the three files in `.github/agents/`, and the `powerbi-dashboard-header` and `powerbi-pbiviz-custom-visuals` skills, say the root `copilot-instructions.md` is "the final authority".

Plain fix, step by step:

1. Open each of the five files: the three `*.agent.md` files and the two `SKILL.md` files.
2. Change the authority sentence to the same wording the README uses, for example: "`.github/copilot-instructions.md` is the primary policy authority; `copilot-instructions.md` is the detailed PBIP reference."
3. Keep the rest of each "Repository Override Rules" section as it is.
4. Ask a reviewer to read the five changed sentences side by side.

G6. `AGENTS.md` lists 10 guardrails; `.github/copilot-instructions.md` lists 12

What we found: the primary file adds two rules: keep SVG-header mode as the default, and treat `docs/theme/Theme.json` as source only. `AGENTS.md` and the README stop at 10. An agent that reads only `AGENTS.md` (for example Claude Code) never sees rules 11 and 12.

Plain fix, step by step:

1. Copy rules 11 and 12 from `.github/copilot-instructions.md`.
2. Paste them as rules 11 and 12 in `AGENTS.md` and in the README's "Non-Negotiable PBIP Guardrails" list.
3. Check that the wording of all twelve matches across the three files.

G7. A typo in the rulebook: `_Measures = BLANK{}`

What we found: step 2 of "Dashboard Creation Steps" in the root `copilot-instructions.md` says "create `_Measures = BLANK{}` if needed". The same file later writes the correct DAX form for the SVG table: `_SVG_Measures = { BLANK() }`. An agent copying `BLANK{}` literally would write an invalid table expression.

Plain fix, step by step:

1. Open `copilot-instructions.md` and find `BLANK{}`.
2. Replace it with `{ BLANK() }`, so the line reads "create `_Measures = { BLANK() }` if needed".

G8. Two visual schema versions

What we found: the root rulebook sets the repository default visual schema to `visualContainer/2.7.0`. The custom-visual kit's `pbir_helpers.py` writes `visualContainer/2.12.0`, and the skill says it was verified with that version. A page could end up mixing the two.

Plain fix, step by step:

1. Do not change any file yet. First, open a test copy of a report in your current Power BI Desktop and check which schema version it writes for a new visual.
2. Agree one default with the team.
3. Update the "Schema (repo default)" line and the five JSON templates in `copilot-instructions.md`, or the `SCHEMA_VIZ` line in `pbir_helpers.py`, so both say the same.
4. Add one sentence explaining why, so the next reader knows it was deliberate.

Group 3: scripts behave differently from the docs

G9. Three hook scripts wait for input when you run them by hand

What we found: the README says the hook wrappers are "run manually". But `pbip-write-guard.ps1`, `pbip-validate-hook.ps1` and `pbip-stop-hook.ps1` begin by reading hook data from standard input (`[Console]::In.ReadToEnd()`), the way an agent tool such as Claude Code passes it. Run directly in a terminal, they sit waiting for typed input.

Plain fix, step by step:

1. **To use them today:** give them an empty input. In PowerShell:

   ```
   '{}' | powershell -NoProfile -File scripts\hooks\pbip-stop-hook.ps1
   ```

2. For everyday manual checks, use the underlying scripts instead: `Test-PbipSemantics.ps1 -Path <file>` (instead of the validate hook) and `Invoke-PbipVerify.ps1 -Mode Review` (instead of the stop hook).

3. **To fix the docs:** add a short note under the README's hook table explaining the `'{}' |` pattern, and that the hooks are designed to be wired into an agent tool's hook settings.

G10. An Azure OpenAI key is sent to the GitHub Models address

What we found: `run-ai-md-test.ps1` uses `GITHUB_MODELS_API_KEY`, or else `AZURE_OPENAI_API_KEY`, and always sends it as a `Bearer` token to `-ApiEndpoint`, which defaults to `https://models.inference.ai.azure.com/chat/completions`. The workflow never passes a different endpoint. So a repository that sets only the Azure OpenAI secret will very likely fail to authenticate, even though the README says either key works.

Plain fix, step by step:

1. **Simplest:** set `GITHUB_MODELS_API_KEY` as the repository secret, and treat the Azure OpenAI option as "not yet supported" in the README and `tests/README.md`.

2. **To support Azure OpenAI properly:** ask a maintainer to add an endpoint input or variable to `ai-testing.yml`, pass it to the script's `-ApiEndpoint`, and send the key in the header your Azure resource expects. Test with a manual workflow run before relying on it.

G11. "CI validates pull requests" is described but not set up

What we found: `docs/CICD-SETUP.md` says "Pull requests into `dev`, `test`, and `main` run CI validation". The only workflow, `ai-testing.yml`, runs on pushes to `test` and on manual runs, not on pull requests. Branch-protection rules that "require status checks from CI" will have no check to require.

Plain fix, step by step:

1. Ask a maintainer to add a small workflow (for example `.github/workflows/validate.yml`) that runs on `pull_request` to `dev`, `test` and `main`.
2. It needs only three steps: check out the code, then run `validate-json.ps1` and `validate-pbip.ps1` with `pwsh`, as `ai-testing.yml` already does.
3. After its first run, select it as a required status check in each branch's protection rule.

Group 4: setup details to update

G12. The Desktop Bridge setting has moved

What we found: the README, `copilot-instructions.md`, `Test-PbipBridge.ps1` and `PbipBridge.Common.ps1` say the checkbox is under *Options > Preview features*. Microsoft's current page, "What is the Power BI Desktop Bridge?", places it under *Options > Security > Desktop Bridge* and says it is on by default. Microsoft's page states no minimum version; the README's 2.155.756.0 is the repository's own finding.

Plain fix, step by step:

1. In the README and `copilot-instructions.md`, change the path to: "*File > Options and settings > Options > Security > Desktop Bridge* (older builds: *Preview features*)".
2. Ask a maintainer to make the same wording change in the "Fix" messages of `Test-PbipBridge.ps1` and `PbipBridge.Common.ps1`.
3. Add a link to Microsoft's page so the next reader can check it.

G13. Two different sets of database secret names

What we found: `.env.example` and the alert guide use `DB_SERVER`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_DRIVER`. Section 9 of `docs/CICD-SETUP.md` lists `SERVER`, `DATABASE`, `USERNAME`, `PASSWORD`, `DRIVER`.

Plain fix, step by step:

1. Keep the `DB_` names, because `.env.example` and the Python guide already use them.
2. In `docs/CICD-SETUP.md` section 9, replace the five names with the `DB_` versions.
3. If any pipeline already uses the old names, rename those secrets at the same time.

G14. `docs/theme/Theme.json` starts with a byte-order mark

What we found: the file begins with an invisible UTF-8 byte-order mark (BOM). Python's standard `json.load` refuses such a file ("Unexpected UTF-8 BOM"), which can trip up a promotion script. The deployable copies do not have one.

Plain fix, step by step:

1. Open `docs/theme/Theme.json` in VS Code.
2. Click the encoding label in the bottom status bar (it shows "UTF-8 with BOM").
3. Choose *Save with Encoding → UTF-8*.
4. Run `validate-json.ps1` to confirm it still parses.

(The files in `tests/results/` also have a BOM, written by PowerShell's `Set-Content`. They are generated, so leave them.)

G15. Placeholders still to fill in

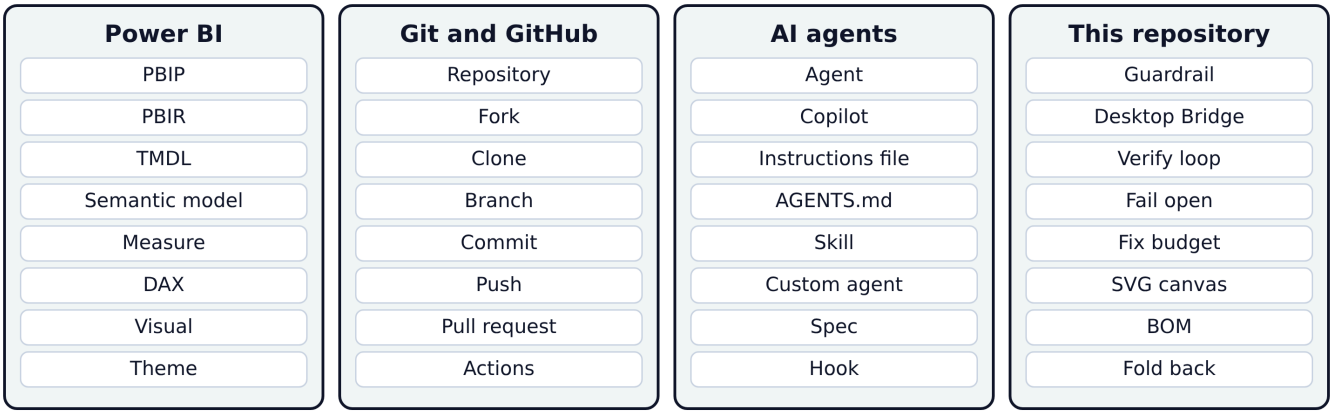
What we found:

- `.github/CODEOWNERS` uses example team handles (`@your-org/bi-platform-team` and others), as its first line says ("Update owners to your org/team handles").
- `docs/REPOSITORY-GUIDE.md` uses `<upstream-owner>/<upstream-repo>` in the upstream command.
- The email guide's file name is spelled `emal_style_instructions.md` (missing "i"), and the README refers to that spelling.

Plain fix, step by step:

1. In your fork, replace each `@your-org/...` handle in `CODEOWNERS` with a real GitHub team or user who should review that folder.
2. Replace `<upstream-owner>/<upstream-repo>` with `GibbonGambits/PGE-PBIP-Template` in your own notes when you run the command.
3. Renaming the email guide is optional. If you rename it to `email_style_instructions.md`, update the README reference in the same change.

17. Beginner glossary



32 core terms in four families. The full alphabetical glossary follows.

Figure 17.1 — Four families of terms. If a word in this guide is new, it is probably in one of these four columns.

Walkthrough of Figure 17.1:

1. **Power BI** words describe the report and its data.
2. **Git and GitHub** words describe saving and sharing changes.
3. **AI agent** words describe how the AI assistant is guided.
4. **This repository** words are the template's own vocabulary.

A to Z

Term	Plain meaning	Comparison	First explained
Actions (GitHub Actions)	GitHub's automation service; runs workflows on cloud computers	A timer that switches an appliance on	Section 11
Agent (AI coding agent)	An AI assistant that can read files, edit them and run commands, not only chat	A contractor working in your house	Section 04
AGENTS.md	A cross-tool instruction file for AI agents	A safety card on the wall	Section 04
API key	A secret code that lets a program use an online service	A door key card	Section 11
applyTo	The file pattern that decides when a `.instructions.md` file loads	"Workshop rules only" sign	Section 04
Artifact	A file a GitHub workflow saves for you to download	A receipt from a machine	Section 11
Base theme	Microsoft's default theme underneath your own	Primer under the paint	Section 10

Term	Plain meaning	Comparison	First explained
BOM (byte-order mark)	An invisible marker at the start of a text file; TMDL files must not have one	An invisible sticker on a page	Section 02
Bookmark	A saved view of a report page, used for toggles	A page marker	Section 14
BPA (Best Practice Analyzer)	Microsoft's rule set for checking data models	An inspection checklist	Section 11
Branch	A separate line of work in Git	A draft copy of a document	Section 03
Card (KPI card)	A box showing one headline number	A scoreboard	Section 08
`card_y + 30` rule	A visual must start 30 pixels below its card's top so it does not cover the title	Leave room under a heading	Section 08
Checklist row	One numbered quality check in `docs/checklists/`	One line on an inspection form	Section 05
CLI (command-line interface)	A program you control by typing commands	A remote control with words	Section 07
Clone	Download a full copy of a repository, with its history	Photocopying a whole binder	Section 03
CODEOWNERS	A GitHub file naming who reviews which folders	A sign-off sheet	Section 03
Commit	A saved snapshot of changes, with a message	A dated save point	Section 03
Compatibility level	A version number for the model engine	A voltage rating	Section 02
Copilot (GitHub Copilot)	GitHub's AI assistant, available in VS Code and as a CLI	A built-in assistant	Section 04
Custom agent	A saved "expert role" for Copilot (`.agent.md`)	Calling in a specialist	Section 04
Custom visual	A chart type your team writes in code (`.pbiviz`)	Custom-built furniture	Section 12
DAX	Data Analysis Expressions, Power BI's formula language	A recipe language	Section 02
Desktop Bridge	A local connection that lets programs control a running Power BI Desktop	A remote control for Power BI	Section 01
Dimension table	A table describing things (products, dates, customers)	A catalogue	Section 02

Term	Plain meaning	Comparison	First explained
`.env` file	A private text file of `NAME=value` settings such as passwords	A locked drawer	Section 07
Fabric (Microsoft Fabric)	Microsoft’s online analytics platform	The head office	Section 02
Fact table	A table of events or transactions (sales, orders)	A till receipt roll	Section 02
Fail open	When a check breaks, it lets you continue rather than block you	A door that unlocks in a power cut	Section 06
Field parameter	A Power BI feature that lets readers switch which field a visual shows	A channel selector	Section 14
Fix budget	The verify loop allows one fix round, then lets the turn end	A limited number of retakes	Section 09
Fold back	Moving a rule from a spec into the permanent rulebook	Adding a lesson to the manual	Section 06
Fork	Your own copy of someone else’s repository on GitHub	Your own copy of a shared recipe book	Section 03
Git	Free software that records the history of files	Unlimited undo with labels	Section 03
GitHub	A website that hosts Git repositories and adds collaboration	A shared photo album for snapshots	Section 03
`.gitignore`	A list of files Git must never save	A “do not pack” list	Section 03
Guardrail	One of the repository’s non-negotiable rules	A road barrier	Section 01
GUID	A long unique identifier	A serial number	Section 12
Hook	A small script that runs at a fixed moment	A smoke alarm	Section 06
IBCS	A set of design standards for business charts	A style guide for finance charts	Section 12
Image visual	A Power BI visual that shows a picture; here, an SVG from a measure	A picture frame	Section 08
JSON	A text format of named values in curly braces	A labelled form	Section 02
KPI	Key performance indicator: a headline number	A car’s speedometer	Section 08

Term	Plain meaning	Comparison	First explained
Markdown (`.md`)	Plain text with simple marks for headings, lists and links	Notes with underlines	Section 06
Measure	A named calculation, such as “Total Sales”	A recipe	Section 02
Merge	Combining one branch’s changes into another	Folding a draft into the final copy	Section 03
Node.js / npm	A JavaScript runtime and its package installer	An engine and its parts shop	Section 07
Page (report page)	One screen of a report	A magazine page	Section 02
PBIP	Power BI Project: a report saved as a folder of text files	A labelled filing cabinet	Section 02
PBIR	Power BI’s enhanced report format: the report as JSON files	Labelled drawers	Section 02
`.pbix`	Power BI’s classic single-file format	A sealed box	Section 02
Power BI Desktop	Microsoft’s free Windows app for building reports	The workshop	Section 02
Power Query	Power BI’s tool for loading and shaping data	The delivery bay	Section 02
PowerShell	Microsoft’s command shell and scripting language	A command line with a toolbox	Section 07
Preflight	A check that everything is ready before you start	A pilot’s checklist	Section 07
Pull request (PR)	A request to review and merge a branch	Asking an editor to approve a draft	Section 03
Push / pull	Upload commits to GitHub / download others’ commits	Posting / collecting mail	Section 03
Relationship	A link between two tables	Matching labels	Section 02
Repository (repo)	A project folder whose history Git tracks	A binder that remembers every change	Section 00
Runner	A cloud computer that runs a GitHub workflow	A hired worker for one job	Section 11
Secret (GitHub secret)	A hidden setting a workflow can use but nobody can read	A sealed envelope	Section 11
Semantic model	The tables, relationships and measures behind a report	The kitchen and pantry	Section 02
Skill	A folder with a `SKILL.md` procedure the agent loads when the task matches	A recipe card	Section 04

Term	Plain meaning	Comparison	First explained
Slicer	An on-page filter control	A dial on a radio	Section 14
Spec	A plan for one dashboard or page, written before building	An architect’s drawing	Section 06
Star schema	A model with fact tables in the middle and dimension tables around them	A hub with spokes	Section 02
Steering	The permanent rule files that guide every session	The building code	Section 06
SVG	Scalable Vector Graphics: a text-based picture format	A drawing made of instructions	Section 08
SVG canvas background	The page background picture that draws cards and titles	A printed form	Section 08
Swimlane diagram	A diagram with one column per person or tool	Lanes in a pool	Section 03
Theme	A JSON file of colours and fonts for a report	Tableware and colour scheme	Section 10
TMDL	Tabular Model Definition Language: the model as indented text	An indented recipe book	Section 02
UTF-8	The standard way to store text characters in a file	The alphabet a file is written in	Section 02
Verify loop	Validate, reload Power BI, screenshot every page, review, fix	A camera that photographs every room	Section 09
Visual	One chart, table, card or image on a page	A photo on a page	Section 02
VS Code	Visual Studio Code, Microsoft’s free code editor	A word processor for code	Section 04
Workflow	An automated process defined in a YAML file	A recipe the robot follows	Section 11
YAML	An indentation-based text format used for workflows	A neat outline	Section 11

18. Verified official references

Group	Official site	Links
Power BI and Microsoft	`learn.microsoft.com`	17
Git and GitHub	`git-scm.com, docs.github.com`	12
VS Code and Copilot in VS Code	`code.visualstudio.com`	6
Claude and Anthropic	`code.claude.com, platform.claude.com`	4
Other tools	`nodejs.org, python.org, github.com/microsoft`	3

Every link was opened on 10 November 2026 and its content checked. Links that failed to open were left out.

Figure 18.1 — Where the references come from. Official documentation only, grouped by technology.

Walkthrough of Figure 18.1:

1. Each link below was opened on 10 November 2026, and the fact this guide uses was checked against the page.
2. Links that would not open were left out (for example, the npm package pages for the two Power BI CLIs refused automated access, so they are not listed).
3. Each entry says what the page is for, in one line.

Power BI and Microsoft (Microsoft Learn)

Page	Use it for
Power BI Desktop projects (PBIP)	What a `.pbip` project is and what saving one creates
Power BI Desktop project report folder	The PBIR files: `definition.pbir`, `report.json`, `pages.json`, `page.json`, `visual.json`
Power BI Desktop project semantic model folder	`definition.pbism`, the TMDL folder, and which `.pbi` files Git ignores
Git integration with Power BI Desktop projects	Putting a PBIP folder under Git in VS Code
Tabular Model Definition Language (TMDL)	How TMDL files are structured
Use report themes in Power BI	Theme JSON files and `dataColors`
What is the Power BI Desktop Bridge?	The bridge, its Security setting, and the `powerbi-desktop` CLI commands
Download Power BI Desktop	Installing Power BI Desktop and its system requirements
Set up an environment for developing a Power BI visual	Node.js and the `pbiviz` tools for custom visuals
Capabilities and properties of Power BI visuals	What `capabilities.json` declares
Use field parameters in Power BI reports	The feature behind the field-parameter skill
Row-level security (RLS) with Power BI	Restricting which rows each person sees

Page	Use it for
Understand star schema and the importance for Power BI	Fact and dimension tables
Use Performance Analyzer to examine report performance	Timing visuals for the performance skill
DAX overview	The DAX formula language
Overview of Fabric Git integration	Syncing Fabric workspaces with Git
What is PowerShell?	The shell the repository's scripts use

The six files in `docs/checklists/` also contain a Microsoft Learn link for every checklist row.

Git and GitHub

Page	Use it for
Git book: About Version Control	What version control is
What is GitHub?	How Git and GitHub differ
Fork a repository	Forking, and adding the `upstream` remote
About pull requests	Proposing and reviewing changes
Ignoring files	How `.gitignore` works
About code owners	How `CODEOWNERS` requests reviewers
Understanding GitHub Actions	Workflows, jobs and runners
Events that trigger workflows	`push` and `workflow_dispatch`
Using secrets in GitHub Actions	Adding the API-key secrets
Adding repository custom instructions for GitHub Copilot	`.github/copilot-instructions.md`, `.instructions.md` files and `AGENTS.md`
About agent skills	Where Copilot looks for skills
Installing GitHub Copilot CLI	Installing Copilot in the terminal

VS Code and Copilot in VS Code

Page	Use it for
Download Visual Studio Code	Installing VS Code
Source control in VS Code	Committing and pushing from the editor
AI in VS Code overview	What an agent does in VS Code
Use custom instructions in VS Code	Instruction files and `applyTo` patterns
Custom agents in VS Code	`.agent.md` expert modes
Use Agent Skills in VS Code	`SKILL.md` format: `name` and `description`

Claude and Anthropic

Page	Use it for
Claude Code overview	What Claude Code is

Page	Use it for
Claude Code hooks reference	Hook events and exit code 2, which this repository’s hook scripts follow
How Claude remembers your project	`CLAUDE.md`, and when Claude reads `AGENTS.md`
Agent Skills overview	How skills work for Claude

Other tools

Page	Use it for
Download Node.js	Installing Node.js and npm
Download Python	Installing Python 3
microsoft/skills-for-fabric	The Copilot CLI plugin collection named in the README

For Git itself, use the download link on the Git website’s home page (`git-scm.com`). The direct downloads address only showed a redirect notice when checked, so it is not linked here.

19. Appendix: the Stage 1 inventory table

This is the complete list of every file and folder in the repository, as found in Stage 1 of building this guide. It is the checklist used to make sure Section 14 misses nothing.

- **Safety** is the fill colour from the legend in Section 00: Green = safe to edit, Amber = edit with care, Red = do not edit (you may still read or run it).
- **AI file** = Yes means an AI agent (GitHub Copilot or Claude) reads the file as instructions. In diagrams these items have a blue dashed outline.
- Paths are relative to the repository root. Folders end with `/`.

#	Path	Type	Purpose	Safety	AI file
1	`.env.example`	Env template	Blank list of database/email secret names	Amber	
2	`.github/`	Folder	GitHub + Copilot configuration	Amber	Yes
3	`.github/agents/`	Folder	Copilot custom agent modes	Amber	Yes
4	`.github/agents/power-bi-data-modeling-expert.agent.md`	Agent mode	Star-schema modelling expert persona	Amber	Yes
5	`.github/agents/power-bi-dax-expert.agent.md`	Agent mode	DAX formula expert persona	Amber	Yes
6	`.github/agents/power-bi-visualization-expert.agent.md`	Agent mode	Report design expert persona	Amber	Yes
7	`.github/CODEOWNERS`	GitHub config	Who must review which folders	Amber	
8	`.github/copilot-instructions.md`	AI steering	Primary Copilot policy, auto-loaded	Amber	Yes
9	`.github/instructions/`	Folder	Rules that load by file type	Amber	Yes
10	`.github/instructions/power-bi-custom-visuals-development.instructions.md`	Scoped rules	Custom visual coding (TS/JS/LESS files)	Amber	Yes
11	`.github/instructions/power-bi-devops-alm-best-practices.instructions.md`	Scoped rules	CI/CD and deployment (YAML/PS1/JSON)	Amber	Yes
12	`.github/instructions/power-bi-security-rls-best-practices.instructions.md`	Scoped rules	Row-level security patterns	Amber	Yes
13	`.github/ISSUE_TEMPLATE/`	Folder	Issue forms	Green	
14	`.github/ISSUE_TEMPLATE/bug_report.md`	GitHub template	Bug report form	Green	
15	`.github/ISSUE_TEMPLATE/feature_request.md`	GitHub template	Feature request form	Green	
16	`.github/PULL_REQUEST_TEMPLATE.md`	GitHub template	Pre-filled pull request form	Green	
17	`.github/skills/`	Folder	Copilot skills (step-by-step recipes)	Amber	Yes
18	`.github/skills/power-bi-model-design-review/`	Folder	Skill: model audit	Amber	Yes
19	`.github/skills/power-bi-model-design-review/SKILL.md`	Skill	Audit a model against checklists	Amber	Yes

#	Path	Type	Purpose	Safety	AI file
20	`.github/skills/power-bi-performance-troubleshooting/`	Folder	Skill: speed triage	Amber	Yes
21	`.github/skills/power-bi-performance-troubleshooting/SKILL.md`	Skill	Diagnose slow visuals with thresholds	Amber	Yes
22	`.github/skills/powerbi-dashboard-header/`	Folder	Skill: page header	Amber	Yes
23	`.github/skills/powerbi-dashboard-header/SKILL.md`	Skill	Standard header band and page chrome	Amber	Yes
24	`.github/skills/powerbi-fieldparameter/`	Folder	Skill: field parameters	Amber	Yes
25	`.github/skills/powerbi-fieldparameter/fieldparam_reportbuilder.md`	Skill boilerplate	Fill-in-the-blanks file templates	Amber	Yes
26	`.github/skills/powerbi-fieldparameter/SKILL.md`	Skill	Build table/bar toggle page, 11 steps	Amber	Yes
27	`.github/skills/powerbi-modeling/`	Folder	Skill: modelling	Amber	Yes
28	`.github/skills/powerbi-modeling/SKILL.md`	Skill	Build/review measures and relationships	Amber	Yes
29	`.github/skills/powerbi-pbiviz-custom-visuals/`	Folder	Skill: custom visuals kit	Amber	Yes
30	`.github/skills/powerbi-pbiviz-custom-visuals/assets/`	Folder	Starter-kit code	Amber	Yes
31	`.github/skills/powerbi-pbiviz-custom-visuals/assets/project/`	Folder	Base project files	Amber	Yes
32	`.github/skills/powerbi-pbiviz-custom-visuals/assets/project/assets/`	Folder	Icon folder	Amber	Yes
33	`.github/skills/powerbi-pbiviz-custom-visuals/assets/project/assets/icon.png`	Image	20x20 visual icon	Green	Yes
34	`.github/skills/powerbi-pbiviz-custom-visuals/assets/project/eslint.config.mjs`	Lint config	Code-style checker settings	Red	Yes
35	`.github/skills/powerbi-pbiviz-custom-visuals/assets/project/package.json`	npm manifest	Pinned visual dependencies	Red	Yes
36	`.github/skills/powerbi-pbiviz-custom-visuals/assets/project/pbiviz.json`	Visual manifest	Name/GUID/author placeholders	Red	Yes
37	`.github/skills/powerbi-pbiviz-custom-visuals/assets/project/tsconfig.json`	TS config	TypeScript compiler settings	Red	Yes
38	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/`	Folder	Shared visual library	Amber	Yes
39	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/base.less`	Stylesheet	Shared fonts and colours	Red	Yes
40	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/dom.ts`	TypeScript	Safe HTML/SVG element builders	Red	Yes
41	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/format.ts`	TypeScript	Number/percent/month formatting	Red	Yes
42	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/interaction.ts`	TypeScript	Click, tooltip, render events	Red	Yes
43	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/paths.ts`	TypeScript	Bar/donut/pin shape drawing	Red	Yes
44	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/props.ts`	TypeScript	Format-pane settings reader/builder	Red	Yes
45	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/scale.ts`	TypeScript	Value-to-pixel scales, ticks	Red	Yes
46	`.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/text.ts`	TypeScript	Measure/truncate label text	Red	Yes

#	Path	Type	Purpose	Safety	AI file
47	.github/skills/powerbi-pbiviz-custom-visuals/assets/shared/tokens.ts	TypeScript	Design colour tokens	Red	Yes
48	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/	Folder	Three starter visuals	Amber	Yes
49	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/	Folder	Minimal skeleton visual	Amber	Yes
50	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/capabilities.json	Visual contract	Fields and format options	Red	Yes
51	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/src/	Folder	Source code	Amber	Yes
52	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/src/visual.ts	TypeScript	Simple bar visual	Red	Yes
53	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/style/	Folder	Styles	Amber	Yes
54	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/blank/style/visual.less	Stylesheet	Imports shared base	Red	Yes
55	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/	Folder	Period chart visual	Amber	Yes
56	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/capabilities.json	Visual contract	Fields and format options	Red	Yes
57	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/src/	Folder	Source code	Amber	Yes
58	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/src/visual.ts	TypeScript	Columns/line + variance strip	Red	Yes
59	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/style/	Folder	Styles	Amber	Yes
60	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceChart/style/visual.less	Stylesheet	Imports shared base	Red	Yes
61	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/	Folder	Statement table visual	Amber	Yes
62	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/capabilities.json	Visual contract	Fields and format options	Red	Yes
63	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/src/	Folder	Source code	Amber	Yes
64	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/src/visual.ts	TypeScript	P&L-style variance table	Red	Yes
65	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/style/	Folder	Styles	Amber	Yes
66	.github/skills/powerbi-pbiviz-custom-visuals/assets/templates/varianceTable/style/visual.less	Stylesheet	Table layout styles	Red	Yes
67	.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/	Folder	Kit scripts	Amber	Yes

#	Path	Type	Purpose	Safety	AI file
68	.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/new_visual.py	Python	Scaffold a new visual + GUID	Red	Yes
69	.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/pbir_helpers.py	Python	Write visual.json bindings safely	Red	Yes
70	.github/skills/powerbi-pbiviz-custom-visuals/assets/tools/sync-visuals.mjs	Node script	Build, embed, register visuals	Red	Yes
71	.github/skills/powerbi-pbiviz-custom-visuals/references/	Folder	Deep-dive docs	Amber	Yes
72	.github/skills/powerbi-pbiviz-custom-visuals/references/CAPABILITIES-AND-DATAVIEW.md	Doc	Designing fields and data shape	Amber	Yes
73	.github/skills/powerbi-pbiviz-custom-visuals/references/DESIGN-IBCS.md	Doc	Finance chart design conventions	Amber	Yes
74	.github/skills/powerbi-pbiviz-custom-visuals/references/FORMATTING-PANE.md	Doc	Adding format-pane options	Amber	Yes
75	.github/skills/powerbi-pbiviz-custom-visuals/references/INTERACTIVITY.md	Doc	Selection, tooltips, highlight	Amber	Yes
76	.github/skills/powerbi-pbiviz-custom-visuals/references/PBIP-EMBEDDING.md	Doc	Payload/registration/reference triad	Amber	Yes
77	.github/skills/powerbi-pbiviz-custom-visuals/references/REPLACING-LICENSED-VISUALS.md	Doc	Playbook to replace paid visuals	Amber	Yes
78	.github/skills/powerbi-pbiviz-custom-visuals/references/TEMPLATES.md	Doc	What each starter visual does	Amber	Yes
79	.github/skills/powerbi-pbiviz-custom-visuals/references/TROUBLESHOOTING.md	Doc	Symptom -> cause -> fix tables	Amber	Yes
80	.github/skills/powerbi-pbiviz-custom-visuals/SKILL.md	Skill	Build/embed own custom visuals (9 steps)	Amber	Yes
81	.github/skills/powerbi-theme-governance/	Folder	Skill: themes	Amber	Yes
82	.github/skills/powerbi-theme-governance/SKILL.md	Skill	Turn Theme.json into deployable copy	Amber	Yes
83	.github/skills/powerbi-visual-verify/	Folder	Skill: screenshot review	Amber	Yes
84	.github/skills/powerbi-visual-verify/SKILL.md	Skill	Review rendered screenshots	Amber	Yes
85	.github/skills/README.md	Doc	Index of the 8 skills	Green	Yes
86	.github/workflows/	Folder	GitHub Actions automation	Red	
87	.github/workflows/ai-testing.yml	CI workflow	AI test on push to test branch	Red	
88	.gitignore	Git config	Files Git must never track	Amber	
89	AGENTS.md	AI steering	Short rulebook digest for AI agents	Amber	Yes
90	CODE_OF_CONDUCT.md	Community doc	Behaviour rules for contributors	Green	
91	CONTRIBUTING.md	Community doc	How to add skills, specs, tests	Green	
92	copilot-instructions.md	AI reference	Deep PBIP rulebook (727 lines)	Amber	Yes
93	docs/	Folder	Human guides and standards	Green	

#	Path	Type	Purpose	Safety	AI file
94	docs/AI-CONTEXT.md	Doc	Why steering/specs/hooks are separate	Green	
95	docs/checklists/	Folder	Six quality checklists	Green	
96	docs/checklists/accessibility-and-performance.md	Checklist	Rows 21-22, 67-70	Green	
97	docs/checklists/dax-and-measures.md	Checklist	Rows 28-39	Green	
98	docs/checklists/power-query-and-data-sources.md	Checklist	Rows 2-10	Green	
99	docs/checklists/README.md	Doc	Index of checklist rows 2-80	Green	
100	docs/checklists/report-design-and-interactions.md	Checklist	Rows 50-66, 71-75	Green	
101	docs/checklists/security-governance-testing-documentation.md	Checklist	Rows 40-49, 76-80	Green	
102	docs/checklists/semantic-model-and-relationships.md	Checklist	Rows 11-20, 23-27	Green	
103	docs/CICD-SETUP.md	Doc	Dev/Test/Prod branch + secrets setup	Green	
104	docs/REPOSITORY-GUIDE.md	Doc	Repo map, what to edit, upstream sync	Green	
105	docs/STARTER-PACK-CHECKLIST.md	Doc	Roll-out checklist	Green	
106	docs/theme/	Folder	Theme source and output	Amber	
107	docs/theme/README.md	Doc	Theme promotion rules	Green	
108	docs/theme/Theme.deployable.json	Theme output	Cleaned, schema-safe copy	Amber	
109	docs/theme/Theme.json	Theme source	Editable PGEColors palette	Amber	
110	LICENSE	Legal	MIT open-source licence	Red	
111	python/	Folder	Optional Python how-to guides	Green	
112	python/db_alerts/	Folder	Alert email guide	Green	
113	python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md	Doc	DB-triggered alert email recipe	Green	
114	python/email_reporting/	Folder	Report email guide	Green	
115	python/email_reporting/emal_style_instructions.md	Doc	Email report + DAX-to-Python rules	Green	
116	python/README.md	Doc	Build-your-own value validator	Green	
117	README.md	Doc	Front page: goals, setup, guardrails	Green	
118	scripts/	Folder	PowerShell checks and bridge loop	Red	
119	scripts/Get-PbipLoadError.ps1	PowerShell	Reads Desktop error pop-ups	Red	
120	scripts/hooks/	Folder	Hook wrapper scripts	Red	
121	scripts/hooks/pbip-session-brief.ps1	Hook	Prints rules + bridge status	Red	
122	scripts/hooks/pbip-stop-hook.ps1	Hook	Forces screenshot review	Red	

#	Path	Type	Purpose	Safety	AI file
123	scripts/hooks/pbip-validate-hook.ps1	Hook	Checks each file as written	Red	
124	scripts/hooks/pbip-write-guard.ps1	Hook	Blocks writes to protected paths	Red	
125	scripts/Invoke-PbipRefresh.ps1	PowerShell	Clicks Refresh in Desktop	Red	
126	scripts/Invoke-PbipVerify.ps1	PowerShell	Validate, reload, screenshot loop	Red	
127	scripts/PbipBridge.Common.ps1	PowerShell	Shared helper functions	Red	
128	scripts/run-ai-md-test.ps1	PowerShell	Sends PBIP to AI model for audit	Red	
129	scripts/Test-PbipBridge.ps1	PowerShell	Preflight: is everything installed?	Red	
130	scripts/Test-PbipSemantics.ps1	PowerShell	Finds load errors in source	Red	
131	scripts/validate-json.ps1	PowerShell	Checks every JSON parses	Red	
132	scripts/validate-pbip.ps1	PowerShell	Checks PBIP folder wiring	Red	
133	specs/	Folder	One plan per dashboard/page	Green	
134	specs/EXAMPLE-sales-overview.md	Example	Filled-in sample spec	Green	
135	specs/README.md	Doc	What a spec is, when needed	Green	
136	specs/TEMPLATE.md	Template	Blank 8-section spec to copy	Green	
137	Template.pbip	PBIP entry	Double-click file that opens the project	Red	
138	Template.Report/	Folder	The report (pages, visuals, themes)	Amber	
139	Template.Report/.platform	Fabric metadata	Report type, name, ID for Fabric Git	Red	
140	Template.Report/definition/	Folder	Report definition (PBIR)	Amber	
141	Template.Report/definition.pbir	Report pointer	Links report to semantic model	Red	
142	Template.Report/definition/pages/	Folder	One subfolder per page	Amber	
143	Template.Report/definition/pages/6e86f24a804ae4a905eb/	Folder	The single starter page	Amber	
144	Template.Report/definition/pages/6e86f24a804ae4a905eb/page.json	Page config	‘Page 1’, 1280x720, white	Amber	
145	Template.Report/definition/pages/pages.json	Report config	Page order and active page	Amber	
146	Template.Report/definition/report.json	Report config	Themes, resources, report settings	Amber	
147	Template.Report/definition/version.json	Report config	PBIR format version (exact)	Red	
148	Template.Report/StaticResources/	Folder	Images and themes	Amber	
149	Template.Report/StaticResources/RegisteredResources/	Folder	Your themes/SVGs (registered)	Amber	
150	Template.Report/StaticResources/RegisteredResources/Theme.deployable.json	Theme	Live PGEColors theme copy	Amber	

#	Path	Type	Purpose	Safety	AI file
151	`Template.Report/StaticResources/SharedResources/`	Folder	Microsoft-supplied resources	Red	
152	`Template.Report/StaticResources/SharedResources/BaseThemes/`	Folder	Base themes	Red	
153	`Template.Report/StaticResources/SharedResources/BaseThemes/CY26SU02.json`	Theme	Microsoft default base theme	Red	
154	`Template.SemanticModel/`	Folder	The data model (TMDL)	Amber	
155	`Template.SemanticModel/.platform`	Fabric metadata	Model type, name, ID for Fabric Git	Red	
156	`Template.SemanticModel/definition/`	Folder	TMDL model files	Amber	
157	`Template.SemanticModel/definition.pbism`	Model pointer	Semantic model format settings	Red	
158	`Template.SemanticModel/definition/cultures/`	Folder	Language settings	Red	
159	`Template.SemanticModel/definition/cultures/en-US.tmdl`	TMDL	English (US) culture	Red	
160	`Template.SemanticModel/definition/database.tmdl`	TMDL	Compatibility level 1600	Red	
161	`Template.SemanticModel/definition/model.tmdl`	TMDL	Model settings; tables get ref’d here	Amber	
162	`Template.SemanticModel/diagramLayout.json`	Layout	Model diagram positions (empty)	Amber	
163	`tests/`	Folder	Test prompts and results	Green	
164	`tests/prompts/`	Folder	Reusable AI test prompts	Amber	Yes
165	`tests/prompts/PBIP_Documentation_Page.md`	AI prompt	Generate a Documentation page	Amber	Yes
166	`tests/prompts/PowerBI_Test.md`	AI prompt	Measure Killer-style + BPA audit	Amber	Yes
167	`tests/prompts/Python_Data_Value_Test.md`	AI prompt	Plan a SQL value checker	Amber	Yes
168	`tests/README.md`	Doc	How to run tests	Green	
169	`tests/results/`	Folder	Test outputs (generated)	Red	
170	`tests/results/history/`	Folder	Timestamped past results	Red	
171	`tests/results/history/.gitkeep`	Placeholder	Keeps empty folder in Git	Red	
172	`tests/results/history/test_results_20260417_124749.md`	Generated	Run at 12:47:49 UTC (skipped)	Red	
173	`tests/results/history/test_results_20260417_124810.md`	Generated	Run at 12:48:10 UTC (skipped)	Red	
174	`tests/results/latest_run.json`	Generated	Last run metadata (skipped)	Red	
175	`tests/results/README.md`	Doc	Explains result files	Green	
176	`tests/results/test_results.md`	Generated	Latest AI test report (skipped)	Red	

Total: 176 items = 122 files + 54 folders. The hidden `.git/` folder (Git’s own database) and the new `guide/` folder are not counted.